학습 교안

PostgreSQL
데이터베이스 기초부터 심화까지

SQL 기초 · JOIN · 윈도우 함수 · 인덱스 · 트랜잭션 · PL/pgSQL · JSONB · 운영 · Python 연동

강사 김대철

작성일 2026.10.23

실습 환경 PostgreSQL 18 (Docker) · DBeaver Community

예제 저장소 https://github.com/sinaboro/postgresql-tutorial

이 교재의 모든 '실행 결과'는 저장소의 예제 SQL 을 PostgreSQL 18 에서 실제로 실행해 넣은 것입니다.

교재 안내

교재 안내 2
이 교재의 구성 8
실습 환경 8
저장소 구조 8
과정 일정 (8일) 9
표기 규칙 9
실습 데이터베이스 (shop) 한눈에 보기 9
01장 · PostgreSQL 시작하기 11
학습 목표 11
데이터베이스와 DBMS 11
왜 PostgreSQL 인가 11
실습 서버 준비 - Docker 로 PostgreSQL 실행 12
접속 도구 - 수업은 DBeaver 로 진행 13
실습 데이터베이스 확인과 초기화 15
예제 15
정리 18
02장 · 테이블 만들기 - 자료형과 제약 조건 19
학습 목표 19
테이블의 기본 문법 19
자주 쓰는 자료형 20
제약 조건 (Constraint) 21
외래 키(FOREIGN KEY)와 참조 무결성 22
구조 변경 - ALTER TABLE 23
데이터 삭제 vs 테이블 삭제 24
실습 문제 25
정리 25
03장 · 데이터 다루기 - INSERT / UPDATE / DELETE 26
학습 목표 26
INSERT 26
UPDATE 와 DELETE 27
shop 데이터 수정하기 29
UPSERT 와 MERGE 29
실습 문제 31

정리 . . . 31
04장 · SELECT 기초 - 원하는 행과 열 골라내기 . . . 32
학습 목표 . . . 32
SELECT 의 기본 구조 . . . 32
WHERE - 행 거르기 . . . 33
NULL - 값이 "없음" . . . 35
정렬과 개수 제한 . . . 36
날짜 조건과 실행 순서 . . . 37
실습 문제 . . . 38
정리 . . . 39
05장 · 내장 함수 - 문자열, 숫자, 날짜, 조건식 . . . 40
학습 목표 . . . 40
문자열 함수 . . . 40
숫자 함수와 형변환 . . . 41
날짜와 시간 . . . 42
조건식 - CASE, COALESCE, NULLIF . . . 44
실습 문제 . . . 46
정리 . . . 46
06장 · 집계와 그룹화 - GROUP BY, HAVING . . . 47
학습 목표 . . . 47
집계 함수 . . . 47
GROUP BY . . . 47
HAVING - 그룹 거르기 . . . 49
조건부 집계와 문자열 집계 . . . 50
[심화] 소계와 총계 - ROLLUP, GROUPING SETS . . . 51
실습 문제 . . . 52
정리 . . . 52
07장 · JOIN - 여러 테이블 연결하기 . . . 53
학습 목표 . . . 53
조인의 원리 . . . 53
INNER JOIN . . . 53
OUTER JOIN . . . 54
여러 테이블 조인과 집계 . . . 55
ON 과 WHERE - LEFT JOIN 의 함정 . . . 56
SELF JOIN, CROSS JOIN, 비등가 조인 . . . 57
실습 문제 . . . 59

정리 . . . 59
08장 · 서브쿼리 - 쿼리 안의 쿼리 . . . 60
학습 목표 . . . 60
서브쿼리의 종류 . . . 60
NOT IN 과 NULL . . . 61
EXISTS 와 상관 서브쿼리 . . . 62
인라인 뷰와 ANY / ALL . . . 62
서브쿼리로 데이터 수정 . . . 63
[심화] LATERAL . . . 64
실습 문제 . . . 65
정리 . . . 65
09장 · 집합 연산과 CTE - 재귀 쿼리까지 . . . 66
학습 목표 . . . 66
집합 연산 . . . 66
CTE (Common Table Expression) . . . 68
재귀 CTE . . . 69
[심화] generate_series 와 데이터 변경 CTE . . . 70
실습 문제 . . . 71
정리 . . . 71
10장 · 윈도우 함수 - 순위, 누적, 이전 행 비교 . . . 72
학습 목표 . . . 72
윈도우 함수란 . . . 72
순위 함수 . . . 72
누적 합과 비중 . . . 74
LAG / LEAD - 이전·다음 행 . . . 74
윈도우 프레임과 이동 평균 . . . 75
실습 문제 . . . 77
정리 . . . 78
11장 · 데이터 모델링과 정규화 . . . 79
학습 목표 . . . 79
모델링 절차 . . . 79
이상 현상 (Anomaly) . . . 80
정규화 . . . 81
관계의 종류와 구현 . . . 82
반정규화 . . . 83
정리 . . . 84

12장 · 뷰, 머티리얼라이즈드 뷰, 시퀀스, 스키마 85

학습 목표 85

뷰 (VIEW) 85

머티리얼라이즈드 뷰 87

시퀀스 88

스키마 89

실습 문제 90

정리 90

13장 · 인덱스와 실행 계획 91

학습 목표 91

인덱스의 원리 91

실행 계획 읽기 92

복합 인덱스 93

인덱스를 쓰지 못하는 경우 94

표현식 인덱스와 부분 인덱스 95

LIKE 와 인덱스 96

조인 계획과 외래 키 인덱스 96

인덱스 관리 97

실습 문제 98

정리 98

14장 · 트랜잭션과 동시성 99

학습 목표 99

트랜잭션과 ACID 99

오류가 나면 트랜잭션 전체가 취소된다 100

MVCC - 읽기와 쓰기가 서로 막지 않는다 102

격리 수준 102

잠금과 갱신 손실 103

데드락 104

실습 문제 105

정리 105

15장 · 함수, 프로시저, 트리거 - PL/pgSQL 106

학습 목표 106

함수의 기본 구조 106

예외 처리와 업무 규칙 108

프로시저 110

트리거 110

실습 문제 . . . 112
정리 . . . 113
16장 · JSONB, 배열, 전문 검색 . . . 114
학습 목표 . . . 114
JSON 과 JSONB . . . 114
GIN 인덱스 . . . 116
배열 . . . 117
전문 검색 (Full Text Search) . . . 118
[확장] pg_trgm - 부분 문자열 검색 . . . 119
실습 문제 . . . 120
정리 . . . 120
17장 · 사용자와 권한 - ROLE, GRANT, 행 수준 보안 . . . 121
학습 목표 . . . 121
ROLE . . . 121
GRANT 와 REVOKE . . . 122
행 수준 보안 (Row Level Security) . . . 124
접속 제어 - pg_hba.conf . . . 126
정리 . . . 126
18장 · 운영 기초 - 모니터링, VACUUM, 백업과 복구 . . . 127
학습 목표 . . . 127
모니터링 기본 쿼리 . . . 127
서버 설정 . . . 128
VACUUM 과 ANALYZE . . . 129
확장 (Extension) . . . 130
백업과 복구 . . . 131
데이터 가져오기 / 내보내기 . . . 132
정리 . . . 133
19장 · Python 으로 PostgreSQL 다루기 . . . 134
학습 목표 . . . 134
환경 준비 . . . 134
접속과 조회 . . . 134
파라미터 바인딩과 SQL 인젝션 . . . 135
INSERT 와 트랜잭션 . . . 136
pandas 로 분석하기 . . . 137
동시성 데모 (14장 보충) . . . 138
정리 . . . 140

20장 · 종합 프로젝트 - 쇼핑몰 매출 분석 리포트 141

프로젝트 개요 141

요구사항 141

1단계 - 분석용 뷰 141

2단계 - KPI 와 성장률 142

3단계 - 고객 분석 (RFM, 재구매율, 코호트) 142

4단계 - 상품과 영업 성과 144

5단계 - 재사용 가능한 리포트 함수 145

조별 과제 146

실습 문제 146

과정 총정리 147

부록 · 빠른 참조 148

SQL 문법 요약 148

자주 만나는 오류 메시지 148

psql 메타 명령 149

이 교재의 구성

이 교재는 **SQL을 처음 배우는 분**을 위해 PostgreSQL 설치부터 실무 수준의 심화 주제(인덱스 튜닝, 트랜잭션과 동시성, PL/pgSQL, JSONB, 권한·보안, 운영, Python 연동)까지 차례로 다룹니다. 각 장은 같은 순서로 구성되어 있습니다.

구성 요소	내용
학습 목표	그 장을 마치면 할 수 있어야 하는 것
개념 설명	문법, 그림, 비교표
예제	저장소의 예제 SQL 과 **실제 실행 결과**. 직접 실행해 결과를 비교해 보세요
핵심 포인트	꼭 기억할 내용, 이해를 돕는 비유
TIP / 주의 / 참고	알아 두면 좋은 기능, 자주 틀리는 부분, 보충 설명
자주 묻는 질문	수업에서 자주 나오는 질문과 답
실습 문제	장별 확인 문제 (문제지 `exercises/`, 정답 `solutions/`)

> ■ 참고 **실행 결과에 대하여**
> 교재의 모든 '실행 결과'는 저장소의 예제 파일을 **PostgreSQL 18**에서 실제로 실행해 넣은 것입니다. 여러분도 같은 Docker 환경과 같은 샘플 데이터를 쓰므로 같은 결과가 나옵니다. 단, 현재 시각(`now()`), 실행 계획의 비용·시간, 객체 크기, 내부 번호(xmin 등)는 실행할 때마다 다릅니다.
> 교재의 결과는 psql 형식(글자로 그린 표)입니다. **DBeaver 에서는 같은 내용이 결과 그리드(표)로** 나오고, `UPDATE 1` 같은 메시지는 Statistics 탭에 표시됩니다.

실습 환경

구성	도구	설명
데이터베이스 서버	**Docker** + PostgreSQL 18	`docker compose up -d` 한 번으로 서버와 실습 DB(shop)가 준비됩니다 (1장)
SQL 실행 도구	**DBeaver Community**	모든 실습을 DBeaver 의 SQL 편집기에서 합니다 (1장)
보조 도구	psql (컨테이너 안에 포함)	18장 백업·CSV 실습에서 사용
프로그래밍	Python 3.10+	19장 Python 연동

저장소 구조

예제 저장소: **https://github.com/sinaboro/postgresql-tutorial**

경로	내용
`docker-compose.yml`	PostgreSQL 18 실습 서버 설정
`setup/`	실습 DB 스키마(`01_schema.sql`)와 샘플 데이터(`02_seed.sql`)
`examples/`	장별 예제 SQL (`01_intro.sql` ~ `20_project.sql`). 교재의 '예제 NN-MM' 번호와 같습니다
`exercises/`	실습 문제지
`solutions/`	실습 문제 정답 (먼저 직접 풀어 본 뒤 확인하세요)
`python/`	19장 Python 연동 예제
`work/`	Docker 컨테이너와 파일(백업, CSV)을 주고받는 폴더

과정 일정 (8일)

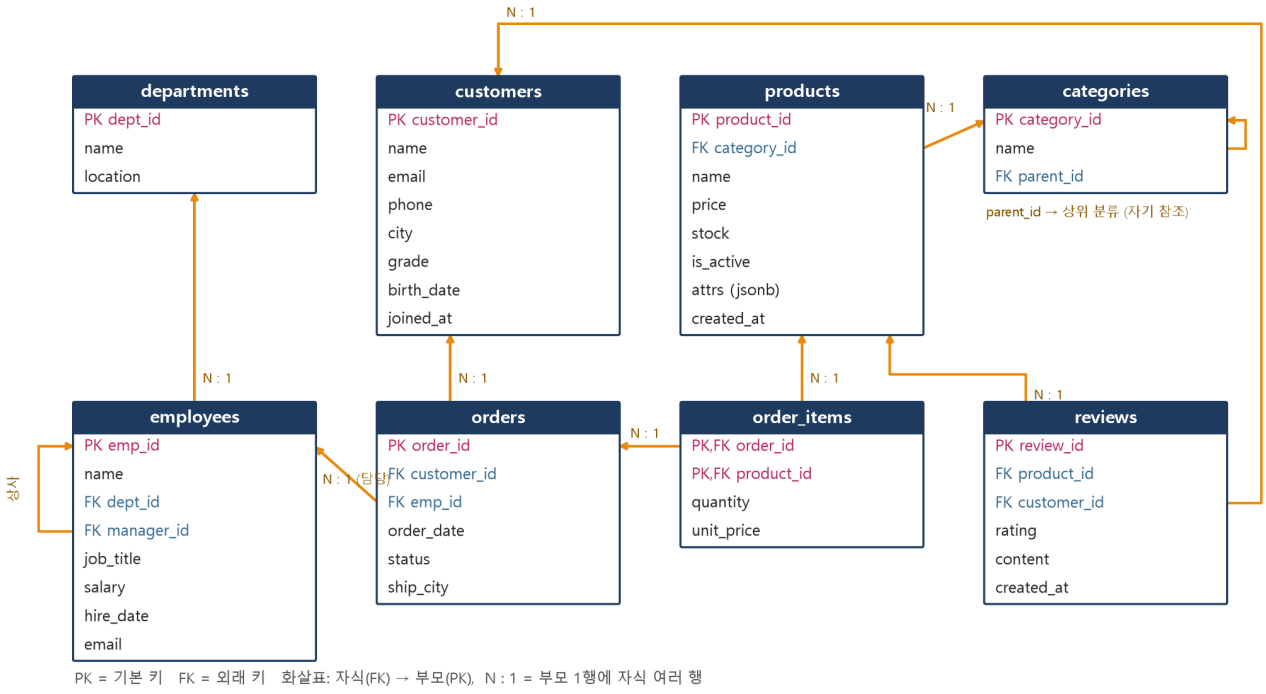
일차	장	주제	수준
1일	01 ~ 03	환경 준비(Docker, DBeaver), 테이블 만들기(DDL), 데이터 다루기(DML)	기초
2일	04 ~ 06	SELECT 기초, 내장 함수, 집계와 GROUP BY	기초
3일	06 ~ 08	고급 그룹핑, JOIN, 서브쿼리	기초 → 중급
4일	09 ~ 10	집합 연산·CTE·재귀 쿼리, 윈도우 함수	중급
5일	11 ~ 13	데이터 모델링·정규화, 뷰·시퀀스·스키마, 인덱스와 실행 계획	중급
6일	13 ~ 15	인덱스(계속), 트랜잭션과 동시성, PL/pgSQL·트리거	심화
7일	16 ~ 19	JSONB·배열·전문검색, 권한·RLS, 운영 기초, Python 연동	심화
8일	19 ~ 20	Python 연동(계속), 종합 프로젝트(매출 분석 리포트)	종합

표기 규칙

- SQL 키워드는 대문자(`SELECT`), 테이블·컬럼 이름은 소문자(`customers`)로 씁니다. PostgreSQL 은 대소문자를 구분하지 않지만 읽기 쉽게 하기 위한 관례입니다.
- `psql` 에서 `\` 로 시작하는 명령(`\dt`, `\d`)은 SQL 이 아니라 psql 프로그램의 **메타 명령**이라 DBeaver 에서는 동작하지 않습니다. 그래서 예제는 모두 SQL 로 작성했습니다(18장 `\copy` 만 psql 전용).
- `[심화]` : 처음 공부할 때는 건너뛰어도 되는 내용, `[error]` : 일부러 오류를 보여 주는 예제입니다.
- 금액 단위는 원, 직원 연봉(`salary`)은 만원 단위입니다.

실습 데이터베이스 (shop) 한눈에 보기

모든 장은 하나의 가상 쇼핑몰 데이터베이스 `shop` 을 사용합니다. 고객이 상품을 주문하고, 직원이 주문을 담당하고, 고객이 리뷰를 남기는 구조입니다.



테이블	행 수	설명	특징
departments	5	부서	직원이 없는 조건으로 RIGHT/FULL JOIN 실습
employees	15	직원	manager_id 자기참조 → SELF JOIN, 재귀 CTE. 부서 없는 인턴 1명
customers	20	고객	전화번호·생일 NULL 존재, 주문 없는 고객 2명(19, 20번)
categories	11	상품 분류	3단계 트리(전자제품 > 노트북 > 게이밍 노트북)
products	20	상품	attrs JSONB 컬럼, 한 번도 안 팔린 상품 2개(19, 20번), 재고 0 상품
orders	120	주문 헤더	2025-01 ~ 2026-06, 상태 5종, 온라인 주문은 담당 직원 NULL
order_items	237	주문 상세	복합 PK (order_id, product_id), 주문 당시 가격 unit_price
reviews	62	상품 리뷰	평점 1~5, 내용 NULL 존재

01장 · PostgreSQL 시작하기

학습 목표

- 데이터베이스, DBMS, 관계형 데이터베이스, SQL 이 무엇인지 설명할 수 있다.
- Docker 로 PostgreSQL 서버를 실행하고 DBeaver 로 연결할 수 있다.
- DBeaver SQL 편집기에서 쿼리와 스크립트를 실행하고, psql 메타 명령과의 차이를 설명할 수 있다.
- 실습용 `shop` 데이터베이스를 확인하고 처음 상태로 되돌릴 수 있다.

데이터베이스와 DBMS

데이터베이스(Database) 는 여러 사람이 함께 쓰기 위해 체계적으로 모아 둔 데이터의 집합이고, 이를 저장·검색·수정·보호하는 소프트웨어가 **DBMS(Database Management System)** 입니다. PostgreSQL, MySQL, Oracle, SQL Server 가 대표적인 DBMS 입니다.

비교	엑셀 파일	데이터베이스(DBMS)
동시 사용	한 사람이 열면 다른 사람은 읽기 전용	수천 명이 동시에 읽고 써도 일관성 유지
데이터 규칙	아무 값이나 입력 가능	자료형·제약 조건으로 잘못된 값 차단
데이터 양	약 100만 행 한계, 느려짐	수억 행도 인덱스로 빠르게 검색
장애 대응	파일이 깨지면 끝	트랜잭션·로그·백업으로 복구
권한	파일 단위	테이블·컬럼·행 단위 권한

관계형 데이터베이스(RDB) 는 데이터를 행(row)과 열(column)로 된 **테이블**에 저장하고, 테이블끼리는 **키(key)** 로 관계를 맺습니다. 테이블에 질문하는 표준 언어가 **SQL(Structured Query Language)** 입니다.

SQL 분류	의미	주요 명령	교재
DDL (Definition)	구조 정의	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	2장
DML (Manipulation)	데이터 조작	`INSERT`, `UPDATE`, `DELETE`, `MERGE`	3장
DQL (Query)	데이터 조회	`SELECT`	4~10장
DCL (Control)	권한 제어	`GRANT`, `REVOKE`	17장
TCL (Transaction)	트랜잭션 제어	`BEGIN`, `COMMIT`, `ROLLBACK`	14장

왜 PostgreSQL 인가

- **완전한 오픈소스**(PostgreSQL License): 상용 서비스에도 무료로 쓸 수 있습니다.
- **표준 SQL 준수도가 높고 기능이 풍부**: 윈도우 함수, CTE, JSONB, 배열, 전문 검색, 사용자 정의 타입·함수, 확장(Extension).
- **신뢰성**: MVCC 기반 동시성 제어, 강력한 트랜잭션, 30년 넘게 검증된 엔진.
- **시장**: Stack Overflow 개발자 설문에서 수년째 가장 많이 쓰고 선호하는 데이터베이스로 꼽히며, AWS RDS/Aurora, Google Cloud SQL, Azure, Supabase 등 대부분의 클라우드가 기본 제공합니다.
- **버전 정책**: 매년 가을 메이저 버전 출시, 각 버전 5년 지원. 이 교재은 **PostgreSQL 18** 기준이며 17 이상이면 모든 예제가 동작합니다(13장 skip scan 설명만 18 전용).

실습 서버 준비 - Docker 로 PostgreSQL 실행

PostgreSQL 을 PC 에 직접 설치하지 않고 **Docker 컨테이너**로 실행합니다. 모든 사람이 같은 버전·같은 설정·같은 데이터로 실습할 수 있고, 망가져도 명령 한 줄로 처음 상태로 되돌릴 수 있습니다.

용어	의미
이미지(image)	프로그램 설치가 끝난 상태를 찍어 둔 틀 (`postgres:18`)
컨테이너(container)	이미지로 실행한 프로그램 하나 (`pg-shop`)
볼륨(volume)	컨테이너를 지워도 남는 데이터 저장소 (`pgdata`)
docker compose	`docker-compose.yml` 에 적은 설정대로 컨테이너를 실행하는 명령

1단계 - Docker Desktop 설치

1. https://www.docker.com/products/docker-desktop/ 에서 **Docker Desktop** 을 받아 설치합니다. (Windows 는 설치 중 WSL 2 사용을 권장하는 기본값 그대로)
2. 설치 후 재부팅하고 Docker Desktop 을 실행합니다. 화면 왼쪽 아래가 **Engine running** 이면 준비된 것입니다.
3. 명령 프롬프트(또는 PowerShell, 터미널)에서 확인합니다.

```
docker --version
docker compose version
```

2단계 - 저장소 받기와 서버 시작

```
git clone https://github.com/sinaboro/postgresql-tutorial.git
cd postgresql-tutorial
docker compose up -d
docker compose ps
```

git 이 없으면 GitHub 페이지의 **Code → Download ZIP** 으로 받아 압축을 풀고, 그 폴더에서 `docker compose up -d` 를 실행합니다. 처음에는 `postgres:18` 이미지를 내려받느라 1~2분 걸립니다. `docker compose ps` 의 STATUS 가 **healthy** 가 되면 준비 완료입니다.

처음 시작할 때 `setup/01_schema.sql`, `setup/02_seed.sql` 이 자동으로 실행되어 **shop 데이터베이스와 샘플 데이터가 이미 만들어져 있습니다.**

항목	값
호스트 / 포트	`localhost` / `5432`
데이터베이스	`shop`
사용자 / 비밀번호	`postgres` / `postgres`

자주 쓰는 Docker 명령

명령	하는 일
`docker compose up -d`	서버 시작 (처음이면 생성 + 샘플 데이터 입력)
`docker compose stop`	서버 중지 (데이터는 유지)
`docker compose ps`	상태 확인
`docker compose logs -f`	서버 로그 보기 (오류 원인 확인, Ctrl+C 로 빠져나옴)

명령	하는 일
`docker exec -it pg-shop psql -U postgres -d shop`	컨테이너 안의 psql 로 접속 (`\q` 로 종료)
`docker compose down -v`	컨테이너와 데이터 볼륨 삭제 → 다음 `up` 때 shop 을 처음 상태로 다시 생성

■ 주의 **자주 생기는 문제**

- `docker` **명령을 찾을 수 없음 / 연결 오류**: Docker Desktop 이 실행 중인지 확인합니다. 작업 표시줄의 고래 아이콘이 초록색(running)이어야 합니다.
- **5432 포트 사용 중(port is already allocated)**: PC 에 PostgreSQL 이 따로 설치되어 있는 경우입니다. `docker-compose.yml` 의 `"5432:5432"` 를 `"5433:5432"` 로 바꾸고 다시 `docker compose up -d` 한 뒤, DBeaver 에서 포트를 5433 으로 씁니다.
- **샘플 데이터가 없음**: 샘플 데이터는 볼륨이 **처음 만들어질 때만** 들어갑니다. `docker compose down -v` 후 `docker compose up -d` 로 다시 만듭니다.
- **회사·학교 PC 에서 WSL 2 / 가상화 오류**: BIOS 의 가상화(VT-x / AMD-V) 설정과 Windows 기능의 "Linux용 Windows 하위 시스템"이 켜져 있어야 합니다.

접속 도구 - 수업은 DBeaver 로 진행

도구	특징	이 과정에서
DBeaver (Community, 무료)	여러 DBMS 를 지원하는 범용 GUI. 자동완성, 결과 그리드, ER 다이어그램, 데이터 가져오기/내보내기	**수업 표준 도구**. 모든 실습을 DBeaver 로 진행
psql	PostgreSQL 공식 명령줄 도구 (Docker 컨테이너 안에 포함)	18장 `\copy`·백업 명령
pgAdmin 4	공식 웹 GUI	이 과정에서는 사용하지 않음

DBeaver 설치

1. https://dbeaver.io/download/ → **DBeaver Community** 의 Windows (installer) 를 받아 설치합니다. (macOS 는 dmg, Linux 는 deb/rpm)
2. 첫 실행 시 "샘플 데이터베이스를 만들까요?" 질문은 **아니요**, 통계 전송 동의는 원하는 대로 선택합니다.
3. 메뉴가 영어라면 그대로 쓰는 것을 권장합니다(검색·질문할 때 영어 메뉴 이름이 편함). 교재도 영어 메뉴 이름으로 적습니다.

PostgreSQL 연결 만들기

1. 왼쪽 위 **New Database Connection**(플러그 + 아이콘) 또는 메뉴 `Database → New Database Connection` → **PostgreSQL** 선택 → Next
2. Main 탭 입력

항목	값
Connect by	Host
Host / Port	`localhost` / `5432`
Database	`shop`
Authentication	Database Native
Username / Password	`postgres` / `postgres`, **Save password** 체크

3. PostgreSQL 탭에서 **Show all databases** 를 체크하면 왼쪽 트리에 모든 DB 가 보입니다.
4. **Test Connection** → 처음이면 "Download driver files" 창이 뜨고 **Download** 를 누르면 JDBC 드라이버를 받습니다. `Connected` 가 나오면 Finish.

> ■ 참고 **드라이버 다운로드**
> DBeaver 는 첫 연결 때 PostgreSQL JDBC 드라이버를 인터넷에서 내려받습니다. 다운로드 창이 뜨지 않거나 실패하면 인터넷 연결을 확인하고, `Database → Driver Manager → PostgreSQL → Download/Update` 로 다시 받습니다.

SQL 편집기 사용법

연결을 선택하고 `Ctrl+]` (또는 메뉴 `SQL Editor → New SQL script`) 로 편집기를 엽니다. 저장소의 예제 파일은 `File → Open File...` 로 열고, 편집기 위쪽의 **활성 연결/데이터베이스 선택 상자**에서 `shop` 을 고릅니다.

단축키	동작	언제
`Ctrl+Enter`	커서가 있는 **문장 하나** 실행 (선택했으면 선택 영역을 한 문장으로 실행)	쿼리 하나씩 실행
`Alt+X`	**스크립트 실행** - 여러 문장을 차례로 실행, 결과는 탭마다	예제 블록 전체, setup 파일
`Ctrl+Space`	자동완성 (테이블·컬럼 이름)	항상
`Ctrl+Shift+F`	SQL 정렬(포맷)	긴 쿼리 정리
`Ctrl+/`	줄 주석 켜기/끄기	
`Ctrl+Shift+E`	실행 계획(EXPLAIN) 보기	13장
`Ctrl+Shift+O`	서버 출력(NOTICE) 창 켜기/끄기	15장 `RAISE NOTICE`

> ■ 핵심 포인트 **예제 파일 실행 요령**
> • 예제 파일은 `-- @ex 04-01 제목` 주석으로 예제가 나뉩니다. **예제 블록 전체를 마우스로 선택** → `Alt+X` 가 가장 확실합니다. 선택하지 않고 `Alt+X` 를 누르면 파일 전체가 실행되니 주의하세요.
> • DBeaver 는 기본 설정에서 **빈 줄을 문장 구분으로 볼 수 있어**, 빈 줄이 들어간 긴 문장에 `Ctrl+Enter` 를 쓰면 일부만 실행됩니다. 문장은 `;` 로 확실히 끝내고, 헷갈리면 선택 후 실행하세요. (설정: `Window → Preferences → Editors → SQL Editor → SQL Processing → Blank line is statement delimiter`)
> • 결과는 아래 **결과 그리드**에 표로 나옵니다. INSERT/UPDATE 결과(`UPDATE 1`)는 **Statistics** 탭의 Updated Rows 로 확인합니다. 교재의 '실행 결과'는 psql 형식이라 모양만 다를 뿐 내용은 같습니다.
> • 오류가 나면 아래쪽에 빨간 메시지가 뜹니다. 트랜잭션 모드(14장)에서 오류가 났다면 Rollback 을 눌러야 다음 실행이 됩니다.

psql 메타 명령과 DBeaver 대응표

`\` 로 시작하는 psql 메타 명령은 SQL 이 아니어서 **DBeaver 에서는 실행되지 않습니다**. 그래서 이 저장소의 예제는 메타 명령 대신 같은 정보를 SQL(`information_schema`)로 조회하도록 작성했습니다(예: 01-03, 01-04). 18장의 `\copy` 만 psql 전용입니다.

psql	의미	DBeaver 에서는
`\l` / `\c shop`	DB 목록 / DB 전환	왼쪽 Database Navigator, 편집기 위 데이터베이스 선택 상자
`\dt` / `\dv`	테이블 / 뷰 목록	Navigator 의 `shop → Schemas → public → Tables / Views`
`\d 테이블`	테이블 구조	테이블 더블클릭 → **Properties**(Columns, Constraints, Indexes) / **DDL** 탭
`\df`	함수 목록	Navigator 의 `Functions` (Procedures 포함)
`\du`	롤 목록	Navigator 의 연결 아래 `Roles`

psql	의미	DBeaver 에서는
`\i 파일`	파일 실행	파일 열기 → `Alt+X`
`\timing`	실행 시간	결과 그리드 아래 상태 표시줄에 자동 표시
`\x`	세로 출력	결과 그리드에서 `Tab` 키 (Record 보기 전환)
`\copy`	CSV 입출력	테이블 우클릭 → Export Data / Import Data (18장)

실습 데이터베이스 확인과 초기화

Docker 로 서버를 시작하면 shop DB 가 이미 준비되어 있습니다. DBeaver 에서 연결한 뒤 새 편집기에 다음을 실행해 **120** 이 나오면 준비 완료입니다.

```
SELECT count(*) FROM orders;
```

■ TIP 실습 데이터 초기화 - 두 가지 방법

3장처럼 데이터를 바꾸는 실습 뒤에는 처음 상태로 되돌린 다음 다음 장으로 넘어가세요.

- **DBeaver 에서 (빠름)**: `setup/01_schema.sql` 을 열고 `Alt+X`, 이어서 `setup/02_seed.sql` 을 열고 `Alt+X`. 테이블을 지우고 다시 만듭니다.
- **Docker 로 (완전 초기화)**: `docker compose down -v` → `docker compose up -d`. 17장에서 만든 사용자(ROLE)까지 모두 지워집니다. 이때 DBeaver 연결은 한 번 끊었다가(Disconnect) 다시 연결합니다.

예제

접속한 서버와 사용자, 데이터베이스를 확인합니다. 문제가 생기면 가장 먼저 확인할 정보입니다.

예제 01-01 · 서버 버전과 접속 정보 확인

```
SHOW server_version;
SELECT current_database(), current_user;
```

▶ 실행 결과

```
 server_version
----------------
 18.6
(1 row)

 current_database ¦ current_user
------------------+--------------
 shop             ¦ postgres
(1 row)
```

SQL 의 `SELECT` 는 테이블 없이 계산기로도 쓸 수 있습니다. **정수끼리 나누면 몫만** 남는다는 점(5장에서 다시 다룸)을 여기서 먼저 보여 줍니다.

예제 01-02 · SQL 은 계산기처럼도 쓸 수 있다

```
SELECT 1 + 2 AS 더하기,
       10 / 3 AS 정수나눗셈,
       10 / 3.0 AS 실수나눗셈,
       2 ^ 10 AS 거듭제곱,
       'Postgre' || 'SQL' AS 문자열연결;
```

▶ 실행 결과

```
 더하기 ¦ 정수나눗셈 ¦     실수나눗셈     ¦ 거듭제곱 ¦ 문자열연결
--------+------------+--------------------+----------+------------
      3 ¦          3 ¦ 3.3333333333333333 ¦     1024 ¦ PostgreSQL
(1 row)
```

실습 DB 의 테이블 목록과 구조를 SQL 로 조회합니다. psql 의 `\dt`, `\d departments` 와 같은 정보이며, DBeaver 에서는 Navigator 에서 테이블을 더블클릭해 Properties / DDL 탭으로도 볼 수 있습니다. 두 방법 모두 해 보세요.

예제 01-03 · 테이블 목록 보기 (psql 에서는 \dt)

```
-- DBeaver: 왼쪽 Database Navigator 에서 shop > Schemas > public > Tables
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

▶ 실행 결과

```
 table_name
-------------
 categories
 customers
 departments
 employees
 order_items
 orders
 products
 reviews
(8 rows)
```

예제 01-04 · 테이블 구조 보기 (psql 에서는 \d departments)

```
-- DBeaver: 테이블을 더블클릭 → Properties(컬럼) / DDL 탭
SELECT column_name, data_type, character_maximum_length AS max_len,
       is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'departments'
ORDER BY ordinal_position;
```

▶ 실행 결과

```
 column_name ¦     data_type     ¦ max_len ¦ is_nullable ¦ column_default
-------------+-------------------+---------+-------------+----------------
 dept_id     ¦ integer           ¦         ¦ NO          ¦
 name        ¦ character varying ¦      50 ¦ NO          ¦
 location    ¦ character varying ¦      50 ¦ YES         ¦
(3 rows)
```

첫 번째 SELECT 입니다. `*` 는 "모든 컬럼" 을 뜻합니다.

예제 01-05 · 첫 번째 SELECT - 부서 테이블 전체 조회

```
SELECT * FROM departments;
```

▶ 실행 결과

```
 dept_id ┆   name   ┆ location
---------+----------+----------
       1 ┆ 경영지원 ┆ 서울
       2 ┆ 영업     ┆ 서울
       3 ┆ 마케팅   ┆ 서울
       4 ┆ 개발     ┆ 판교
       5 ┆ 물류     ┆ 인천
(5 rows)
```

SQL 키워드와 테이블·컬럼 이름은 대소문자를 구분하지 않지만, **따옴표 안의 문자열 값은 구분**합니다.

예제 01-06 · SQL 은 대소문자를 구분하지 않는다 (문자열 값은 구분)

```
select NAME, Location from DEPARTMENTS where location = '서울';
SELECT name FROM departments WHERE location = 'SEOUL';
```

▶ 실행 결과

```
   name   ┆ location
----------+----------
 경영지원 ┆ 서울
 영업     ┆ 서울
 마케팅   ┆ 서울
(3 rows)

 name
------
(0 rows)
```

예제 01-07 · 주석 작성법

```
-- 한 줄 주석은 하이픈 두 개
/* 여러 줄 주석은
   C 언어 스타일 */
SELECT name /* 이름만 */ FROM departments WHERE dept_id = 1;
```

▶ 실행 결과

```
   name
----------
 경영지원
(1 row)
```

■ Q&A; 자주 묻는 질문

- **Q. 세미콜론(;)을 꼭 붙여야 하나요?** psql 은 `;` 를 만나야 실행합니다. 붙이지 않으면 프롬프트가 `shop-#` 으로 바뀌며 다음 줄 입력을 기다립니다. GUI 에서는 한 문장만 실행할 때 생략해도 되지만 습관적으로 붙이세요.
- **Q. 데이터베이스와 스키마는 뭐가 다른가요?** DB 는 독립된 저장소(다른 DB 의 테이블을 바로 조인할 수 없음), 스키마는 DB 안의 폴더입니다. 12장에서 다룹니다.
- **Q. 대문자로 만든 테이블 이름은?** `CREATE TABLE "Users"` 처럼 큰따옴표로 만들면 그 뒤로도 항상 `"Users"` 로 써야 합니다. 이름은 소문자 + 밑줄(snake_case)로 짓는 것이 PostgreSQL 관례입니다.

정리

- DBMS 는 데이터를 안전하게, 동시에, 빠르게 다루기 위한 소프트웨어이며 SQL 로 대화한다.
- 실습은 DBeaver 로: 블록 선택 후 `Alt+X`, 한 문장은 `Ctrl+Enter`.
- psql 메타 명령(`\dt`, `\d`)은 SQL 이 아니라 psql 의 기능이라 DBeaver 에서는 동작하지 않는다.
- 실습 DB 는 `setup/` 의 파일 3개로 언제든 초기 상태로 되돌릴 수 있다.

02장 · 테이블 만들기 - 자료형과 제약 조건

학습 목표

- `CREATE TABLE` 로 테이블을 만들고 `\d` 로 구조를 확인할 수 있다.
- 숫자·문자·날짜·논리 자료형의 차이를 알고 알맞게 고를 수 있다.
- PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY 제약 조건을 설정할 수 있다.
- `ALTER TABLE` 로 구조를 바꾸고 `TRUNCATE` 와 `DROP` 의 차이를 설명할 수 있다.

예제 파일: `examples/02_ddl.sql`

테이블의 기본 문법

```
CREATE TABLE 테이블명 (
    컬럼명  자료형  [제약 조건],
    컬럼명  자료형  [제약 조건],
    ...
    [테이블 제약 조건]
);
```

예제 02-01 · 가장 단순한 테이블 만들기

```
CREATE TABLE notices (
    id      INTEGER,
    title   VARCHAR(100),
    content TEXT
);
-- 구조 확인 (psql 에서는 \d notices)
SELECT column_name, data_type, character_maximum_length AS max_len,
       is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'notices'
ORDER BY ordinal_position;
```

▶ 실행 결과

```
CREATE TABLE
 column_name ┆     data_type     ┆ max_len ┆ is_nullable ┆ column_default
-------------+-------------------+---------+-------------+----------------
 id          ┆ integer           ┆         ┆ YES         ┆
 title       ┆ character varying ┆     100 ┆ YES         ┆
 content     ┆ text              ┆         ┆ YES         ┆
(3 rows)
```

> ■ TIP **DBeaver 에서 테이블 구조 보기**
> 예제에서는 `information_schema.columns` 로 구조를 조회했습니다(psql 의 `\d notices` 와 같은 정보). DBeaver 에서는 왼쪽 Navigator 에서 테이블을 더블클릭하면 **Properties** 탭(Columns, Constraints, Foreign Keys, Indexes)과 **DDL** 탭(CREATE TABLE 문)으로 더 자세히 볼 수 있습니다. 새로 만든 테이블이 보이지 않으면 Navigator 에서 `F5`(Refresh)를 누릅니다. 테이블 생성은 GUI 로도 가능하지만, 이 과정에서는 SQL 로 작성합니다.

자주 쓰는 자료형

분류	자료형	설명	사용 예
정수	SMALLINT / INTEGER / BIGINT	2 / 4 / 8바이트 정수	수량, ID (큰 테이블은 BIGINT)
정확한 소수	NUMERIC(p, s)	전체 p자리, 소수 s자리. 오차 없음	**금액**, 환율, 비율
근사 실수	REAL / DOUBLE PRECISION	부동소수점, 계산 오차 있음	과학 측정값, 좌표
문자	VARCHAR(n) / TEXT / CHAR(n)	가변(최대 n) / 무제한 / 고정 길이	이름, 본문
날짜·시간	DATE / TIMESTAMP / TIMESTAMPTZ	날짜 / 날짜+시간 / 시간대 포함	생일 / 주문일시 / 로그
기간	INTERVAL	시간 간격	'3 days', '2 hours'
논리	BOOLEAN	true / false / NULL	사용 여부
기타	UUID, JSONB, 배열(TEXT[])	고유 ID, 반정형 데이터, 목록	16장

예제 02-02 · 자주 쓰는 자료형 맛보기

```
CREATE TABLE typetest (
    a_int     INTEGER,          -- 정수 (약 ±21억)
    a_big     BIGINT,           -- 큰 정수
    a_num     NUMERIC(8,2),     -- 정확한 소수 (전체 8자리, 소수 2자리)
    a_real    DOUBLE PRECISION, -- 근사 실수
    a_char    CHAR(5),          -- 고정 길이 문자
    a_varchar VARCHAR(10),      -- 가변 길이 문자 (최대 10자)
    a_text    TEXT,             -- 길이 제한 없는 문자
    a_bool    BOOLEAN,
    a_date    DATE,
    a_ts      TIMESTAMP,
    a_tstz    TIMESTAMPTZ       -- 시간대 포함 타임스탬프
);
INSERT INTO typetest VALUES
    (2147483647, 9223372036854775807, 123456.789, 0.1 + 0.2,
     'ab', '가나다', '아주 긴 글도 저장 가능', true,
     '2026-10-23', '2026-10-23 14:30:00', '2026-10-23 14:30:00+09');
SELECT a_num, a_real, '[' || a_char || ']' AS a_char, length(a_char) AS char_len, a_bool, a_tstz
FROM typetest;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 1
   a_num   | a_real | a_char | char_len | a_bool |         a_tstz
-----------+--------+--------+----------+--------+------------------------
 123456.79 |    0.3 | [ab]   |        2 | t      | 2026-10-23 14:30:00+09
(1 row)
```

예제 02-03 · 범위를 벗어난 값은 오류

```
INSERT INTO typetest (a_int) VALUES (2147483648);
INSERT INTO typetest (a_varchar) VALUES ('열한글자를넘는긴문자열');
```

▶ 실행 결과

```
ERROR:  integer out of range
ERROR:  value too long for type character varying(10)
```

■ 핵심 포인트 자료형 고르는 요령

- `VARCHAR(n)` 과 `TEXT` 는 PostgreSQL 에서 **성능 차이가 없습니다.** 길이 제한이 업무 규칙일 때만 `VARCHAR(n)` 을 쓰면 됩니다.
- `CHAR(n)` 은 남는 자리를 공백으로 채우므로 실무에서 거의 쓰지 않습니다. 예제의 `char_len` 결과(2)와 `[ab]` 처럼 비교·연결할 때 뒤 공백이 무시되어 오히려 헷갈립니다.
- `TIMESTAMPTZ` 는 저장할 때 UTC 로 바꿔 두고, 보여 줄 때 세션의 timezone(실습 서버는 Asia/Seoul)으로 변환합니다. 여러 나라 사용자가 있는 서비스라면 TIMESTAMPTZ 를 권장합니다.

금액은 반드시 NUMERIC

`0.1 + 0.2` 는 이진 부동소수점으로 정확히 표현되지 않아 `0.3` 과 다릅니다. 돈 계산에 `REAL`/`DOUBLE PRECISION` 을 쓰면 원 단위 오차가 누적됩니다.

예제 02-04 · NUMERIC 과 실수형의 차이 (돈 계산은 NUMERIC!)

```
SELECT 0.1::double precision + 0.2::double precision = 0.3 AS 실수비교,
       0.1::numeric + 0.2::numeric = 0.3 AS 숫자비교;
```

▶ 실행 결과

```
 실수비교 ¦ 숫자비교
----------+----------
 f        ¦ t
(1 row)
```

제약 조건 (Constraint)

제약 조건은 **잘못된 데이터가 들어오는 것을 DB 가 막아 주는 규칙**입니다. 애플리케이션 코드에서 검사하더라도 DB 에 한 번 더 걸어 두는 것이 원칙입니다(코드는 여러 개, DB 는 하나).

제약 조건	의미	위반 시
`PRIMARY KEY`	행을 유일하게 식별 (= UNIQUE + NOT NULL), 테이블당 1개	duplicate key 오류
`NOT NULL`	값 필수	null value 오류
`UNIQUE`	중복 불가 (NULL 은 여러 개 허용)	duplicate key 오류
`CHECK (조건)`	조건식이 참이어야 함	check constraint 오류
`DEFAULT 값`	값을 생략하면 넣을 기본값	(제약이라기보다 기본값)
`REFERENCES 부모(컬럼)`	외래 키, 부모 테이블에 존재하는 값만 허용	foreign key 오류

예제 02-05 · 제약 조건을 갖춘 테이블 - PK, NOT NULL, UNIQUE, CHECK, DEFAULT

```
CREATE TABLE members (
    member_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email      VARCHAR(100) NOT NULL UNIQUE,
    name       VARCHAR(50)  NOT NULL,
    age        INTEGER CHECK (age BETWEEN 0 AND 150),
    level      VARCHAR(10)  NOT NULL DEFAULT 'NORMAL',
    created_at TIMESTAMP    NOT NULL DEFAULT now()
);
INSERT INTO members (email, name, age) VALUES ('a@test.com', '홍길동', 30);
SELECT member_id, email, name, age, level FROM members;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 1
 member_id ¦   email    ¦  name  ¦ age ¦ level
-----------+------------+--------+-----+--------
```

```
        1 ¦ a@test.com ¦ 홍길동 ¦  30 ¦ NORMAL
(1 row)
```

예제 02-06 · 제약 조건 위반 사례

```
INSERT INTO members (email, name, age) VALUES ('a@test.com', '중복이메일', 20);  -- UNIQUE
INSERT INTO members (email, age) VALUES ('b@test.com', 25);                  -- NOT NULL
INSERT INTO members (email, name, age) VALUES ('c@test.com', '나이오류', 200); -- CHECK
```

▶ 실행 결과

```
ERROR:  duplicate key value violates unique constraint "members_email_key"
DETAIL:  Key (email)=(a@test.com) already exists.
ERROR:  null value in column "name" of relation "members" violates not-null constraint
DETAIL:  Failing row contains (3, b@test.com, null, 25, NORMAL, 2026-10-23 18:00:29.20851).
ERROR:  new row for relation "members" violates check constraint "members_age_check"
DETAIL:  Failing row contains (4, c@test.com, 나이오류, 200, NORMAL, 2026-10-23 18:00:29.209003).
```

■ 참고 **IDENTITY - 자동 증가 번호**

`GENERATED ALWAYS AS IDENTITY` 는 SQL 표준 방식의 자동 증가 컬럼입니다. 예전 방식인 `SERIAL` 보다 권장됩니다(12장에서 차이를 실습). `ALWAYS` 는 사용자가 값을 직접 넣지 못하게 막고, `BY DEFAULT` 는 직접 넣는 것도 허용합니다. 실습 DB 는 샘플 데이터를 번호까지 지정해서 넣기 위해 `BY DEFAULT` 를 씁니다.

외래 키(FOREIGN KEY)와 참조 무결성

학생-수강-과목처럼 **N:M 관계**는 가운데에 연결 테이블(`enrollments`)을 두고 양쪽을 외래 키로 참조합니다. 연결 테이블의 기본 키는 두 외래 키를 합친 **복합 기본 키**가 자연스럽습니다.

예제 02-07 · 외래 키(FOREIGN KEY)로 테이블 연결

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    name       VARCHAR(50) NOT NULL
);
CREATE TABLE courses (
    course_id  INTEGER PRIMARY KEY,
    title      VARCHAR(100) NOT NULL
);
CREATE TABLE enrollments (
    student_id INTEGER REFERENCES students (student_id) ON DELETE CASCADE,
    course_id  INTEGER REFERENCES courses (course_id),
    enrolled_on DATE NOT NULL DEFAULT current_date,
    PRIMARY KEY (student_id, course_id)          -- 복합 기본 키
);
INSERT INTO students VALUES (1, '김학생'), (2, '이학생');
INSERT INTO courses  VALUES (10, 'PostgreSQL 기초'), (20, '파이썬');
INSERT INTO enrollments (student_id, course_id) VALUES (1, 10), (1, 20), (2, 10);
SELECT student_id, course_id FROM enrollments ORDER BY 1, 2;
```

▶ 실행 결과

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 2
INSERT 0 3
 student_id ¦ course_id
------------+-----------
          1 ¦        10
          1 ¦        20
          2 ¦        10
(3 rows)
```

예제 02-08 · 존재하지 않는 부모를 참조하면 오류

```
INSERT INTO enrollments (student_id, course_id) VALUES (3, 10);
```

▶ 실행 결과

```
ERROR:  insert or update on table "enrollments" violates foreign key constraint "enrollments_student_id_fkey"
DETAIL:  Key (student_id)=(3) is not present in table "students".
```

부모 행을 지울 때의 동작은 `ON DELETE` 옵션으로 정합니다.

옵션	부모 삭제 시 자식은	쓰임새
`NO ACTION` / `RESTRICT` (기본)	자식이 있으면 삭제 거부	대부분의 경우 (실수 방지)
`CASCADE`	자식도 함께 삭제	주문 → 주문상세처럼 부모 없이는 의미 없는 자식
`SET NULL`	자식의 FK 를 NULL 로	담당자 퇴사 시 담당자 비우기
`SET DEFAULT`	자식의 FK 를 기본값으로	드묾

예제 02-09 · ON DELETE CASCADE - 부모를 지우면 자식도 함께 삭제

```
DELETE FROM students WHERE student_id = 1;
SELECT student_id, course_id FROM enrollments;
```

▶ 실행 결과

```
DELETE 1
 student_id ¦ course_id
------------+-----------
          2 ¦        10
(1 row)
```

■ 주의 CASCADE 는 신중하게

`ON DELETE CASCADE` 는 편리하지만 고객 한 명을 지웠는데 주문·리뷰·포인트 내역이 연쇄로 사라질 수 있습니다. shop 스키마에서는 `order_items → orders` 에만 CASCADE 를 썼고, `orders → customers` 는 기본값(거부)입니다. 3장 예제 03-10 에서 직접 확인합니다.

구조 변경 - ALTER TABLE

작업	문법
컬럼 추가	`ALTER TABLE t ADD COLUMN c 자료형 [제약]`
컬럼 삭제	`ALTER TABLE t DROP COLUMN c`
자료형 변경	`ALTER TABLE t ALTER COLUMN c TYPE 새자료형 [USING 변환식]`
NOT NULL 설정/해제	`ALTER TABLE t ALTER COLUMN c SET NOT NULL` / `DROP NOT NULL`
기본값	`ALTER TABLE t ALTER COLUMN c SET DEFAULT 값`
이름 변경	`ALTER TABLE t RENAME COLUMN a TO b` / `RENAME TO 새테이블명`
제약 추가/삭제	`ALTER TABLE t ADD CONSTRAINT 이름 ...` / `DROP CONSTRAINT 이름`

예제 02-10 · ALTER TABLE - 컬럼 추가/변경/삭제, 이름 바꾸기

```
ALTER TABLE notices ADD COLUMN created_at TIMESTAMP DEFAULT now();
ALTER TABLE notices ALTER COLUMN title SET NOT NULL;
ALTER TABLE notices ALTER COLUMN title TYPE VARCHAR(200);
ALTER TABLE notices ADD PRIMARY KEY (id);
ALTER TABLE notices RENAME COLUMN content TO body;
ALTER TABLE notices DROP COLUMN created_at;
SELECT column_name, data_type, character_maximum_length AS max_len,
       is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'notices'
ORDER BY ordinal_position;
```

▶ 실행 결과

```
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
 column_name ┆     data_type     ┆ max_len ┆ is_nullable ┆ column_default
-------------+-------------------+---------+-------------+----------------
 id          ┆ integer           ┆         ┆ NO          ┆
 title       ┆ character varying ┆     200 ┆ NO          ┆
 body        ┆ text              ┆         ┆ YES         ┆
(3 rows)
```

예제 02-11 · 제약 조건에 이름 붙이고 추가/삭제하기

```
ALTER TABLE members ADD CONSTRAINT chk_members_level
    CHECK (level IN ('NORMAL', 'SILVER', 'GOLD'));
SELECT conname, contype   -- p: 기본키, u: 유니크, c: 체크, f: 외래키
FROM pg_constraint
WHERE conrelid = 'members'::regclass
ORDER BY conname;
ALTER TABLE members DROP CONSTRAINT chk_members_level;
```

▶ 실행 결과

```
ALTER TABLE
           conname            ┆ contype
------------------------------+---------
 chk_members_level            ┆ c
 members_age_check            ┆ c
 members_created_at_not_null  ┆ n
 members_email_key            ┆ u
 members_email_not_null       ┆ n
 members_level_not_null       ┆ n
 members_member_id_not_null   ┆ n
 members_name_not_null        ┆ n
 members_pkey                 ┆ p
(9 rows)

ALTER TABLE
```

데이터 삭제 vs 테이블 삭제

명령	지우는 것	되돌리기	속도
`DELETE FROM t`	행 (조건 가능)	트랜잭션 안이면 ROLLBACK 가능	행마다 처리 → 느림
`TRUNCATE t`	모든 행 (구조 유지)	PostgreSQL 은 트랜잭션 안이면 ROLLBACK 가능	매우 빠름
`DROP TABLE t`	테이블 자체	트랜잭션 안이면 ROLLBACK 가능	빠름

예제 02-12 · TRUNCATE 와 DROP TABLE

```
TRUNCATE TABLE typetest;              -- 데이터만 모두 삭제 (구조는 유지)
SELECT count(*) FROM typetest;
DROP TABLE typetest;                  -- 테이블 자체를 삭제
DROP TABLE IF EXISTS typetest;        -- 없어도 오류 없이 넘어감
```

▶ 실행 결과

```
TRUNCATE TABLE
 count
-------
     0
(1 row)

DROP TABLE
NOTICE:  table "typetest" does not exist, skipping
DROP TABLE
```

■ Q&A; **자주 묻는 질문**

- **Q. 제약 조건 이름은 꼭 지어야 하나요?** 생략하면 `members_email_key` 처럼 자동으로 붙습니다. 오류 메시지에 이름이 나오므로 팀 규칙(예: `chk_테이블_내용`)을 정해 직접 짓는 것이 좋습니다.
- **Q. PRIMARY KEY 로 주민번호·이메일 같은 실제 값을 써도 되나요?** 바뀔 수 있는 값(이메일)이나 민감한 값(주민번호)은 피하고, 의미 없는 대리 키(IDENTITY)를 PK 로, 실제 값은 UNIQUE 로 두는 것이 일반적입니다.
- **Q. 이미 데이터가 있는 테이블에 NOT NULL 컬럼을 추가하면?** 기존 행에 넣을 값이 없어 오류가 납니다. `ADD COLUMN c INT NOT NULL DEFAULT 0` 처럼 기본값을 함께 주면 됩니다.

실습 문제

번호	문제
02-Q1	도서(books) 테이블 만들기: book_id(자동증가 PK), title(필수, 200자), author(100자), price(0 이상 정수), published(날짜), in_stock(기본 true)
02-Q2	회원(book_members)과 대출(loans) 테이블을 만들고 loans 가 books, book_members 를 참조하도록 외래 키를 거세요 (대출일 기본값 오늘)
02-Q3	books 에 isbn 컬럼(13자, 중복 불가)을 추가하고, price 컬럼 이름을 list_price 로 바꾸세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 자료형은 "어떤 값을 담을지", 제약 조건은 "어떤 값을 거부할지"를 정한다.
- 금액은 `NUMERIC`, ID 는 `IDENTITY`, 문자열은 `TEXT` 또는 `VARCHAR(n)`.
- 외래 키는 관계를 보장하고, `ON DELETE` 로 부모 삭제 시 동작을 정한다.

03장 · 데이터 다루기 - INSERT / UPDATE / DELETE

학습 목표

- INSERT 로 한 행·여러 행·조회 결과를 넣을 수 있다.
- UPDATE / DELETE 를 WHERE 조건과 함께 안전하게 사용할 수 있다.
- RETURNING 으로 변경된 행을 바로 확인할 수 있다.
- UPSERT(`ON CONFLICT`)와 `MERGE` 로 "있으면 수정, 없으면 입력"을 처리할 수 있다.

예제 파일: `examples/03_dml.sql` (실습 후 `setup/` 스크립트로 데이터 초기화)

INSERT

```
INSERT INTO 테이블 (컬럼1, 컬럼2, ...) VALUES (값1, 값2, ...);
INSERT INTO 테이블 (컬럼1, 컬럼2) VALUES (...), (...), (...);    -- 여러 행
INSERT INTO 테이블 (컬럼1, 컬럼2) SELECT ... ;                    -- 조회 결과
```

예제 03-01 · INSERT - 한 행 넣기 (컬럼 목록을 쓰는 습관!)

```
INSERT INTO todo (task, priority) VALUES ('PostgreSQL 설치하기', 1);
INSERT INTO todo (task) VALUES ('psql 접속해 보기');          -- priority, done 은 DEFAULT
SELECT * FROM todo;
```

▶ 실행 결과

```
INSERT 0 1
INSERT 0 1
 todo_id ┆        task         ┆ priority ┆ done
---------+---------------------+----------+------
       1 ┆ PostgreSQL 설치하기 ┆        1 ┆ f
       2 ┆ psql 접속해 보기    ┆        3 ┆ f
(2 rows)
```

예제 03-02 · INSERT - 여러 행을 한 번에

```
INSERT INTO todo (task, priority) VALUES
    ('SELECT 연습', 2),
    ('JOIN 이해하기', 2),
    ('인덱스 공부', 4);
SELECT * FROM todo ORDER BY todo_id;
```

▶ 실행 결과

```
INSERT 0 3
 todo_id ┆        task         ┆ priority ┆ done
---------+---------------------+----------+------
       1 ┆ PostgreSQL 설치하기 ┆        1 ┆ f
       2 ┆ psql 접속해 보기    ┆        3 ┆ f
       3 ┆ SELECT 연습         ┆        2 ┆ f
       4 ┆ JOIN 이해하기       ┆        2 ┆ f
       5 ┆ 인덱스 공부         ┆        4 ┆ f
(5 rows)
```

■ 핵심 포인트 **INSERT 에는 항상 컬럼 목록을**
`INSERT INTO todo VALUES (...)` 처럼 컬럼 목록을 생략하면 테이블의 컬럼 순서에 의존하게 됩니다. 나중에 컬럼이 추가되면 모든 INSERT 가 깨지므로 **항상 컬럼 목록을 쓰는 습관**을 들이세요. psql 이 출력하는 `INSERT 0 3` 의 0 은 과거 호환용 값(OID), 3 은 들어간 행 수입니다.

RETURNING 은 PostgreSQL 의 강력한 기능입니다. 자동 생성된 ID 를 다시 조회할 필요 없이 바로 받아 볼 수 있어 애플리케이션 개발 시 매우 자주 씁니다.

예제 03-03 · RETURNING - 방금 넣은 행(자동 생성된 ID)을 바로 돌려받기

```
INSERT INTO todo (task, priority) VALUES ('트랜잭션 공부', 3)
RETURNING todo_id, task;
```

▶ 실행 결과

```
 todo_id ┆     task
---------+---------------
       6 ┆ 트랜잭션 공부
(1 row)

INSERT 0 1
```

예제 03-04 · INSERT ... SELECT - 조회 결과를 그대로 넣기

```
INSERT INTO todo (task, priority)
SELECT name ¦¦ ' 부서 미팅', 5 FROM departments WHERE location = '서울';
SELECT * FROM todo WHERE priority = 5;
```

▶ 실행 결과

```
INSERT 0 3
 todo_id ┆        task        ┆ priority ┆ done
---------+--------------------+----------+------
       7 ┆ 경영지원 부서 미팅 ┆        5 ┆ f
       8 ┆ 영업 부서 미팅     ┆        5 ┆ f
       9 ┆ 마케팅 부서 미팅   ┆        5 ┆ f
(3 rows)
```

UPDATE 와 DELETE

```
UPDATE 테이블 SET 컬럼 = 값 [, 컬럼 = 값 ...] WHERE 조건;
DELETE FROM 테이블 WHERE 조건;
```

예제 03-05 · UPDATE - 조건에 맞는 행 수정

```
UPDATE todo SET done = true WHERE todo_id IN (1, 2);
UPDATE todo SET priority = priority - 1, task = task ¦¦ ' (중요)'
WHERE priority >= 4
RETURNING todo_id, task, priority;
```

▶ 실행 결과

```
UPDATE 2
 todo_id ┆           task            ┆ priority
---------+---------------------------+----------
       5 ┆ 인덱스 공부 (중요)        ┆        3
       7 ┆ 경영지원 부서 미팅 (중요) ┆        4
       8 ┆ 영업 부서 미팅 (중요)     ┆        4
       9 ┆ 마케팅 부서 미팅 (중요)   ┆        4
(4 rows)

UPDATE 4
```

예제 03-06 · DELETE - 조건에 맞는 행 삭제

```
DELETE FROM todo WHERE task LIKE '%부서 미팅%';
DELETE FROM todo WHERE done RETURNING *;
SELECT * FROM todo ORDER BY todo_id;
```

▶ 실행 결과

```
DELETE 3
 todo_id ¦        task         ¦ priority ¦ done
---------+---------------------+----------+------
       1 ¦ PostgreSQL 설치하기 ¦        1 ¦ t
       2 ¦ psql 접속해 보기    ¦        3 ¦ t
(2 rows)

DELETE 2
 todo_id ¦        task        ¦ priority ¦ done
---------+--------------------+----------+------
       3 ¦ SELECT 연습        ¦        2 ¦ f
       4 ¦ JOIN 이해하기      ¦        2 ¦ f
       5 ¦ 인덱스 공부 (중요) ¦        3 ¦ f
       6 ¦ 트랜잭션 공부      ¦        3 ¦ f
(4 rows)
```

■ 주의 WHERE 없는 UPDATE / DELETE

실무 사고 1순위입니다. 예제 03-07 처럼 **먼저** `BEGIN` **으로 트랜잭션을 열고** 실행 → 결과 확인 → `COMMIT` 또는 `ROLLBACK` 하는 습관을 들이세요. 또 UPDATE/DELETE 를 실행하기 전에 같은 WHERE 로 `SELECT` 를 먼저 실행해 대상 행을 확인하는 것도 좋은 습관입니다.

예제 03-07 · [주의] WHERE 없는 UPDATE/DELETE 는 모든 행에 적용된다

```
BEGIN;                                  -- 안전하게 트랜잭션 안에서 실험
UPDATE todo SET priority = 0;           -- 모든 행이 바뀐다!
SELECT todo_id, priority FROM todo ORDER BY todo_id;
ROLLBACK;                               -- 없던 일로
SELECT todo_id, priority FROM todo ORDER BY todo_id;
```

▶ 실행 결과

```
BEGIN
UPDATE 4
 todo_id ¦ priority
---------+----------
       3 ¦        0
       4 ¦        0
       5 ¦        0
       6 ¦        0
(4 rows)

ROLLBACK
 todo_id ¦ priority
---------+----------
       3 ¦        2
       4 ¦        2
       5 ¦        3
       6 ¦        3
(4 rows)
```

■ 참고 행의 순서

03-07 에서 UPDATE 한 뒤 ORDER BY 없이 조회하면 행 순서가 바뀌어 보일 수 있습니다. PostgreSQL 은 UPDATE 시 행의 새 버전을 다른 위치에 쓰기 때문입니다(14장 MVCC). **순서가 필요하면 반드시 ORDER BY** 를 쓴다는 점을 강조합니다.

shop 데이터 수정하기

예제 03-08 · 실제 shop 데이터 수정 - 재고 입고와 가격 인상

```
UPDATE products SET stock = stock + 30 WHERE product_id = 9 RETURNING name, stock;
UPDATE products SET price = price * 1.1
WHERE category_id = 9
RETURNING product_id, name, price;
```

▶ 실행 결과

```
    name    ¦ stock
------------+-------
 USB-C 허브 ¦    30
(1 row)

UPDATE 1
 product_id ¦      name      ¦ price
------------+----------------+-------
         14 ¦ 수제 쿠키 세트 ¦ 16500
         15 ¦ 견과류 믹스    ¦ 24200
(2 rows)

UPDATE 2
```

예제 03-09 · 다른 테이블을 참고해서 UPDATE (UPDATE ... FROM)

```
-- 주문이 10건 이상인 BRONZE/SILVER 고객을 GOLD 로 승급
UPDATE customers c
SET grade = 'GOLD'
FROM (SELECT customer_id, count(*) AS cnt
      FROM orders
      GROUP BY customer_id) o
WHERE c.customer_id = o.customer_id
  AND o.cnt >= 10
  AND c.grade IN ('BRONZE', 'SILVER')
RETURNING c.customer_id, c.name, c.grade, o.cnt;
```

▶ 실행 결과

```
 customer_id ¦  name  ¦ grade ¦ cnt
-------------+--------+-------+-----
           3 ¦ 박도윤 ¦ GOLD  ¦  13
(1 row)

UPDATE 1
```

`UPDATE ... FROM` 은 다른 테이블(또는 서브쿼리)의 값을 참고해 수정할 때 씁니다. 표준 SQL 에는 없는 PostgreSQL 확장 문법이며, 표준 방식은 8장의 서브쿼리 UPDATE 입니다.

예제 03-10 · 참조되고 있는 행은 지울 수 없다

```
DELETE FROM customers WHERE customer_id = 1;     -- orders 가 참조 중
```

▶ 실행 결과

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

UPSERT 와 MERGE

"이미 있으면 수정, 없으면 입력"은 장바구니 담기, 재고 동기화, 일별 집계 테이블 갱신 등에 흔히 필요합니다.

방법	특징
`INSERT ... ON CONFLICT (키) DO UPDATE`	PostgreSQL 전용, 동시 실행에도 안전(원자적), 가장 많이 씀

방법	특징
`INSERT ... ON CONFLICT DO NOTHING`	중복이면 조용히 건너뜀
`MERGE` (15+)	SQL 표준, 원본 테이블과 비교해 INSERT/UPDATE/DELETE 를 한 번에

`EXCLUDED` 는 "넣으려다 충돌한 새 행"을 가리키는 특별한 이름입니다.

예제 03-11 · UPSERT - INSERT ... ON CONFLICT

```
CREATE TABLE inventory (
    sku   VARCHAR(20) PRIMARY KEY,
    qty   INTEGER NOT NULL
);
INSERT INTO inventory VALUES ('A-100', 10), ('B-200', 5);
-- 이미 있으면 수량을 더하고, 없으면 새로 넣는다
INSERT INTO inventory (sku, qty) VALUES ('A-100', 3), ('C-300', 7)
ON CONFLICT (sku) DO UPDATE SET qty = inventory.qty + EXCLUDED.qty;
-- 이미 있으면 아무것도 하지 않는다
INSERT INTO inventory (sku, qty) VALUES ('B-200', 999)
ON CONFLICT (sku) DO NOTHING;
SELECT * FROM inventory ORDER BY sku;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 2
INSERT 0 2
INSERT 0 0
  sku  ¦ qty
-------+-----
 A-100 ¦  13
 B-200 ¦   5
 C-300 ¦   7
(3 rows)
```

예제 03-12 · MERGE (PostgreSQL 15+) - 조건에 따라 INSERT/UPDATE/DELETE 를 한 문장으로

```
CREATE TABLE inventory_in (sku VARCHAR(20), qty INTEGER);
INSERT INTO inventory_in VALUES ('A-100', -13), ('B-200', 5), ('D-400', 2);
MERGE INTO inventory t
USING inventory_in s ON t.sku = s.sku
WHEN MATCHED AND t.qty + s.qty <= 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = t.qty + s.qty
WHEN NOT MATCHED THEN INSERT (sku, qty) VALUES (s.sku, s.qty);
SELECT * FROM inventory ORDER BY sku;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 3
MERGE 3
  sku  ¦ qty
-------+-----
 B-200 ¦  10
 C-300 ¦   7
 D-400 ¦   2
(3 rows)
```

■ Q&A; 자주 묻는 질문

- **Q. DELETE 한 데이터를 되살릴 수 있나요?** COMMIT 했다면 SQL 로는 되돌릴 수 없습니다. 백업(18장)에서 복구해야 합니다. 그래서 실무에서는 실제 삭제 대신 `deleted_at` 컬럼을 두는 **소프트 삭제**를 많이 씁니다.
- **Q. 대량 입력은 INSERT 를 반복하면 되나요?** 수만 행 이상이면 `COPY` (18장)나 여러 행 INSERT 를 쓰면 수십 배 빠릅니다.

실습 문제

번호	문제
03-Q1	books 에 책 3권을 한 번에 넣고 생성된 book_id 와 title 을 RETURNING 으로 확인하세요
03-Q2	2023년 이전에 출간된 책의 가격을 10% 인하하고 바뀐 행을 확인하세요
03-Q3	shop: 재고가 0 인 상품의 is_active 를 false 로 바꾸고, 바뀐 상품명을 출력하세요
03-Q4	shop: inventory_log(sku PK, qty) 에 ('P-1', 5) 를 UPSERT 로 두 번 실행해 qty 가 10 이 되게 하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- INSERT 는 컬럼 목록을 명시하고, RETURNING 으로 결과를 바로 받는다.
- UPDATE/DELETE 는 WHERE 를 먼저 SELECT 로 검증하고, 트랜잭션 안에서 실행한다.
- 있으면 수정·없으면 입력은 `ON CONFLICT` 또는 `MERGE`.

04장 · SELECT 기초 - 원하는 행과 열 골라내기

학습 목표

- SELECT 로 필요한 컬럼과 계산식을 조회하고 별칭(AS)을 붙일 수 있다.
- WHERE 에서 비교·논리 연산자, IN, BETWEEN, LIKE, IS NULL 을 사용할 수 있다.
- ORDER BY, LIMIT / OFFSET, FETCH FIRST 로 정렬과 페이지 나누기를 할 수 있다.
- SQL 의 논리적 실행 순서를 설명할 수 있다.

예제 파일: `examples/04_select_basic.sql`

SELECT 의 기본 구조

```
SELECT   컬럼, 식 AS 별칭        -- ⑤ 무엇을 보여 줄까
FROM     테이블                  -- ① 어디에서
WHERE    행 조건                 -- ② 어떤 행을
GROUP BY 그룹 기준               -- ③ (6장)
HAVING   그룹 조건               -- ④ (6장)
ORDER BY 정렬 기준               -- ⑥ 어떤 순서로
LIMIT    개수 OFFSET 건너뛸 수;   -- ⑦ 몇 개만
```

작성 순서와 **실행 순서(①~⑦)** 가 다르다는 점이 이 장의 핵심입니다.

예제 04-01 · 필요한 컬럼만 고르기 + 별칭(AS)

```
SELECT name AS 상품명, price AS 가격, stock AS "재고 수량"
FROM products
LIMIT 5;
```

▶ 실행 결과

```
      상품명       ¦  가격   ¦ 재고 수량
--------------------+---------+-----------
 울트라북 14        ¦ 1290000 ¦        15
 비즈니스 노트북 15 ¦  990000 ¦        22
 게이밍 노트북 X17  ¦ 2490000 ¦         6
 스마트폰 S25       ¦ 1150000 ¦        30
 스마트폰 미니      ¦  690000 ¦        18
(5 rows)
```

예제 04-02 · 계산된 컬럼

```
SELECT name,
       price,
       price * 0.9         AS 할인가,
       price * stock       AS 재고금액
FROM products
WHERE category_id = 6;
```

▶ 실행 결과

```
     name      ¦ price  ¦  할인가  ¦ 재고금액
---------------+--------+----------+----------
 무선 마우스   ¦  29000 ¦  26100.0 ¦  3480000
 기계식 키보드 ¦  89000 ¦  80100.0 ¦  4005000
 27인치 모니터 ¦ 329000 ¦ 296100.0 ¦  3948000
 USB-C 허브    ¦  45000 ¦  40500.0 ¦        0
 노트북 거치대 ¦  35000 ¦  31500.0 ¦  2450000
(5 rows)
```

예제 04-03 · DISTINCT - 중복 제거

```
SELECT DISTINCT city FROM customers ORDER BY city;
SELECT DISTINCT city, grade FROM customers WHERE city = '서울' ORDER BY grade;
```

▶ 실행 결과

```
 city
------
 광주
 대구
 대전
 부산
 서울
 수원
 인천
(7 rows)

 city ¦ grade
------+--------
 서울 ¦ BRONZE
 서울 ¦ GOLD
 서울 ¦ SILVER
 서울 ¦ VIP
(4 rows)
```

■ 핵심 포인트 **작은따옴표와 큰따옴표**
별칭에 공백이나 대문자를 쓰려면 큰따옴표("재고 수량")를 씁니다. **작은따옴표는 문자열 값, 큰따옴표는 이름(식별자)**입니다. 한글 별칭은 따옴표 없이 써도 되지만 실무 코드에서는 영문 snake_case 를 권장합니다.

WHERE - 행 거르기

연산자	의미	예
=, <> (!=)	같다, 다르다	grade <> 'BRONZE'
>, >=, <, <=	크기 비교	price >= 1000000
AND, OR, NOT	논리 연산 (우선순위 NOT > AND > OR)	a AND (b OR c)
IN (목록)	목록 중 하나	city IN ('대구', '대전')
BETWEEN a AND b	a 이상 b 이하 (양끝 포함)	price BETWEEN 30000 AND 60000
LIKE / ILIKE	패턴 (% 0자 이상, _ 1자) / 대소문자 무시	name LIKE '%노트북%'
IS NULL / IS NOT NULL	NULL 여부	phone IS NULL

예제 04-04 · WHERE + 비교 연산자

```
SELECT name, price FROM products WHERE price >= 1000000;
SELECT name, grade FROM customers WHERE grade <> 'BRONZE' AND city = '서울';
```

▶ 실행 결과

```
       name        ¦  price
-------------------+---------
 울트라북 14       ¦ 1290000
 게이밍 노트북 X17 ¦ 2490000
 스마트폰 S25      ¦ 1150000
(3 rows)

  name  ¦ grade
--------+--------
 김민준 ¦ VIP
```

```
 박도윤 ¦ SILVER
 강시우 ¦ SILVER
 임준서 ¦ GOLD
 오건우 ¦ SILVER
(5 rows)
```

예제 04-05 · AND / OR 우선순위 - 괄호를 꼭 쓰자

```
-- (의도) 서울 또는 부산에 사는 VIP 고객
SELECT name, city, grade FROM customers
WHERE city = '서울' OR city = '부산' AND grade = 'VIP';      -- 잘못된 결과!
SELECT name, city, grade FROM customers
WHERE (city = '서울' OR city = '부산') AND grade = 'VIP';    -- 올바른 결과
```

▶ 실행 결과

```
  name  ¦ city ¦ grade
--------+------+--------
 김민준 ¦ 서울 ¦ VIP
 박도윤 ¦ 서울 ¦ SILVER
 강시우 ¦ 서울 ¦ SILVER
 임준서 ¦ 서울 ¦ GOLD
 오건우 ¦ 서울 ¦ SILVER
 권하린 ¦ 서울 ¦ BRONZE
 류다은 ¦ 서울 ¦ BRONZE
(7 rows)

  name  ¦ city ¦ grade
--------+------+-------
 김민준 ¦ 서울 ¦ VIP
(1 row)
```

■ 주의 AND 가 OR 보다 먼저

04-05 의 첫 쿼리는 `city = '서울' OR (city = '부산' AND grade = 'VIP')` 로 해석되어 서울 고객이 모두 나옵니다. AND 와 OR 을 섞을 때는 **무조건 괄호**를 쓰세요.

예제 04-06 · IN, BETWEEN

```
SELECT name, city FROM customers WHERE city IN ('대구', '대전', '광주');
SELECT name, price FROM products WHERE price BETWEEN 30000 AND 60000 ORDER BY price;
```

▶ 실행 결과

```
  name  ¦ city
--------+------
 최지우 ¦ 대구
 조수아 ¦ 광주
 윤지호 ¦ 대전
 신우진 ¦ 대구
 안채원 ¦ 광주
 송현우 ¦ 대전
(6 rows)

     name      ¦ price
---------------+-------
 노트북 거치대 ¦ 35000
 남성 셔츠     ¦ 39000
 USB-C 허브    ¦ 45000
 여성 니트     ¦ 49000
 남성 청바지   ¦ 59000
(5 rows)
```

예제 04-07 · LIKE / ILIKE - 패턴 검색 (% 는 0개 이상, _ 는 정확히 1글자)

```
SELECT name FROM products WHERE name LIKE '%노트북%';
SELECT name FROM customers WHERE name LIKE '김__';
SELECT name FROM products WHERE name ILIKE 'usb%';          -- 대소문자 무시
```

▶ 실행 결과

```
        name
--------------------
 비즈니스 노트북 15
 게이밍 노트북 X17
 노트북 거치대
(3 rows)

  name
--------
 김민준
(1 row)

    name
------------
 USB-C 허브
(1 row)
```

NULL - 값이 "없음"

NULL 은 0 도 빈 문자열도 아닌 **"알 수 없음"** 입니다. NULL 과의 비교·연산 결과는 참도 거짓도 아닌 NULL(unknown)이 되고, WHERE 는 참인 행만 통과시키므로 `= NULL` 은 항상 0건입니다.

예제 04-08 · NULL 다루기 - = NULL 은 항상 실패한다

```
SELECT name, phone FROM customers WHERE phone = NULL;        -- 0건!
SELECT name, phone FROM customers WHERE phone IS NULL;
SELECT count(*) AS 전화번호있음 FROM customers WHERE phone IS NOT NULL;
```

▶ 실행 결과

```
 name ¦ phone
------+-------
(0 rows)

  name  ¦ phone
--------+-------
 박도윤 ¦
 조수아 ¦
 서지아 ¦
 홍길동 ¦
(4 rows)

 전화번호있음
--------------
           16
(1 row)
```

예제 04-09 · NULL 과 연산하면 결과도 NULL

```
SELECT NULL = NULL AS 같은가, NULL + 1 AS 더하기, 'a' || NULL AS 연결,
       NULL IS NULL AS is_null,
       NULL IS DISTINCT FROM NULL AS 다른가;
```

▶ 실행 결과

```
 같은가 ¦ 더하기 ¦ 연결 ¦ is_null ¦ 다른가
--------+--------+------+---------+--------
        ¦        ¦      ¦ t       ¦ f
(1 row)
```

A	B	A AND B	A OR B
TRUE	NULL	NULL	TRUE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

> ■ **TIP IS DISTINCT FROM**
> "NULL 도 하나의 값처럼 비교하고 싶을 때" `a IS DISTINCT FROM b` (다른가), `a IS NOT DISTINCT FROM b` (같은가)를 씁니다. 15장 트리거 예제에서 "가격이 실제로 바뀌었을 때만"을 판단할 때 사용합니다.

정렬과 개수 제한

예제 04-10 · ORDER BY - 정렬 (여러 기준, 내림차순)

```
SELECT name, grade, city FROM customers
ORDER BY grade, city DESC, name
LIMIT 8;
```

▶ 실행 결과

```
  name  ¦ grade  ¦ city
--------+--------+------
 서지아 ¦ BRONZE ¦ 인천
 권하린 ¦ BRONZE ¦ 서울
 류다은 ¦ BRONZE ¦ 서울
 한소율 ¦ BRONZE ¦ 부산
 홍길동 ¦ BRONZE ¦ 부산
 송현우 ¦ BRONZE ¦ 대전
 최지우 ¦ BRONZE ¦ 대구
 안채원 ¦ BRONZE ¦ 광주
(8 rows)
```

예제 04-11 · NULL 의 정렬 위치 지정 (NULLS FIRST / LAST)

```
SELECT name, birth_date FROM customers
ORDER BY birth_date DESC NULLS LAST
LIMIT 5;
```

▶ 실행 결과

```
  name  ¦ birth_date
--------+------------
 서지아 ¦ 2003-08-19
 류다은 ¦ 2002-06-06
 조수아 ¦ 2001-01-08
 안채원 ¦ 2000-11-11
 최지우 ¦ 1999-04-17
(5 rows)
```

- `ORDER BY` 는 기본 오름차순(`ASC`), `DESC` 는 내림차순. 여러 기준은 쉼표로 나열합니다.

- PostgreSQL 은 NULL 을 **가장 큰 값**으로 취급해 오름차순이면 맨 뒤, 내림차순이면 맨 앞에 둡니다. `NULLS FIRST/LAST` 로 바꿉니다.
- 한글 정렬 순서는 데이터베이스 생성 시의 로케일(Collation)을 따릅니다.

예제 04-12 · LIMIT / OFFSET - 페이지 나누기 (한 페이지 5건, 2페이지)

```
SELECT product_id, name FROM products
ORDER BY product_id
LIMIT 5 OFFSET 5;
```

▶ 실행 결과

```
 product_id ┆     name
------------+---------------
          6 ┆ 무선 마우스
          7 ┆ 기계식 키보드
          8 ┆ 27인치 모니터
          9 ┆ USB-C 허브
         10 ┆ 남성 셔츠
(5 rows)
```

예제 04-13 · 표준 문법 FETCH FIRST … ROWS ONLY / WITH TIES

```
SELECT name, price FROM products
ORDER BY price DESC
FETCH FIRST 3 ROWS ONLY;
SELECT name, stock FROM products
ORDER BY stock
FETCH FIRST 1 ROWS WITH TIES;               -- 1등과 같은 값이면 함께 출력
```

▶ 실행 결과

```
       name        ┆  price
-------------------+---------
 게이밍 노트북 X17 ┆ 2490000
 울트라북 14       ┆ 1290000
 스마트폰 S25      ┆ 1150000
(3 rows)

    name     ┆ stock
-------------+-------
 탄산수 24입 ┆     0
 USB-C 허브  ┆     0
(2 rows)
```

■ 참고 **OFFSET 페이지네이션의 한계**

`OFFSET 100000` 은 앞의 10만 행을 읽고 버리므로 뒤 페이지일수록 느려집니다. 실무에서는 "마지막으로 본 id 보다 큰 것" `WHERE id > :last_id ORDER BY id LIMIT 20` 방식(키셋 페이지네이션)을 씁니다. 13장 인덱스와 연결해 설명할 수 있습니다.

날짜 조건과 실행 순서

예제 04-14 · 날짜 조건 - 2026년 1분기 주문

```
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE order_date >= '2026-01-01' AND order_date < '2026-04-01'
ORDER BY order_date
LIMIT 6;
```

▶ 실행 결과

```
 order_id ¦ customer_id ¦     order_date      ¦  status
----------+-------------+---------------------+-----------
       82 ¦           8 ¦ 2026-01-04 14:29:00 ¦ DELIVERED
       83 ¦          18 ¦ 2026-01-09 16:35:00 ¦ DELIVERED
       84 ¦          10 ¦ 2026-01-14 09:35:00 ¦ DELIVERED
       85 ¦           6 ¦ 2026-01-15 16:43:00 ¦ DELIVERED
       86 ¦          16 ¦ 2026-01-16 08:14:00 ¦ DELIVERED
       87 ¦           8 ¦ 2026-01-17 10:34:00 ¦ CANCELLED
(6 rows)
```

■ TIP 날짜 범위는 반열림 구간으로

`order_date BETWEEN '2026-01-01' AND '2026-03-31'` 은 3월 31일 0시까지만 포함해 31일 낮 주문이 빠집니다. TIMESTAMP 컬럼은 `>= 시작 AND < 다음날` 형태로 쓰는 것이 안전합니다.

예제 04-15 · SQL 실행 순서: FROM → WHERE → SELECT → ORDER BY → LIMIT

```
-- SELECT 에서 만든 별칭은 ORDER BY 에서는 쓸 수 있지만 WHERE 에서는 쓸 수 없다
SELECT name, price * stock AS 재고금액 FROM products ORDER BY 재고금액 DESC LIMIT 3;
```

▶ 실행 결과

```
        name        ¦ 재고금액
--------------------+----------
 스마트폰 S25       ¦ 34500000
 비즈니스 노트북 15 ¦ 21780000
 울트라북 14        ¦ 19350000
(3 rows)
```

예제 04-16 · WHERE 에서 SELECT 별칭을 쓰면 오류

```
SELECT name, price * stock AS 재고금액 FROM products WHERE 재고금액 > 1000000;
```

▶ 실행 결과

```
ERROR:  column "재고금액" does not exist
LINE 1: ...me, price * stock AS 재고금액 FROM products WHERE 재고금액 >...
                                                             ^
```

■ Q&A; 자주 묻는 질문

- **Q. `SELECT *` 를 써도 되나요?** 학습·탐색용으로는 편하지만, 프로그램 코드에서는 필요한 컬럼만 적어야 합니다(불필요한 데이터 전송, 컬럼 추가 시 영향).
- **Q. ORDER BY 없이 조회하면 입력 순서대로 나오나요?** 보장되지 않습니다. 지금은 그렇게 보여도 수정·VACUUM·병렬 처리 후 바뀔 수 있습니다.
- **Q. LIKE 에서 % 자체를 찾으려면?** `LIKE '%10\%%'` 처럼 역슬래시로 이스케이프합니다.

실습 문제

번호	문제
04-Q1	가격이 50,000원 이상 300,000원 이하인 상품의 이름과 가격을 가격 내림차순으로 조회하세요
04-Q2	이메일이 'j' 로 시작하는 고객의 이름, 이메일을 조회하세요

번호	문제
04-Q3	서울·인천에 사는 1990년 이후 출생 고객을 생년월일 순으로 조회하세요
04-Q4	주문 상태(status)의 종류를 중복 없이 조회하세요
04-Q5	가장 최근에 가입한 고객 3명의 이름과 가입일시를 조회하세요
04-Q6	2025년 12월에 들어온 주문 중 담당 직원이 없는(온라인) 주문을 조회하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 실행 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- NULL 비교는 `IS NULL`, AND/OR 혼용 시 괄호 필수.
- 순서가 필요하면 ORDER BY, 날짜 범위는 `>= AND <`.

05장 · 내장 함수 - 문자열, 숫자, 날짜, 조건식

학습 목표

- 문자열·숫자·날짜 함수로 값을 가공할 수 있다.
- `CAST` / `::` 로 형변환하고 `to_char` 로 서식을 지정할 수 있다.
- `CASE`, `COALESCE`, `NULLIF` 로 조건에 따라 값을 바꿀 수 있다.

예제 파일: `examples/05_functions.sql`

문자열 함수

함수	설명	예 → 결과
`length(s)`	글자 수 (한글도 1글자)	`length('데이터')` → 3
`upper` / `lower`	대/소문자	`upper('sql')` → SQL
`substring(s FROM n FOR m)`	부분 문자열	`substring('abcdef' FROM 2 FOR 3)` → bcd
`left(s, n)` / `right(s, n)`	왼쪽/오른쪽 n글자	`left('010-1234', 3)` → 010
`||`, `concat`, `concat_ws`	연결 (concat 은 NULL 무시)	`concat_ws('-', 'a', 'b')` → a-b
`trim` / `ltrim` / `rtrim`	공백(또는 지정 문자) 제거	
`replace(s, a, b)`	a 를 b 로 모두 바꿈	
`lpad` / `rpad`	왼쪽/오른쪽 채우기	`lpad('42', 5, '0')` → 00042
`split_part(s, 구분자, n)`	구분자로 나눈 n번째	`split_part('a@b.com', '@', 2)` → b.com
`position(a IN s)`	위치 (없으면 0)	
`regexp_replace` / `~`	정규식 치환 / 정규식 일치	15장 트리거 예제

예제 05-01 · 문자열 함수 1 - 길이, 대소문자, 자르기

```
SELECT length('PostgreSQL')        AS 길이,
       length('데이터베이스')      AS 한글길이,
       upper('sql'), lower('SQL'),
       substring('PostgreSQL' FROM 1 FOR 8) AS 앞8글자,
       left('010-1234-5678', 3)    AS 왼쪽3,
       right('010-1234-5678', 4)   AS 오른쪽4;
```

▶ 실행 결과

```
 길이 ¦ 한글길이 ¦ upper ¦ lower ¦ 앞8글자  ¦ 왼쪽3 ¦ 오른쪽4
------+----------+-------+-------+----------+-------+---------
   10 ¦        6 ¦ SQL   ¦ sql   ¦ PostgreS ¦ 010   ¦ 5678
(1 row)
```

예제 05-02 · 문자열 함수 2 - 연결, 공백 제거, 바꾸기, 채우기

```
SELECT concat('김', '대', '철')            AS concat,
       concat_ws('-', '2026', '10', '23')   AS concat_ws,
       '[' || trim('   공백   ') || ']'     AS trim,
       replace('010-1234-5678', '-', '')    AS replace,
       lpad('42', 5, '0')                   AS lpad,
       split_part('a@test.com', '@', 2)     AS 도메인,
       position('@' IN 'a@test.com')        AS 위치;
```

▶ 실행 결과

```
 concat | concat_ws  |  trim  |   replace   | lpad  |  도메인  | 위치
--------+------------+--------+-------------+-------+----------+------
 김대철 | 2026-10-23 | [공백] | 01012345678 | 00042 | test.com |    2
(1 row)
```

예제 05-03 · 문자열 함수 활용 - 이메일 아이디와 전화번호 마스킹

```
SELECT name,
       split_part(email, '@', 1)                          AS 아이디,
       left(phone, 4) || '****' || right(phone, 5)        AS 마스킹번호
FROM customers
WHERE phone IS NOT NULL
LIMIT 4;
```

▶ 실행 결과

```
  name  | 아이디  |  마스킹번호
--------+---------+---------------
 김민준 | minjun  | 010-****-0001
 이서연 | seoyeon | 010-****-0002
 최지우 | jiwoo   | 010-****-0004
 정하은 | haeun   | 010-****-0005
(4 rows)
```

■ **핵심 포인트** **연결 연산자와 NULL**
`'a' || NULL` 은 NULL 이지만 `concat('a', NULL)` 은 `'a'` 입니다(4장 04-09 결과와 비교). 이름 + 별명처럼 NULL 이 섞일 수 있는 연결은 `concat` 이나 `COALESCE` 를 쓰세요.

숫자 함수와 형변환

예제 05-04 · 숫자 함수

```
SELECT round(1234.5678, 2) AS round2,
       round(1234.5678, -2) AS round_m2,
       trunc(1234.5678, 1) AS trunc1,
       ceil(4.1) AS ceil, floor(4.9) AS floor,
       abs(-7) AS abs, mod(17, 5) AS mod, 17 % 5 AS 나머지,
       power(2, 8) AS power, sqrt(144) AS sqrt;
```

▶ 실행 결과

```
 round2  | round_m2 | trunc1 | ceil | floor | abs | mod | 나머지 | power | sqrt
---------+----------+--------+------+-------+-----+-----+--------+-------+------
 1234.57 |     1200 | 1234.5 |    5 |     4 |   7 |   2 |      2 |   256 |   12
(1 row)
```

예제 05-05 · 정수 나눗셈 주의 - 하나라도 실수/NUMERIC 이면 소수가 나온다

```
SELECT 7 / 2 AS 정수끼리, 7 / 2.0 AS 실수포함, 7::numeric / 2 AS 형변환,
       round(7::numeric / 3, 2) AS 반올림;
```

▶ 실행 결과

```
 정수끼리 ¦      실수포함      ¦       형변환       ¦ 반올림
----------+--------------------+--------------------+--------
        3 ¦ 3.5000000000000000 ¦ 3.5000000000000000 ¦   2.33
(1 row)
```

> ■ 주의 **정수 ÷ 정수 = 정수**
> 평균, 비율, 퍼센트 계산에서 가장 흔한 실수입니다. `count(*) / 3` 이나 `a / b * 100` 은 소수점이 잘립니다. `100.0 * a / b` 처럼 앞에 실수를 곱하거나 `::numeric` 으로 바꿔 계산하세요. 6장 이후 비율 계산 예제가 모두 `100.0 *` 로 시작하는 이유입니다.

형변환은 표준 문법 `CAST(값 AS 타입)` 과 PostgreSQL 단축 문법 `값::타입` 이 있습니다.

예제 05-06 · 형변환 - CAST 와 :: 연산자

```
SELECT CAST('123' AS INTEGER) + 1    AS cast,
       '2026-10-23'::date + 7        AS 일주일뒤,
       123::text ¦¦ '번'             AS 숫자를문자로,
       '3.14'::numeric * 2           AS 문자를숫자로,
       to_char(1234567, 'FM9,999,999') ¦¦ '원' AS 천단위;
```

▶ 실행 결과

```
 cast ¦   일주일뒤   ¦ 숫자를문자로 ¦ 문자를숫자로 ¦    천단위
------+------------+--------------+--------------+-------------
  124 ¦ 2026-10-30 ¦ 123번        ¦         6.28 ¦ 1,234,567원
(1 row)
```

예제 05-07 · 형변환 실패

```
SELECT 'abc'::integer;
```

▶ 실행 결과

```
ERROR:  invalid input syntax for type integer: "abc"
LINE 1: SELECT 'abc'::integer;
               ^
```

날짜와 시간

함수 / 연산	설명
`current_date`, `current_time`, `now()`	오늘 날짜, 현재 시각, 현재 일시(트랜잭션 시작 시각)
`날짜 + 정수` / `날짜 - 날짜`	n일 후 / 두 날짜 사이 일수(정수)
`+ interval '1 month'`	기간 더하기 (`'2 hours'`, `'1 year 3 days'` 등)
`age(a, b)`	a - b 를 년·월·일로
`EXTRACT(필드 FROM 일시)`	YEAR, MONTH, DAY, HOUR, DOW(0=일), ISODOW(1=월), QUARTER, EPOCH 등
`date_trunc('month', 일시)`	해당 단위의 시작으로 자르기 (월별·주별 집계의 핵심)
`to_char(일시, '서식')`	문자열로 (YYYY, MM, DD, HH24, MI, SS, Day, Q 등)
`to_date` / `to_timestamp`	문자열을 날짜로

예제 05-08 · 현재 날짜와 시간

```
SELECT current_date, current_time(0), now()::timestamp(0) AS now,
       current_date - 1 AS 어제;
```

▶ 실행 결과

```
 current_date ¦ current_time ¦         now         ¦    어제
--------------+--------------+---------------------+------------
 2026-10-23   ¦ 18:00:32+09  ¦ 2026-10-23 18:00:32 ¦ 2026-10-22
(1 row)
```

예제 05-09 · 날짜 계산 - INTERVAL, 날짜 빼기, age()

```
SELECT date '2026-10-23' + interval '1 month'        AS 한달뒤,
       timestamp '2026-10-23 09:00' + interval '90 minutes' AS 구십분뒤,
       date '2026-12-25' - date '2026-10-23'         AS 크리스마스까지_일,
       age(date '2026-10-23', date '1990-05-15')     AS 나이;
```

▶ 실행 결과

```
       한달뒤        ¦      구십분뒤       ¦ 크리스마스까지_일 ¦          나이
---------------------+---------------------+------------------+-----------------------
 2026-11-23 00:00:00 ¦ 2026-10-23 10:30:00 ¦               63 ¦ 36 years 5 mons 8 days
(1 row)
```

예제 05-10 · 날짜에서 부분 추출 - EXTRACT, date_part, date_trunc

```
SELECT order_id, order_date,
       EXTRACT(YEAR FROM order_date)    AS 연,
       EXTRACT(MONTH FROM order_date)   AS 월,
       EXTRACT(DOW FROM order_date)     AS 요일번호,   -- 0=일요일
       date_trunc('month', order_date)  AS 월초
FROM orders
WHERE order_id <= 3;
```

▶ 실행 결과

```
 order_id ¦     order_date      ¦  연  ¦ 월 ¦ 요일번호 ¦        월초
----------+---------------------+------+----+----------+---------------------
        1 ¦ 2025-01-03 21:23:00 ¦ 2025 ¦  1 ¦        5 ¦ 2025-01-01 00:00:00
        2 ¦ 2025-01-04 08:19:00 ¦ 2025 ¦  1 ¦        6 ¦ 2025-01-01 00:00:00
        3 ¦ 2025-01-21 10:41:00 ¦ 2025 ¦  1 ¦        2 ¦ 2025-01-01 00:00:00
(3 rows)
```

예제 05-11 · 날짜 서식 지정 - to_char / to_date

```
SELECT to_char(order_date, 'YYYY-MM-DD')      AS 날짜,
       to_char(order_date, 'YYYY"년" MM"월" DD"일" HH24:MI') AS 한글서식,
       to_char(order_date, 'FMDay')           AS 요일
FROM orders WHERE order_id = 1;
SELECT to_date('2026/10/23', 'YYYY/MM/DD')    AS to_date,
       to_timestamp('20261023 1530', 'YYYYMMDD HH24MI') AS to_timestamp;
```

▶ 실행 결과

```
    날짜    ¦        한글서식        ¦  요일
------------+------------------------+--------
 2025-01-03 ¦ 2025년 01월 03일 21:23 ¦ Friday
(1 row)

  to_date   ¦      to_timestamp
------------+------------------------
 2026-10-23 ¦ 2026-10-23 15:30:00+09
(1 row)
```

> ■ 참고 **now() 는 트랜잭션 시작 시각**
> 한 트랜잭션 안에서는 `now()` 를 몇 번 호출해도 같은 값입니다. 실제 시각이 필요하면 `clock_timestamp()` 를 씁니다. 05-08 의 결과는 교재을 만든 시각이므로 수업 시간에는 다르게 나옵니다.

예제 05-12 · 고객 나이 계산

```
SELECT name, birth_date,
       EXTRACT(YEAR FROM age(date '2026-10-23', birth_date)) AS 만나이
FROM customers
WHERE birth_date IS NOT NULL
ORDER BY birth_date
LIMIT 5;
```

▶ 실행 결과

```
  name  ¦ birth_date ¦ 만나이
--------+------------+--------
 강시우 ¦ 1979-12-25 ¦     46
 황민재 ¦ 1981-07-27 ¦     45
 윤지호 ¦ 1983-06-12 ¦     43
 박도윤 ¦ 1985-11-02 ¦     40
 오건우 ¦ 1987-05-05 ¦     39
(5 rows)
```

조건식 - CASE, COALESCE, NULLIF

```
CASE WHEN 조건1 THEN 결과1          CASE 식
     WHEN 조건2 THEN 결과2               WHEN 값1 THEN 결과1
     ELSE 기본값                         WHEN 값2 THEN 결과2
END                                      ELSE 기본값 END
```

CASE 는 위에서부터 차례로 검사해 **처음 참인 WHEN** 의 결과를 돌려줍니다. 그래서 범위 조건은 큰 값부터 씁니다(05-13).

예제 05-13 · CASE - 조건에 따라 다른 값 (검색 CASE)

```
SELECT name, price,
       CASE
           WHEN price >= 1000000 THEN '고가'
           WHEN price >= 100000  THEN '중가'
           ELSE '저가'
       END AS 가격대
FROM products
WHERE product_id IN (1, 5, 8, 14);
```

▶ 실행 결과

```
      name      ¦  price  ¦ 가격대
----------------+---------+--------
 울트라북 14    ¦ 1290000 ¦ 고가
 스마트폰 미니  ¦  690000 ¦ 중가
 27인치 모니터  ¦  329000 ¦ 중가
 수제 쿠키 세트 ¦   15000 ¦ 저가
(4 rows)
```

예제 05-14 · CASE - 값 비교형 (단순 CASE)

```
SELECT order_id, status,
       CASE status
           WHEN 'DELIVERED' THEN '배송완료'
           WHEN 'CANCELLED' THEN '취소'
           WHEN 'SHIPPED'   THEN '배송중'
           ELSE '처리중'
       END AS 상태
FROM orders
ORDER BY order_id DESC
LIMIT 5;
```

▶ 실행 결과

```
 order_id ¦ status  ¦  상태
----------+---------+--------
      120 ¦ PAID    ¦ 처리중
      119 ¦ PAID    ¦ 처리중
      118 ¦ PAID    ¦ 처리중
      117 ¦ SHIPPED ¦ 배송중
      116 ¦ SHIPPED ¦ 배송중
(5 rows)
```

예제 05-15 · NULL 처리 함수 - COALESCE, NULLIF

```
SELECT name, COALESCE(phone, '(번호 없음)') AS phone
FROM customers WHERE customer_id IN (2, 3, 7);
-- NULLIF(a, b): a = b 이면 NULL → 0 으로 나누기 방지
SELECT 100 / NULLIF(0, 0) AS 안전한나눗셈;
```

▶ 실행 결과

```
  name  ¦     phone
--------+---------------
 이서연 ¦ 010-1234-0002
 박도윤 ¦ (번호 없음)
 조수아 ¦ (번호 없음)
(3 rows)

 안전한나눗셈
--------------

(1 row)
```

예제 05-16 · GREATEST / LEAST

```
SELECT GREATEST(3, 7, 5) AS 최대, LEAST(3, 7, 5) AS 최소,
       GREATEST(price - 50000, 0) AS 쿠폰적용가
FROM products WHERE product_id = 6;
```

▶ 실행 결과

```
 최대 ¦ 최소 ¦ 쿠폰적용가
------+------+------------
    7 ¦    3 ¦          0
(1 row)
```

■ Q&A; 자주 묻는 질문

- **Q. CASE 에서 ELSE 를 빼면?** 어느 WHEN 에도 맞지 않으면 NULL 이 됩니다.
- **Q. 함수 목록을 다 외워야 하나요?** 아닙니다. psql 의 `\df *trim*` 이나 공식 문서 "Functions and Operators" 장을 찾아보는 법을 알려 주는 것이 더 중요합니다.

실습 문제

번호	문제
05-Q1	고객 이름과 이메일 도메인(@ 뒤)을 출력하고, 이메일 아이디는 앞 2글자만 보이고 나머지는 * 로 가리세요 (고객 3명)
05-Q2	상품 가격을 '1,290,000원' 형식의 문자열로 출력하세요 (노트북 분류 4)
05-Q3	직원의 근속 연수(2026-10-23 기준, 만)를 구하고 근속 10년 이상이면 '장기근속', 아니면 '일반'으로 표시하세요
05-Q4	주문을 요일별(월~일)로 집계하세요. 요일은 to_char 로 이름을 출력하고 ISO 요일 번호(1=월) 순으로 정렬
05-Q5	전화번호가 없으면 '미등록', 생일이 없으면 '알수없음'을 표시하세요 (해당 고객만)

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 비율 계산은 `100.0 *`, 날짜 범위 집계는 `date_trunc`, 서식은 `to_char`.
- NULL 대체는 `COALESCE`, 0 나누기 방지는 `NULLIF(x, 0)`.
- CASE 는 위에서부터 첫 번째로 참인 조건을 쓴다.

06장 · 집계와 그룹화 - GROUP BY, HAVING

학습 목표

- 집계 함수(count, sum, avg, min, max)를 사용하고 NULL 처리 방식을 설명할 수 있다.
- GROUP BY 로 그룹별 통계를 내고 HAVING 으로 그룹을 거를 수 있다.
- FILTER 절, string_agg 로 보고서 형태의 결과를 만들 수 있다.
- [심화] ROLLUP / GROUPING SETS 로 소계·총계를 만들 수 있다.

예제 파일: `examples/06_aggregate.sql`

집계 함수

예제 06-01 · 집계 함수 기본

```
SELECT count(*)      AS 상품수,
       sum(stock)    AS 총재고,
       avg(price)    AS 평균가,
       round(avg(price)) AS 평균가_반올림,
       min(price)    AS 최저가,
       max(price)    AS 최고가
FROM products;
```

▶ 실행 결과

```
 상품수 ¦ 총재고 ¦       평균가        ¦ 평균가_반올림 ¦ 최저가 ¦ 최고가
--------+--------+---------------------+--------------+--------+---------
     20 ¦   1023 ¦ 387250.000000000000 ¦        387250 ¦  12000 ¦ 2490000
(1 row)
```

예제 06-02 · count(*) vs count(컬럼) vs count(DISTINCT 컬럼)

```
SELECT count(*)              AS 전체행,
       count(phone)          AS 전화있음,      -- NULL 은 세지 않음
       count(DISTINCT city)  AS 도시수
FROM customers;
```

▶ 실행 결과

```
 전체행 ¦ 전화있음 ¦ 도시수
--------+----------+--------
     20 ¦       16 ¦      7
(1 row)
```

> ■ 핵심 포인트 **count 세 가지**
> `count(*)` 는 행 수, `count(컬럼)` 은 NULL 이 아닌 값의 수, `count(DISTINCT 컬럼)` 은 서로 다른 값의 수입니다. 고객 20명 중 전화번호가 있는 16명이 바로 이 차이입니다. `sum`, `avg` 등 다른 집계 함수도 **NULL 은 건너뜁니다** (avg 는 NULL 을 0 으로 치지 않음).

GROUP BY

GROUP BY 는 같은 값끼리 행을 묶어 **그룹마다 한 행**으로 만듭니다. 그 결과 SELECT 에는 "그룹마다 하나로 정해지는 값"만 올 수 있습니다.

예제 06-03 · GROUP BY - 그룹별 집계

```
SELECT city, count(*) AS 고객수
FROM customers
GROUP BY city
ORDER BY 고객수 DESC, city;
```

▶ 실행 결과

```
 city ¦ 고객수
------+--------
 서울 ¦      7
 부산 ¦      3
 광주 ¦      2
 대구 ¦      2
 대전 ¦      2
 수원 ¦      2
 인천 ¦      2
(7 rows)
```

예제 06-04 · 여러 컬럼으로 그룹화

```
SELECT status, EXTRACT(YEAR FROM order_date) AS 연도, count(*) AS 주문수
FROM orders
GROUP BY status, EXTRACT(YEAR FROM order_date)
ORDER BY 연도, status;
```

▶ 실행 결과

```
  status   ¦ 연도 ¦ 주문수
-----------+------+--------
 CANCELLED ¦ 2025 ¦     13
 DELIVERED ¦ 2025 ¦     68
 CANCELLED ¦ 2026 ¦      2
 DELIVERED ¦ 2026 ¦     32
 PAID      ¦ 2026 ¦      3
 SHIPPED   ¦ 2026 ¦      2
(6 rows)
```

예제 06-05 · GROUP BY 규칙 위반

```
-- SELECT 에는 GROUP BY 컬럼 또는 집계 함수만 올 수 있다
SELECT city, name, count(*) FROM customers GROUP BY city;
```

▶ 실행 결과

```
ERROR:  column "customers.name" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT city, name, count(*) FROM customers GROUP BY city;
                     ^
```

■ 주의 **GROUP BY 규칙 위반**

06-05 처럼 그룹에 속한 여러 고객 중 누구의 이름을 보여 줘야 할지 정할 수 없으므로 오류가 납니다. MySQL 의 느슨한 모드에 익숙하면 자주 막히는 부분입니다. 해결법은 ① GROUP BY 에 추가 ② 집계 함수로 감싸기(`string_agg(name, ',')`, `min(name)`). 단, **기본 키로 GROUP BY 하면 그 테이블의 다른 컬럼은 함께 써도 됩니다**(함수 종속).

매출 집계 - 주문 상세(order_items)가 기준

매출은 수량 × 판매 단가 의 합입니다. 상품 가격(`products.price`)은 바뀔 수 있으므로 **주문 당시 가격** `order_items.unit_price` 를 써야 합니다(11장 모델링에서 다시 다룸).

예제 06-06 · 매출 계산 - 주문 상세(order_items)로 상품별 판매량/매출

```
SELECT product_id,
       sum(quantity)              AS 판매수량,
       sum(quantity * unit_price) AS 매출
FROM order_items
GROUP BY product_id
ORDER BY 매출 DESC
LIMIT 5;
```

▶ 실행 결과

```
 product_id ¦ 판매수량 ¦   매출
------------+----------+----------
          1 ¦       18 ¦ 23091000
          4 ¦       18 ¦ 20470000
          2 ¦       18 ¦ 17424000
          5 ¦       26 ¦ 17388000
          8 ¦       17 ¦  5560100
(5 rows)
```

HAVING - 그룹 거르기

구분	WHERE	HAVING
실행 시점	그룹을 만들기 **전** (행 단위)	그룹을 만든 **후** (그룹 단위)
집계 함수	사용 불가	사용 가능
예	취소 주문 제외	주문이 10건 이상인 고객만

예제 06-07 · HAVING - 그룹에 조건 걸기 (WHERE 는 그룹 전, HAVING 은 그룹 후)

```
SELECT customer_id, count(*) AS 주문수
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY customer_id
HAVING count(*) >= 10
ORDER BY 주문수 DESC;
```

▶ 실행 결과

```
 customer_id ¦ 주문수
-------------+--------
           8 ¦     14
           3 ¦     12
           1 ¦     12
          10 ¦     11
           5 ¦     10
(5 rows)
```

06-08 은 두 테이블을 합쳐서 집계합니다. `FROM orders o, order_items oi WHERE o.order_id = oi.order_id` 는 7장에서 배울 JOIN 의 옛 문법입니다. "주문과 주문 상세를 짝지어 붙인다" 정도로 소개하고 넘어갑니다.

예제 06-08 · 월별 매출 추이 (date_trunc 활용)

```
SELECT to_char(date_trunc('month', o.order_date), 'YYYY-MM') AS 월,
       count(DISTINCT o.order_id)          AS 주문수,
       sum(oi.quantity * oi.unit_price)    AS 매출
FROM orders o, order_items oi
WHERE o.order_id = oi.order_id
  AND o.status <> 'CANCELLED'
  AND o.order_date >= '2026-01-01'
GROUP BY 1
ORDER BY 1;
```

▶ 실행 결과

```
   월    ¦ 주문수 ¦   매출
---------+--------+----------
 2026-01 ¦      9 ¦  8668300
 2026-02 ¦      4 ¦  2023000
 2026-03 ¦      3 ¦  6504000
 2026-04 ¦      5 ¦  6616000
 2026-05 ¦      6 ¦  2584100
 2026-06 ¦     10 ¦ 10031000
(6 rows)
```

조건부 집계와 문자열 집계

예제 06-09 · 조건부 집계 - FILTER 절과 CASE

```
SELECT count(*)                                          AS 전체,
       count(*) FILTER (WHERE status = 'DELIVERED')      AS 배송완료,
       count(*) FILTER (WHERE status = 'CANCELLED')      AS 취소,
       sum(CASE WHEN emp_id IS NULL THEN 1 ELSE 0 END)   AS 온라인주문
FROM orders;
```

▶ 실행 결과

```
 전체 ¦ 배송완료 ¦ 취소 ¦ 온라인주문
------+----------+------+------------
  120 ¦      100 ¦   15 ¦         44
(1 row)
```

예제 06-10 · 문자열/배열 집계 - string_agg, array_agg

```
SELECT city,
       string_agg(name, ', ' ORDER BY name) AS 고객명단,
       array_agg(customer_id ORDER BY customer_id) AS id목록
FROM customers
GROUP BY city
ORDER BY city
LIMIT 4;
```

▶ 실행 결과

```
 city ¦        고객명단         ¦  id목록
------+------------------------+-----------
 광주 ¦ 안채원, 조수아          ¦ {7,17}
 대구 ¦ 신우진, 최지우          ¦ {4,14}
 대전 ¦ 송현우, 윤지호          ¦ {8,18}
 부산 ¦ 이서연, 한소율, 홍길동 ¦ {2,11,20}
(4 rows)
```

■ TIP FILTER 는 피벗(교차표)의 기본기

`count(*) FILTER (WHERE 조건)` 를 여러 개 나열하면 상태별·월별 교차표를 한 쿼리로 만들 수 있습니다. 표준 SQL 이며, 다른 DB 에서는 `sum(CASE WHEN 조건 THEN 1 ELSE 0 END)` 로 같은 결과를 냅니다(06-09 의 마지막 컬럼).

[심화] 소계와 총계 - ROLLUP, GROUPING SETS

구문	만들어지는 그룹
`GROUP BY ROLLUP (a, b)`	(a, b), (a), () → 상세 + a별 소계 + 총계
`GROUP BY CUBE (a, b)`	(a, b), (a), (b), () → 가능한 모든 조합
`GROUP BY GROUPING SETS ((a), (b), ())`	지정한 그룹만

예제 06-11 · [심화] ROLLUP - 소계와 총계

```
SELECT COALESCE(location, '[전체]') AS 근무지,
       COALESCE(job_title, '[소계]') AS 직무,
       count(*) AS 인원, sum(salary) AS 연봉합
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
WHERE location <> '서울'
GROUP BY ROLLUP (location, job_title)
ORDER BY location NULLS LAST, job_title NULLS LAST;
```

▶ 실행 결과

```
 근무지 ┆       직무        ┆ 인원 ┆ 연봉합
--------+-------------------+------+--------
 인천   ┆ 물류 담당         ┆    1 ┆   4200
 인천   ┆ 물류팀장          ┆    1 ┆   7600
 인천   ┆ [소계]            ┆    2 ┆  11800
 판교   ┆ 백엔드 개발자     ┆    1 ┆   6800
 판교   ┆ 주니어 개발자     ┆    1 ┆   4500
 판교   ┆ 프론트엔드 개발자 ┆    1 ┆   6100
 판교   ┆ CTO               ┆    1 ┆  12000
 판교   ┆ [소계]            ┆    4 ┆  29400
 [전체] ┆ [소계]            ┆    6 ┆  41200
(9 rows)
```

예제 06-12 · [심화] GROUPING SETS - 여러 기준의 집계를 한 번에

```
SELECT city, grade, count(*)
FROM customers
WHERE city IN ('서울', '부산')
GROUP BY GROUPING SETS ((city), (grade), ())
ORDER BY city NULLS LAST, grade NULLS LAST;
```

▶ 실행 결과

```
 city ┆ grade  ┆ count
------+--------+-------
 부산 ┆        ┆     3
 서울 ┆        ┆     7
      ┆ BRONZE ┆     4
      ┆ GOLD   ┆     2
      ┆ SILVER ┆     3
      ┆ VIP    ┆     1
      ┆        ┆    10
(7 rows)
```

■ 참고 **06-11 의 JOIN**

근무지(location)는 departments 테이블에 있어 JOIN 을 사용했습니다. 소계 행에서는 그룹에서 빠진 컬럼이 NULL 이 되므로 `COALESCE` 로 '[소계]', '[전체]' 를 표시했습니다. 원래 데이터에 NULL 이 있으면 구분이 안 되므로 그때는 `GROUPING(컬럼)` 함수(소계 행이면 1)를 씁니다.

예제 06-13 · [심화] 통계 함수

```
SELECT round(stddev(salary), 1)                          AS 표준편차,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS 중앙값,
       mode() WITHIN GROUP (ORDER BY dept_id)            AS 최빈부서
FROM employees;
```

▶ 실행 결과

```
 표준편차 ¦ 중앙값 ¦ 최빈부서
----------+--------+----------
   3341.7 ¦   6100 ¦        2
(1 row)
```

실습 문제

번호	문제
06-Q1	고객 등급(grade)별 고객 수와 평균 나이(2026-10-23 기준, 소수 1자리)를 구하세요
06-Q2	배송 도시(ship_city)별 주문 수가 15건 이상인 도시만, 주문 수 내림차순으로 조회하세요
06-Q3	2025년 분기별 주문 수와 취소 건수, 취소율(%)을 한 번에 구하세요
06-Q4	리뷰 평점별 개수와 평점 5점 리뷰가 1개 이상인 상품 수를 구하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 집계 함수는 NULL 을 무시한다. count(*) 만 예외.
- GROUP BY 후 SELECT 에는 그룹 컬럼과 집계 함수만.
- 행 조건은 WHERE, 그룹 조건은 HAVING. 조건부 집계는 FILTER.

07장 · JOIN - 여러 테이블 연결하기

학습 목표

- 정규화된 테이블을 JOIN 으로 연결해야 하는 이유를 설명할 수 있다.
- INNER / LEFT / RIGHT / FULL OUTER / CROSS / SELF JOIN 의 결과 차이를 설명할 수 있다.
- 3개 이상의 테이블을 조인하고 GROUP BY 와 결합할 수 있다.
- LEFT JOIN 에서 조건을 ON 과 WHERE 중 어디에 둘지 판단할 수 있다.

예제 파일: `examples/07_join.sql`

조인의 원리

주문(orders)에는 고객 번호(`customer_id`)만 있고 고객 이름은 customers 에 있습니다. **JOIN 은 두 테이블에서 조건(ON)이 맞는 행끼리 옆으로 이어 붙이는 연산**입니다.

```
SELECT ...
FROM   A
[INNER ¦ LEFT ¦ RIGHT ¦ FULL] JOIN B ON A.키 = B.키
JOIN   C ON ...
```

조인	결과에 남는 행	짝이 없을 때
`INNER JOIN` (= `JOIN`)	양쪽 모두 짝이 있는 행	버림
`LEFT [OUTER] JOIN`	왼쪽 테이블의 모든 행	오른쪽 컬럼을 NULL 로
`RIGHT [OUTER] JOIN`	오른쪽 테이블의 모든 행	왼쪽 컬럼을 NULL 로
`FULL [OUTER] JOIN`	양쪽 모든 행	없는 쪽을 NULL 로
`CROSS JOIN`	모든 조합 (행 수 = A × B)	조건 없음

■ 핵심 포인트 조인 결과를 손으로 그려 보기
조인을 벤 다이어그램으로만 이해하면 1:N 조인에서 **행이 늘어나는** 현상을 놓치기 쉽습니다. 종이에 employees 5행, departments 3행을 그리고 ON 조건이 맞는 행끼리 선으로 이어 결과표를 만들어 보세요. 부서 없는 직원(권나래)과 직원 없는 부서가 어떻게 되는지 보면 OUTER JOIN 이 자연스럽게 이해됩니다.

INNER JOIN

예제 07-01 · INNER JOIN - 직원과 부서 이름

```
SELECT e.emp_id, e.name, d.name AS 부서명
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
ORDER BY e.emp_id
LIMIT 6;
```

▶ 실행 결과

```
 emp_id ¦  name  ¦  부서명
--------+--------+----------
      1 ¦ 김대표 ¦ 경영지원
      2 ¦ 이영업 ¦ 영업
      3 ¦ 박마케 ¦ 마케팅
      4 ¦ 최개발 ¦ 개발
      5 ¦ 정물류 ¦ 물류
      6 ¦ 강민수 ¦ 영업
(6 rows)
```

예제 07-02 · INNER JOIN 은 짝이 없는 행을 버린다 (부서 없는 인턴이 빠짐)

```
SELECT count(*) AS 전체직원 FROM employees;
SELECT count(*) AS 조인결과 FROM employees e JOIN departments d ON e.dept_id = d.dept_id;
```

▶ 실행 결과

```
 전체직원
----------
       15
(1 row)

 조인결과
----------
       14
(1 row)
```

테이블 별칭(e, d)은 짧게 짓고, 두 테이블에 같은 이름의 컬럼(name)이 있으면 반드시 별칭.컬럼 으로 구분합니다.

OUTER JOIN

예제 07-03 · LEFT JOIN - 왼쪽 테이블은 모두 남긴다

```
SELECT e.name, e.job_title, d.name AS 부서명
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id >= 13;
```

▶ 실행 결과

```
  name  ¦ job_title ¦ 부서명
--------+-----------+--------
 서유나 ¦ 물류 담당 ¦ 물류
 신동현 ¦ 영업사원  ¦ 영업
 권나래 ¦ 인턴      ¦
(3 rows)
```

예제 07-04 · LEFT JOIN + IS NULL - 주문한 적 없는 고객 찾기

```
SELECT c.customer_id, c.name, c.joined_at::date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.customer_id;
```

▶ 실행 결과

```
 customer_id ¦  name  ¦ joined_at
-------------+--------+------------
          19 ¦ 류다은 ¦ 2026-02-10
          20 ¦ 홍길동 ¦ 2026-05-01
(2 rows)
```

■ TIP **"없는 것 찾기" 패턴**

`LEFT JOIN ... WHERE 오른쪽.키 IS NULL` 은 주문 없는 고객, 판매되지 않은 상품, 리뷰 없는 주문 등 **"~하지 않은 것"** 을 찾는 대표 패턴입니다. 8장의 `NOT EXISTS` 와 같은 결과를 냅니다.

여러 테이블 조인과 집계

예제 07-05 · 3개 이상 테이블 조인 - 주문 상세 내역

```
SELECT o.order_id, c.name AS 고객, p.name AS 상품, oi.quantity AS 수량,
       oi.quantity * oi.unit_price AS 금액
FROM orders o
JOIN customers   c  ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products    p  ON oi.product_id = p.product_id
WHERE o.order_id BETWEEN 1 AND 3
ORDER BY o.order_id, p.name;
```

▶ 실행 결과

```
 order_id ¦  고객  ¦        상품        ¦ 수량 ¦  금액
----------+--------+--------------------+------+---------
        1 ¦ 임준서 ¦ 여성 니트          ¦    1 ¦   49000
        2 ¦ 임준서 ¦ 27인치 모니터      ¦    2 ¦  658000
        2 ¦ 임준서 ¦ 남성 셔츠          ¦    1 ¦   35100
        2 ¦ 임준서 ¦ 비즈니스 노트북 15 ¦    3 ¦ 2673000
        2 ¦ 임준서 ¦ 유기농 녹차        ¦    1 ¦   12000
        3 ¦ 윤지호 ¦ 무선 마우스        ¦    1 ¦   29000
        3 ¦ 윤지호 ¦ 스마트폰 S25       ¦    2 ¦ 2300000
        3 ¦ 윤지호 ¦ USB-C 허브         ¦    1 ¦   45000
(8 rows)
```

예제 07-06 · JOIN + GROUP BY - 고객별 총 구매액 TOP 5

```
SELECT c.name, c.grade, count(DISTINCT o.order_id) AS 주문수,
       sum(oi.quantity * oi.unit_price) AS 총구매액
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.customer_id, c.name, c.grade
ORDER BY 총구매액 DESC
LIMIT 5;
```

▶ 실행 결과

```
  name  ¦ grade  ¦ 주문수 ¦ 총구매액
--------+--------+--------+----------
 임준서 ¦ GOLD   ¦     11 ¦ 20481300
 정하은 ¦ GOLD   ¦     10 ¦  9001100
 윤지호 ¦ VIP    ¦     14 ¦  8927000
```

```
 김민준 ¦ VIP    ¦     12 ¦  7692400
 강시우 ¦ SILVER ¦      4 ¦  7607000
(5 rows)
```

예제 07-07 · 카테고리별 매출 (상품 → 분류 조인)

```
SELECT cat.name AS 분류, sum(oi.quantity * oi.unit_price) AS 매출
FROM order_items oi
JOIN products p     ON oi.product_id = p.product_id
JOIN categories cat ON p.category_id = cat.category_id
GROUP BY cat.name
ORDER BY 매출 DESC;
```

▶ 실행 결과

```
    분류      ¦   매출
--------------+----------
 노트북        ¦ 40515000
 스마트폰      ¦ 37858000
 주변기기      ¦ 10126200
 게이밍 노트북 ¦  2490000
 여성의류      ¦  1970400
 남성의류      ¦  1710100
 음료          ¦  1396800
 간식          ¦  1246100
(8 rows)
```

■ **주의** **1:N 조인 후 집계하면 값이 부풀려진다**
07-06 에서 주문 수를 `count(*)` 로 세면 주문 상세 행 수(상품 개수)가 세어집니다. 그래서 `count(DISTINCT o.order_id)` 를 썼습니다. 조인 후 집계 결과가 이상하게 크면 **조인으로 행이 몇 배가 되었는지**를 먼저 의심하세요.

ON 과 WHERE - LEFT JOIN 의 함정

예제 07-08 · LEFT JOIN 에서 조건 위치 주의 - ON 과 WHERE 의 차이

```
-- (의도) 모든 상품과, 리뷰가 있다면 그중 5점짜리 개수
SELECT p.product_id, count(r.review_id) AS 만점리뷰
FROM products p
LEFT JOIN reviews r ON p.product_id = r.product_id
WHERE r.rating = 5                        -- WHERE 에 두면 LEFT JOIN 이 INNER 처럼 됨
GROUP BY p.product_id ORDER BY p.product_id LIMIT 5;
SELECT p.product_id, count(r.review_id) AS 만점리뷰
FROM products p
LEFT JOIN reviews r ON p.product_id = r.product_id AND r.rating = 5   -- ON 에 둬야 함
GROUP BY p.product_id ORDER BY p.product_id LIMIT 5;
```

▶ 실행 결과

```
 product_id ¦ 만점리뷰
------------+----------
          1 ¦        1
          5 ¦        2
          6 ¦        2
          7 ¦        2
          8 ¦        1
(5 rows)

 product_id ¦ 만점리뷰
------------+----------
          1 ¦        1
          2 ¦        0
          3 ¦        0
          4 ¦        0
          5 ¦        2
```

```
(5 rows)
```

첫 번째 쿼리는 LEFT JOIN 후 WHERE 에서 `r.rating = 5` 를 검사하므로, 리뷰가 없어 NULL 이 된 상품(2, 3, 4번)이 걸러져 INNER JOIN 과 같아집니다. **오른쪽 테이블에 대한 조건은 ON 에** 써야 왼쪽 행이 모두 보존됩니다.

예제 07-09 · RIGHT JOIN 과 FULL OUTER JOIN

```
SELECT d.name AS 부서, e.name AS 직원
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id AND e.salary >= 9000;
SELECT d.name AS 부서, e.name AS 직원
FROM (SELECT * FROM employees WHERE salary < 3000 OR dept_id = 5) e
FULL OUTER JOIN (SELECT * FROM departments WHERE dept_id IN (4, 5)) d
     ON e.dept_id = d.dept_id;
```

▶ 실행 결과

```
   부서   ¦  직원
----------+--------
 경영지원 ¦ 김대표
 영업     ¦ 이영업
 개발     ¦ 최개발
 물류     ¦
 마케팅   ¦
(5 rows)

 부서 ¦  직원
------+--------
 물류 ¦ 정물류
 물류 ¦ 서유나
      ¦ 권나래
 개발 ¦
(4 rows)
```

SELF JOIN, CROSS JOIN, 비등가 조인

같은 테이블을 별칭을 달리해 두 번 쓰면 SELF JOIN 입니다. 직원-상사, 분류-상위분류처럼 자기 자신을 참조하는 관계에서 씁니다.

예제 07-10 · SELF JOIN - 직원과 상사 이름

```
SELECT e.name AS 직원, e.job_title, m.name AS 상사
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
ORDER BY e.emp_id
LIMIT 8;
```

▶ 실행 결과

```
  직원  ¦ job_title  ¦  상사
--------+------------+--------
 김대표 ¦ 대표이사   ¦
 이영업 ¦ 영업본부장 ¦ 김대표
 박마케 ¦ 마케팅팀장 ¦ 김대표
 최개발 ¦ CTO        ¦ 김대표
 정물류 ¦ 물류팀장   ¦ 김대표
 강민수 ¦ 영업대리   ¦ 이영업
 조은지 ¦ 영업과장   ¦ 이영업
 윤서준 ¦ 영업사원   ¦ 조은지
(8 rows)
```

예제 07-11 · CROSS JOIN - 모든 조합 만들기

```
SELECT s.size, c.color
FROM (VALUES ('S'), ('M'), ('L')) AS s(size)
CROSS JOIN (VALUES ('흰색'), ('검정')) AS c(color);
```

▶ 실행 결과

```
 size ¦ color
------+-------
 S    ¦ 흰색
 S    ¦ 검정
 M    ¦ 흰색
 M    ¦ 검정
 L    ¦ 흰색
 L    ¦ 검정
(6 rows)
```

예제 07-12 · USING 과 NATURAL JOIN (컬럼 이름이 같을 때)

```
SELECT order_id, product_id, quantity, customer_id
FROM order_items JOIN orders USING (order_id)
LIMIT 3;
-- NATURAL JOIN 은 이름이 같은 모든 컬럼으로 조인한다 → 의도치 않은 결과 위험!
SELECT count(*) AS natural_join_결과
FROM employees NATURAL JOIN departments;        -- dept_id 와 name 둘 다로 조인됨
```

▶ 실행 결과

```
 order_id ¦ product_id ¦ quantity ¦ customer_id
----------+------------+----------+-------------
        1 ¦         13 ¦        1 ¦          10
        2 ¦          2 ¦        3 ¦          10
        2 ¦          8 ¦        2 ¦          10
(3 rows)

 natural_join_결과
-------------------
                 0
(1 row)
```

예제 07-13 · 비등가 조인 - 가격 구간표와 조인

```
SELECT p.name, p.price, g.label
FROM products p
JOIN (VALUES (0, 50000, '저가'), (50000, 500000, '중가'), (500000, 99999999, '고가'))
     AS g(lo, hi, label)
  ON p.price >= g.lo AND p.price < g.hi
WHERE p.product_id IN (3, 8, 15);
```

▶ 실행 결과

```
       name        ¦  price  ¦ label
-------------------+---------+-------
 게이밍 노트북 X17 ¦ 2490000 ¦ 고가
 27인치 모니터     ¦  329000 ¦ 중가
 견과류 믹스       ¦   22000 ¦ 저가
(3 rows)
```

■ 주의 **NATURAL JOIN 은 쓰지 않는다**

07-12 의 NATURAL JOIN 은 `dept_id` 뿐 아니라 우연히 이름이 같은 `name` 컬럼까지 조건으로 써서 0건이 되었습니다. 나중에 테이블에 같은 이름의 컬럼이 추가되면 쿼리 결과가 조용히 바뀝니다. `USING (컬럼)` 은 조인 컬럼을 명시하므로 써도 괜찮습니다.

■ Q&A; 자주 묻는 질문

- **Q. JOIN 순서가 성능에 영향을 주나요?** INNER JOIN 은 옵티마이저가 순서를 알아서 정합니다. 13장 EXPLAIN 에서 확인합니다. OUTER JOIN 은 순서에 따라 결과가 달라지므로 의미대로 써야 합니다.
- **Q. `FROM a, b WHERE a.id = b.id` 와 JOIN 은 같은가요?** INNER JOIN 과 결과가 같은 옛 문법입니다. 조인 조건을 빠뜨리면 CROSS JOIN 이 되는 위험이 있어 명시적 JOIN 을 권장합니다.

실습 문제

번호	문제
07-Q1	상품명, 분류명, 상위 분류명을 함께 출력하세요 (상위 분류가 없으면 NULL, 상품 1~5)
07-Q2	한 번도 팔리지 않은 상품을 LEFT JOIN 으로 찾으세요
07-Q3	부서별 직원 수와 평균 연봉을 구하되, 직원이 없는 부서도 0명으로 나오게 하세요 (부서 없는 직원 제외)
07-Q4	자기 상사보다 입사일이 빠른 직원이 있는지 SELF JOIN 으로 확인하세요
07-Q5	도시별 매출 TOP 3 (주문의 배송 도시 기준, 취소 제외)

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- INNER 는 짝이 있는 행만, LEFT 는 왼쪽 전체 + 짝 없으면 NULL.
- 1:N 조인 후 집계할 때는 `count(DISTINCT ...)` 로 중복을 주의.
- LEFT JOIN 의 오른쪽 테이블 조건은 ON 에 둔다.

08장 · 서브쿼리 - 쿼리 안의 쿼리

학습 목표

- 스칼라·다중 행·상관 서브쿼리를 구분하고 알맞은 위치(SELECT/FROM/WHERE)에 쓸 수 있다.
- IN, EXISTS, ANY, ALL 을 사용하고 NOT IN 의 NULL 함정을 피할 수 있다.
- 서브쿼리로 UPDATE / DELETE 대상을 정할 수 있다.
- [심화] LATERAL 로 그룹별 TOP-N 을 구할 수 있다.

예제 파일: `examples/08_subquery.sql`

서브쿼리의 종류

종류	반환	위치	예
스칼라 서브쿼리	1행 1열 (값 하나)	SELECT, WHERE 의 비교	`price > (SELECT avg(price) ...)`
다중 행 서브쿼리	여러 행 1열	`IN`, `ANY`, `ALL`	`id IN (SELECT ...)`
상관 서브쿼리	바깥 행마다 다시 실행	WHERE, SELECT, EXISTS	`WHERE x > (SELECT ... WHERE 안.k = 밖.k)`
인라인 뷰	테이블	FROM (별칭 필수)	`FROM (SELECT ...) t`

예제 08-01 · 스칼라 서브쿼리 - 평균보다 비싼 상품

```
SELECT name, price
FROM products
WHERE price > (SELECT avg(price) FROM products)
ORDER BY price DESC;
```

▶ 실행 결과

```
       name        |  price
-------------------+---------
 게이밍 노트북 X17  | 2490000
 울트라북 14        | 1290000
 스마트폰 S25       | 1150000
 비즈니스 노트북 15 |  990000
 스마트폰 미니      |  690000
(5 rows)
```

예제 08-02 · SELECT 절의 스칼라 서브쿼리 - 전체 평균과 비교

```
SELECT name, price,
       (SELECT round(avg(price)) FROM products) AS 전체평균,
       price - (SELECT round(avg(price)) FROM products) AS 차이
FROM products
WHERE category_id = 4;
```

▶ 실행 결과

```
       name        |  price  | 전체평균 |  차이
-------------------+---------+----------+--------
 울트라북 14       | 1290000 |   387250 | 902750
 비즈니스 노트북 15 |  990000 |   387250 | 602750
(2 rows)
```

예제 08-03 · IN 서브쿼리 - 노트북 분류를 산 적 있는 고객

```
SELECT name FROM customers
WHERE customer_id IN (
    SELECT o.customer_id
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    WHERE oi.product_id IN (SELECT product_id FROM products WHERE category_id IN (4, 11))
)
ORDER BY name;
```

▶ 실행 결과

```
  name
--------
 권하린
 박도윤
 신우진
 안채원
 오건우
 윤지호
 임준서
 장예린
 정하은
 조수아
 한소율
(11 rows)
```

■ 핵심 포인트 **서브쿼리는 안쪽부터 읽기**
괄호 안의 쿼리만 먼저 선택해 실행 → 결과(예: 387250)를 확인 → 그 값을 바깥 쿼리에 직접 써 넣은 것과 같다고 생각하세요. 08-03 처럼 중첩이 깊을 때도 안쪽부터 하나씩 실행해 보며 읽으면 쉽습니다.

NOT IN 과 NULL

예제 08-04 · NOT IN 과 NULL 함정

```
-- 담당 직원이 한 번도 맡지 않은(=orders.emp_id 에 없는) 직원을 찾으려 했지만…
SELECT count(*) AS not_in_결과 FROM employees
WHERE emp_id NOT IN (SELECT emp_id FROM orders);            -- emp_id 에 NULL 이 있어 0건!
SELECT count(*) AS not_exists_결과 FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.emp_id = e.emp_id);
```

▶ 실행 결과

```
 not_in_결과
-------------
           0
(1 row)

 not_exists_결과
-----------------
              11
(1 row)
```

`emp_id NOT IN (6, 7, NULL, ...)` 은 `emp_id <> 6 AND emp_id <> 7 AND emp_id <> NULL ...` 로 풀리는데, `<> NULL` 이 NULL 이므로 전체가 절대 참이 될 수 없습니다. orders.emp_id 에는 온라인 주문의 NULL 이 있어 결과가 0건이 되었습니다.

■ 주의 **NOT IN 대신 NOT EXISTS**
서브쿼리 결과에 NULL 이 하나라도 섞일 수 있다면 `NOT IN` 은 조용히 0건을 돌려줍니다. **"없는 것 찾기"는** `NOT EXISTS` **나** `LEFT JOIN ... IS NULL` 로 쓰도록 강하게 권합니다. 실무 버그 사례로 자주 나옵니다.

EXISTS 와 상관 서브쿼리

EXISTS (서브쿼리) 는 서브쿼리가 한 행이라도 돌려주면 참입니다. 무엇을 SELECT 하는지는 상관없어 관례적으로 SELECT 1 을 씁니다.

예제 08-05 · EXISTS - 리뷰를 남긴 적이 있는 고객

```
SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (SELECT 1 FROM reviews r WHERE r.customer_id = c.customer_id)
ORDER BY c.customer_id
LIMIT 5;
```

▶ 실행 결과

```
 customer_id ¦  name
-------------+--------
           1 ¦ 김민준
           3 ¦ 박도윤
           4 ¦ 최지우
           5 ¦ 정하은
           6 ¦ 강시우
(5 rows)
```

예제 08-06 · 상관 서브쿼리 - 자기 분류의 평균 가격보다 비싼 상품

```
SELECT p.name, p.category_id, p.price
FROM products p
WHERE p.price > (SELECT avg(p2.price) FROM products p2
                 WHERE p2.category_id = p.category_id)
ORDER BY p.category_id;
```

▶ 실행 결과

```
      name       ¦ category_id ¦  price
-----------------+-------------+---------
 울트라북 14     ¦           4 ¦ 1290000
 스마트폰 S25    ¦           5 ¦ 1150000
 27인치 모니터   ¦           6 ¦  329000
 남성 정장 세트  ¦           7 ¦  299000
 여성 원피스     ¦           8 ¦   79000
 견과류 믹스     ¦           9 ¦   22000
 콜드브루 원액   ¦          10 ¦   18000
 탄산수 24입     ¦          10 ¦   16000
(8 rows)
```

08-06 의 서브쿼리는 바깥 행(p)의 category_id 를 참조하므로 상품마다 다시 계산됩니다. 이것이 **상관 서브쿼리**입니다. 같은 결과를 10장의 윈도우 함수 avg(price) OVER (PARTITION BY category_id) 로도 구할 수 있습니다.

인라인 뷰와 ANY / ALL

예제 08-07 · FROM 절 서브쿼리(인라인 뷰) - 고객 등급별 평균 구매액

```
SELECT grade, count(*) AS 고객수, round(avg(total)) AS 평균구매액
FROM (
    SELECT c.customer_id, c.grade, sum(oi.quantity * oi.unit_price) AS total
    FROM customers c
    JOIN orders o       ON c.customer_id = o.customer_id
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY c.customer_id, c.grade
) t
GROUP BY grade
ORDER BY 평균구매액 DESC;
```

▶ 실행 결과

```
 grade  ¦ 고객수 ¦ 평균구매액
--------+--------+------------
 GOLD   ¦      4 ¦    9647050
 VIP    ¦      2 ¦    8728800
 SILVER ¦      5 ¦    5383680
 BRONZE ¦      7 ¦    2049771
(4 rows)
```

예제 08-08 · ANY / ALL

```
SELECT name, price FROM products
WHERE price > ALL (SELECT price FROM products WHERE category_id = 5);  -- 모든 스마트폰보다 비쌈
SELECT name, price FROM products
WHERE price = ANY (ARRAY[29000, 45000, 12000]);                        -- IN 과 같은 효과
```

▶ 실행 결과

```
       name        ¦  price
-------------------+---------
 울트라북 14       ¦ 1290000
 게이밍 노트북 X17 ¦ 2490000
(2 rows)

    name     ¦ price
-------------+-------
 무선 마우스 ¦ 29000
 USB-C 허브  ¦ 45000
 유기농 녹차 ¦ 12000
(3 rows)
```

표현	의미
`x > ALL (서브쿼리)`	모든 값보다 크다 (= 최댓값보다 크다)
`x > ANY (서브쿼리)`	하나라도보다 크다 (= 최솟값보다 크다)
`x = ANY (서브쿼리 또는 배열)`	`IN` 과 같다

서브쿼리로 데이터 수정

예제 08-09 · 서브쿼리로 UPDATE / DELETE

```
UPDATE products
SET is_active = false
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items)
RETURNING product_id, name;
```

▶ 실행 결과

```
 product_id ┆      name
------------+----------------
         19 ┆ 남성 정장 세트
         20 ┆ 탄산수 24입
(2 rows)

UPDATE 2
```

여기서는 `NOT IN` 을 썼지만 `order_items.product_id` 는 NOT NULL 컬럼이므로 안전합니다. 이처럼 NULL 가능성을 따져 보고 쓰면 됩니다.

예제 08-10 · 스칼라 서브쿼리는 한 행만 반환해야 한다

```
SELECT name FROM products WHERE price = (SELECT price FROM products WHERE category_id = 5);
```

▶ 실행 결과

```
ERROR:  more than one row returned by a subquery used as an expression
```

[심화] LATERAL

일반 FROM 절 서브쿼리는 앞의 테이블을 참조할 수 없지만, `LATERAL` 을 붙이면 **앞 테이블의 각 행마다** 서브쿼리를 실행할 수 있습니다. "분류별 상위 2개", "고객별 최근 주문 3건" 같은 그룹별 TOP-N 에 유용합니다(10장 윈도우 함수 방식과 비교).

예제 08-11 · [심화] LATERAL - 각 분류별 가장 비싼 상품 2개

```
SELECT c.name AS 분류, t.name AS 상품, t.price
FROM categories c
CROSS JOIN LATERAL (
    SELECT p.name, p.price FROM products p
    WHERE p.category_id = c.category_id
    ORDER BY p.price DESC
    LIMIT 2
) t
WHERE c.category_id IN (5, 6, 7);
```

▶ 실행 결과

```
   분류   ┆      상품      ┆  price
----------+----------------+---------
 스마트폰 ┆ 스마트폰 S25   ┆ 1150000
 스마트폰 ┆ 스마트폰 미니  ┆  690000
 주변기기 ┆ 27인치 모니터  ┆  329000
 주변기기 ┆ 기계식 키보드  ┆   89000
 남성의류 ┆ 남성 정장 세트 ┆  299000
 남성의류 ┆ 남성 청바지    ┆   59000
(6 rows)
```

■ Q&A; 자주 묻는 질문

- **Q. 서브쿼리와 JOIN 중 무엇이 빠른가요?** PostgreSQL 옵티마이저는 IN/EXISTS 서브쿼리를 조인(세미 조인)으로 바꿔 실행하는 경우가 많아 대개 비슷합니다. 읽기 쉬운 쪽으로 쓰고, 느리면 EXPLAIN(13장)으로 확인합니다.
- **Q. SELECT 절 스칼라 서브쿼리가 여러 행을 돌려주면?** 08-10 처럼 오류가 납니다. `LIMIT 1` 이나 집계 함수로 한 행을 보장해야 합니다.

실습 문제

번호	문제
08-Q1	가장 비싼 상품과 같은 분류에 속한 상품을 모두 조회하세요
08-Q2	평균 연봉보다 많이 받는 직원의 이름, 연봉, 평균과의 차이를 구하세요
08-Q3	리뷰를 한 번도 남기지 않은 '구매 고객'을 NOT EXISTS 로 찾으세요
08-Q4	각 고객의 마지막 주문일을 상관 서브쿼리로 구하세요 (고객 1~5)

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 서브쿼리는 안쪽부터 읽는다. 스칼라 서브쿼리는 반드시 한 행.
- "없는 것 찾기"는 NOT IN 이 아닌 NOT EXISTS.
- 그룹별 TOP-N 은 LATERAL 또는 윈도우 함수(10장).

09장 · 집합 연산과 CTE - 재귀 쿼리까지

학습 목표

- UNION / UNION ALL / INTERSECT / EXCEPT 로 결과 집합을 합치고 비교할 수 있다.
- WITH(CTE)로 복잡한 쿼리를 단계별로 나누어 작성할 수 있다.
- 재귀 CTE 로 조직도·분류 트리 같은 계층 데이터를 조회할 수 있다.
- [심화] generate_series 와 데이터 변경 CTE 를 활용할 수 있다.

예제 파일: `examples/09_set_cte.sql`

집합 연산

연산	결과	중복
`A UNION B`	합집합	제거
`A UNION ALL B`	합집합	유지 (정렬·비교가 없어 빠름)
`A INTERSECT B`	교집합	제거
`A EXCEPT B`	차집합 (A 에만 있는 것)	제거

규칙: 두 쿼리의 **컬럼 수가 같고 대응하는 자료형이 호환**되어야 합니다. 결과 컬럼 이름은 첫 번째 쿼리를 따르고, `ORDER BY` 는 맨 마지막에 한 번만 씁니다.

예제 09-01 · UNION / UNION ALL - 결과 합치기

```
SELECT city FROM customers WHERE grade = 'VIP'
UNION                                       -- 중복 제거
SELECT ship_city FROM orders WHERE order_id <= 3;
SELECT city FROM customers WHERE grade = 'VIP'
UNION ALL                                   -- 중복 유지 (더 빠름)
SELECT ship_city FROM orders WHERE order_id <= 3;
```

▶ 실행 결과

```
 city
------
 대전
 서울
(2 rows)

 city
------
 서울
 대전
 서울
 서울
 대전
(5 rows)
```

예제 09-02 · INTERSECT / EXCEPT - 교집합, 차집합

```
-- 2025년에도 2026년에도 주문한 고객
SELECT customer_id FROM orders WHERE order_date <  '2026-01-01'
INTERSECT
SELECT customer_id FROM orders WHERE order_date >= '2026-01-01'
ORDER BY 1;
-- 2025년에는 주문했지만 2026년에는 주문하지 않은 고객 (이탈 고객)
SELECT customer_id FROM orders WHERE order_date <  '2026-01-01'
EXCEPT
SELECT customer_id FROM orders WHERE order_date >= '2026-01-01'
ORDER BY 1;
```

▶ 실행 결과

```
 customer_id
-------------
           1
           2
           3
           5
           6
           8
           9
          10
          12
          14
          16
          17
          18
(13 rows)

 customer_id
-------------
           4
           7
          11
          13
          15
(5 rows)
```

예제 09-03 · 집합 연산 규칙 - 컬럼 수와 타입이 맞아야 한다

```
SELECT name, city FROM customers
UNION
SELECT name FROM employees;
```

▶ 실행 결과

```
ERROR:  each UNION query must have the same number of columns
LINE 3: SELECT name FROM employees;
               ^
```

■ 핵심 포인트 UNION ALL 을 기본으로

중복이 생길 수 없거나 중복을 세어야 하는 경우(로그 합치기 등)에는 UNION ALL 을 씁니다. UNION 은 중복 제거를 위해 정렬·해시를 하므로 큰 데이터에서 눈에 띄게 느립니다. 09-02 의 EXCEPT 결과(이탈 고객)는 20장 종합 프로젝트에서 다시 나옵니다.

CTE (Common Table Expression)

```
WITH 이름1 AS (SELECT ...),
     이름2 AS (SELECT ... FROM 이름1 ...)
SELECT ... FROM 이름2 ...;
```

CTE 는 쿼리 안에서만 쓰는 **임시 이름 붙은 결과**입니다. 인라인 뷰와 같은 일을 하지만 위에서 아래로 읽을 수 있고, 같은 결과를 여러 번 참조할 수 있습니다.

예제 09-04 · CTE(WITH) - 쿼리에 이름 붙이기

```
WITH customer_sales AS (
    SELECT o.customer_id, sum(oi.quantity * oi.unit_price) AS total
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.status <> 'CANCELLED'
    GROUP BY o.customer_id
)
SELECT c.name, cs.total
FROM customer_sales cs JOIN customers c ON c.customer_id = cs.customer_id
WHERE cs.total > (SELECT avg(total) FROM customer_sales)
ORDER BY cs.total DESC;
```

▶ 실행 결과

```
  name  ¦  total
--------+----------
 임준서 ¦ 20481300
 정하은 ¦  9001100
 윤지호 ¦  8927000
 김민준 ¦  7692400
 강시우 ¦  7607000
 장예린 ¦  6774300
 신우진 ¦  6275200
 조수아 ¦  5878000
(8 rows)
```

예제 09-05 · CTE 여러 개 연결하기 - 2026년 상반기 분류별 매출 비중

```
WITH sales AS (
    SELECT p.category_id, oi.quantity * oi.unit_price AS amount
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p     ON oi.product_id = p.product_id
    WHERE o.status <> 'CANCELLED'
      AND o.order_date >= '2026-01-01' AND o.order_date < '2026-07-01'
),
by_cat AS (
    SELECT category_id, sum(amount) AS amount FROM sales GROUP BY category_id
)
SELECT c.name, b.amount,
       round(100.0 * b.amount / (SELECT sum(amount) FROM by_cat), 1) AS 비중
FROM by_cat b JOIN categories c ON c.category_id = b.category_id
ORDER BY b.amount DESC;
```

▶ 실행 결과

```
     name      ¦  amount  ¦ 비중
---------------+----------+------
 스마트폰      ¦ 16721000 ¦ 45.9
 노트북        ¦ 12990000 ¦ 35.7
 게이밍 노트북 ¦  2490000 ¦  6.8
 주변기기      ¦  2088000 ¦  5.7
 여성의류      ¦   714100 ¦  2.0
 남성의류      ¦   601500 ¦  1.7
 음료          ¦   462000 ¦  1.3
 간식          ¦   359800 ¦  1.0
(8 rows)
```

> **■ TIP 복잡한 분석 쿼리 작성법**
> "① 원재료 뽑기 → ② 집계 → ③ 비율·순위 계산" 처럼 단계마다 CTE 하나를 만들고, 각 단계를 `SELECT * FROM 단계명` 으로 확인하며 쌓아 가세요. 20장 프로젝트 쿼리가 모두 이 방식입니다.

재귀 CTE

```
WITH RECURSIVE 이름 AS (
    시작 쿼리                          -- ① 앵커: 처음 한 번
    UNION ALL
    SELECT ... FROM 테이블 JOIN 이름   -- ② 재귀: 직전 결과를 이용해 반복
    WHERE 종료 조건                     -- 새 행이 없으면 멈춤
)
SELECT ... FROM 이름;
```

예제 09-06 · 재귀 CTE 기초 - 1부터 5까지

```
WITH RECURSIVE nums(n) AS (
    SELECT 1                       -- 시작(앵커)
    UNION ALL
    SELECT n + 1 FROM nums         -- 반복(재귀)
    WHERE n < 5                    -- 종료 조건
)
SELECT n FROM nums;
```

▶ 실행 결과

```
 n
---
 1
 2
 3
 4
 5
(5 rows)
```

동작 순서: ① 앵커가 1 을 만든다 → ② 직전 결과(1)로 2 → ③ 직전 결과(2)로 3 … → `n < 5` 가 거짓이 되어 새 행이 없으면 종료.

예제 09-07 · 재귀 CTE - 조직도 (상사→부하 계층)

```
WITH RECURSIVE org AS (
    SELECT emp_id, name, job_title, manager_id, 1 AS depth, name::text AS path
    FROM employees WHERE manager_id IS NULL AND dept_id IS NOT NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.job_title, e.manager_id, o.depth + 1,
           o.path || ' > ' || e.name
    FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT repeat('    ', depth - 1) || name AS 조직도, job_title, depth
FROM org
ORDER BY path;
```

▶ 실행 결과

```
       조직도       |     job_title     | depth
--------------------+-------------------+-------
 김대표             | 대표이사          |     1
     박마케         | 마케팅팀장        |     2
         장하늘     | 마케터            |     3
     이영업         | 영업본부장        |     2
         강민수     | 영업대리          |     3
         조은지     | 영업과장          |     3
             신동현 | 영업사원          |     4
```

```
          윤서준 ¦ 영업사원          ¦     4
     정물류      ¦ 물류팀장          ¦     2
         서유나  ¦ 물류 담당         ¦     3
     최개발      ¦ CTO               ¦     2
         임도윤  ¦ 백엔드 개발자     ¦     3
            오세훈 ¦ 주니어 개발자   ¦     4
         한지민  ¦ 프론트엔드 개발자 ¦     3
(14 rows)
```

예제 09-08 · 재귀 CTE - 분류 트리 경로 (게이밍 노트북의 전체 경로)

```
WITH RECURSIVE tree AS (
    SELECT category_id, name, parent_id, name::text AS full_path
    FROM categories WHERE parent_id IS NULL
    UNION ALL
    SELECT c.category_id, c.name, c.parent_id, t.full_path ¦¦ ' > ' ¦¦ c.name
    FROM categories c JOIN tree t ON c.parent_id = t.category_id
)
SELECT category_id, full_path FROM tree WHERE category_id IN (4, 6, 11) ORDER BY category_id;
```

▶ 실행 결과

```
 category_id ¦             full_path
-------------+----------------------------------
           4 ¦ 전자제품 > 노트북
           6 ¦ 전자제품 > 주변기기
          11 ¦ 전자제품 > 노트북 > 게이밍 노트북
(3 rows)
```

■ 핵심 포인트 조직도 쿼리 읽는 순서

09-07 은 ① 대표(manager_id IS NULL)에서 시작 → ② "직전 단계 사람을 상사로 둔 직원" 을 붙이며 깊이(depth)를 1씩 늘림 → ③ 경로(path) 문자열로 정렬하면 트리 순서가 됩니다. 교재 앞쪽 ERD 의 employees 자기참조 화살표와 함께 보세요.

■ 주의 무한 루프

데이터에 순환(A의 상사가 B, B의 상사가 A)이 있으면 재귀 CTE 가 끝나지 않습니다. 깊이 제한(`WHERE depth < 10`)을 두거나 PostgreSQL 14+ 의 `CYCLE` 절을 사용합니다.

[심화] generate_series 와 데이터 변경 CTE

`generate_series(시작, 끝, 간격)` 은 숫자·날짜의 연속 행을 만듭니다. 주문이 없는 날도 0건으로 보여 줘야 하는 일별 리포트에서 LEFT JOIN 의 기준표로 씁니다.

예제 09-09 · [심화] generate_series 로 빈 날짜 채우기 - 2026년 6월 중순 일자별 주문수

```
SELECT d::date AS 날짜, count(o.order_id) AS 주문수
FROM generate_series('2026-06-12'::date, '2026-06-21'::date, interval '1 day') AS d
LEFT JOIN orders o ON o.order_date::date = d::date
GROUP BY d
ORDER BY d;
```

▶ 실행 결과

```
    날짜    ¦ 주문수
------------+--------
 2026-06-12 ¦      0
 2026-06-13 ¦      0
 2026-06-14 ¦      2
 2026-06-15 ¦      0
 2026-06-16 ¦      0
 2026-06-17 ¦      0
 2026-06-18 ¦      2
```

```
 2026-06-19 ¦      0
 2026-06-20 ¦      0
 2026-06-21 ¦      0
(10 rows)
```

CTE 안에서 `INSERT/UPDATE/DELETE ... RETURNING` 을 쓰면 한 문장으로 "지우고 옮기기" 같은 작업을 원자적으로 할 수 있습니다.

예제 09-10 · [심화] 데이터 변경 CTE - 삭제한 행을 다른 테이블로 옮기기

```
CREATE TABLE orders_archive (LIKE orders);
WITH moved AS (
    DELETE FROM orders
    WHERE status = 'CANCELLED' AND order_date < '2025-03-01'
    RETURNING *
)
INSERT INTO orders_archive SELECT * FROM moved;
SELECT order_id, order_date, status FROM orders_archive;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 3
 order_id ¦     order_date      ¦  status
----------+---------------------+-----------
        6 ¦ 2025-01-30 18:45:00 ¦ CANCELLED
        8 ¦ 2025-02-08 18:55:00 ¦ CANCELLED
       12 ¦ 2025-02-19 09:29:00 ¦ CANCELLED
(3 rows)
```

실습 문제

번호	문제
09-Q1	고객 도시와 직원 근무지(부서 location)를 합쳐 중복 없는 도시 목록을 만드세요
09-Q2	CTE 로 상품별 매출을 구한 뒤, 전체 매출의 10% 이상을 차지하는 상품만 출력하세요
09-Q3	재귀 CTE 로 '오세훈' 사원부터 대표까지의 결재 라인을 출력하세요
09-Q4	'전자제품'(1) 아래의 모든 하위 분류에 속한 상품 수를 재귀 CTE 로 구하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 집합 연산은 컬럼 수·타입이 같아야 하고, 기본은 UNION ALL.
- CTE 로 복잡한 쿼리를 단계별로 나눠 쓰고 검증한다.
- 계층 데이터는 재귀 CTE: 앵커 + UNION ALL + 재귀부.

10장 · 윈도우 함수 - 순위, 누적, 이전 행 비교

학습 목표

- 집계 함수와 윈도우 함수의 차이(행이 줄지 않음)를 설명할 수 있다.
- `OVER (PARTITION BY ... ORDER BY ...)` 의 의미를 이해하고 작성할 수 있다.
- ROW_NUMBER / RANK / DENSE_RANK, 누적 합, 이동 평균, LAG / LEAD 를 활용할 수 있다.
- 그룹별 TOP-N, 전월 대비 증감률 같은 실무 분석 쿼리를 작성할 수 있다.

예제 파일: `examples/10_window.sql`

윈도우 함수란

```
함수() OVER (
    PARTITION BY 그룹 기준     -- 어떤 행끼리 묶어 계산할지 (생략하면 전체)
    ORDER BY 정렬 기준         -- 묶음 안에서의 순서 (순위·누적에 필요)
    ROWS BETWEEN ... AND ...   -- 계산에 포함할 범위(프레임)
)
```

GROUP BY 는 그룹마다 **한 행으로 줄이지만**, 윈도우 함수는 **모든 행을 그대로 두고** 각 행 옆에 계산 결과를 붙입니다. "각 직원의 연봉과 부서 평균을 함께 보기" 처럼 상세와 집계를 동시에 볼 때 씁니다.

예제 10-01 · GROUP BY vs 윈도우 함수 - 행이 줄어들지 않는다

```
SELECT name, dept_id, salary,
       round(avg(salary) OVER ())              AS 전체평균,
       round(avg(salary) OVER (PARTITION BY dept_id)) AS 부서평균
FROM employees
WHERE dept_id IN (2, 4)
ORDER BY dept_id, salary DESC;
```

▶ 실행 결과

```
  name  ¦ dept_id ¦ salary ¦ 전체평균 ¦ 부서평균
--------+---------+--------+----------+----------
 이영업 ¦       2 ¦   9800 ¦     6533 ¦     5880
 조은지 ¦       2 ¦   6300 ¦     6533 ¦     5880
 강민수 ¦       2 ¦   5200 ¦     6533 ¦     5880
 윤서준 ¦       2 ¦   4100 ¦     6533 ¦     5880
 신동현 ¦       2 ¦   4000 ¦     6533 ¦     5880
 최개발 ¦       4 ¦  12000 ¦     6533 ¦     7350
 임도윤 ¦       4 ¦   6800 ¦     6533 ¦     7350
 한지민 ¦       4 ¦   6100 ¦     6533 ¦     7350
 오세훈 ¦       4 ¦   4500 ¦     6533 ¦     7350
(9 rows)
```

순위 함수

함수	동점 처리	예 (값: 0, 0, 5, 6)
`ROW_NUMBER()`	동점이어도 1, 2, 3, 4 (임의 순서)	1, 2, 3, 4
`RANK()`	동점은 같은 순위, 다음은 건너뜀	1, 1, 3, 4
`DENSE_RANK()`	동점은 같은 순위, 건너뛰지 않음	1, 1, 2, 3

예제 10-02 · 순위 함수 - ROW_NUMBER, RANK, DENSE_RANK 비교

```
SELECT name, stock,
       ROW_NUMBER() OVER (ORDER BY stock) AS row_number,
       RANK()       OVER (ORDER BY stock) AS rank,
       DENSE_RANK() OVER (ORDER BY stock) AS dense_rank
FROM products
WHERE stock <= 15            -- 재고가 적은 순서로 순위
ORDER BY stock, name;
```

▶ 실행 결과

```
      name        | stock | row_number | rank | dense_rank
------------------+-------+------------+------+------------
 탄산수 24입      |     0 |          2 |    1 |          1
 USB-C 허브       |     0 |          1 |    1 |          1
 남성 정장 세트   |     5 |          3 |    3 |          2
 게이밍 노트북 X17 |     6 |          4 |    4 |          3
 27인치 모니터    |    12 |          5 |    5 |          4
 울트라북 14      |    15 |          6 |    6 |          5
(6 rows)
```

예제 10-03 · 그룹별 순위 - 부서별 연봉 순위

```
SELECT d.name AS 부서, e.name, e.salary,
       RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS 부서내순위
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
WHERE e.dept_id IN (2, 4)
ORDER BY d.name, 부서내순위;
```

▶ 실행 결과

```
 부서 |  name  | salary | 부서내순위
------+--------+--------+------------
 개발 | 최개발 |  12000 |          1
 개발 | 임도윤 |   6800 |          2
 개발 | 한지민 |   6100 |          3
 개발 | 오세훈 |   4500 |          4
 영업 | 이영업 |   9800 |          1
 영업 | 조은지 |   6300 |          2
 영업 | 강민수 |   5200 |          3
 영업 | 윤서준 |   4100 |          4
 영업 | 신동현 |   4000 |          5
(9 rows)
```

그룹별 TOP-N

윈도우 함수는 WHERE 에서 쓸 수 없으므로(10-11), **서브쿼리나 CTE 로 감싼 뒤 바깥에서 거릅니다.** 실무에서 가장 많이 쓰는 패턴 중 하나입니다.

예제 10-04 · TOP-N per group - 분류별 가장 많이 팔린 상품 1개

```
SELECT 분류, 상품, 판매수량
FROM (
    SELECT c.name AS 분류, p.name AS 상품, sum(oi.quantity) AS 판매수량,
           ROW_NUMBER() OVER (PARTITION BY c.category_id
                              ORDER BY sum(oi.quantity) DESC, p.product_id) AS rn
    FROM order_items oi
    JOIN products p   ON oi.product_id = p.product_id
    JOIN categories c ON p.category_id = c.category_id
    GROUP BY c.category_id, c.name, p.product_id, p.name
) t
WHERE rn = 1
ORDER BY 판매수량 DESC;
```

▶ 실행 결과

```
    분류      |       상품        | 판매수량
```

```
--------------+------------------+----------
 음료         ¦ 콜드브루 원액    ¦       54
 간식         ¦ 견과류 믹스      ¦       36
 주변기기     ¦ 기계식 키보드    ¦       28
 스마트폰     ¦ 스마트폰 미니    ¦       26
 남성의류     ¦ 남성 셔츠        ¦       21
 노트북       ¦ 울트라북 14      ¦       18
 여성의류     ¦ 여성 원피스      ¦       16
 게이밍 노트북 ¦ 게이밍 노트북 X17 ¦        1
(8 rows)
```

■ 핵심 포인트 ROW_NUMBER 와 동점
10-04 처럼 1위가 동점일 수 있으면 `ORDER BY` 에 고유한 컬럼(`p.product_id`)을 추가해 결과를 항상 같게 만듭니다. 동점을 모두 보여 줘야 하면 `RANK() ... WHERE rn = 1` 을 씁니다. "동점이면 어떻게 할지"를 먼저 정하는 것도 요구사항 분석의 일부입니다.

누적 합과 비중

`sum(x) OVER (ORDER BY 월)` 은 처음부터 현재 행까지의 합(누적)이고, `sum(x) OVER ()` 는 전체 합입니다.

예제 10-05 · 누적 합계 - 2026년 월별 매출과 누적 매출

```
WITH monthly AS (
    SELECT date_trunc('month', o.order_date)::date AS 월,
           sum(oi.quantity * oi.unit_price) AS 매출
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.status <> 'CANCELLED' AND o.order_date >= '2026-01-01'
    GROUP BY 1
)
SELECT 월, 매출,
       sum(매출) OVER (ORDER BY 월) AS 누적매출,
       round(100.0 * 매출 / sum(매출) OVER (), 1) AS 비중
FROM monthly
ORDER BY 월;
```

▶ 실행 결과

```
    월     ¦   매출   ¦ 누적매출 ¦ 비중
------------+----------+----------+------
 2026-01-01 ¦  8668300 ¦  8668300 ¦ 23.8
 2026-02-01 ¦  2023000 ¦ 10691300 ¦  5.6
 2026-03-01 ¦  6504000 ¦ 17195300 ¦ 17.9
 2026-04-01 ¦  6616000 ¦ 23811300 ¦ 18.2
 2026-05-01 ¦  2584100 ¦ 26395400 ¦  7.1
 2026-06-01 ¦ 10031000 ¦ 36426400 ¦ 27.5
(6 rows)
```

LAG / LEAD - 이전·다음 행

함수	의미
`LAG(x [, n [, 기본값]])`	n 행 앞(기본 1)의 x
`LEAD(x [, n [, 기본값]])`	n 행 뒤의 x
`FIRST_VALUE(x)` / `LAST_VALUE(x)`	프레임의 처음/마지막 값

예제 10-06 · LAG / LEAD - 전월 대비 증감률

```
WITH monthly AS (
    SELECT date_trunc('month', o.order_date)::date AS 월,
           sum(oi.quantity * oi.unit_price) AS 매출
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.status <> 'CANCELLED' AND o.order_date >= '2026-01-01'
    GROUP BY 1
)
SELECT 월, 매출,
       LAG(매출) OVER (ORDER BY 월) AS 전월매출,
       round(100.0 * (매출 - LAG(매출) OVER (ORDER BY 월))
             / LAG(매출) OVER (ORDER BY 월), 1) AS 증감률
FROM monthly
ORDER BY 월;
```

▶ 실행 결과

```
     월     ┆   매출   ┆ 전월매출 ┆ 증감률
------------+----------+----------+--------
 2026-01-01 ┆  8668300 ┆          ┆
 2026-02-01 ┆  2023000 ┆  8668300 ┆  -76.7
 2026-03-01 ┆  6504000 ┆  2023000 ┆  221.5
 2026-04-01 ┆  6616000 ┆  6504000 ┆    1.7
 2026-05-01 ┆  2584100 ┆  6616000 ┆  -60.9
 2026-06-01 ┆ 10031000 ┆  2584100 ┆  288.2
(6 rows)
```

예제 10-07 · 고객별 직전 주문과의 간격 (재구매 주기)

```
SELECT customer_id, order_id, order_date::date,
       order_date::date - LAG(order_date::date) OVER w AS 이전주문후_일수
FROM orders
WHERE customer_id = 1
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date)
ORDER BY order_date
LIMIT 6;
```

▶ 실행 결과

```
 customer_id ┆ order_id ┆ order_date ┆ 이전주문후_일수
-------------+----------+------------+-----------------
           1 ┆        4 ┆ 2025-01-24 ┆
           1 ┆       11 ┆ 2025-02-17 ┆              24
           1 ┆       47 ┆ 2025-07-20 ┆             153
           1 ┆       64 ┆ 2025-10-22 ┆              94
           1 ┆       70 ┆ 2025-11-09 ┆              18
           1 ┆       75 ┆ 2025-11-20 ┆              11
(6 rows)
```

■ TIP WINDOW 절
10-07 처럼 같은 OVER 정의를 여러 번 쓰면 `WINDOW w AS (...)` 로 이름을 붙여 `OVER w` 로 재사용할 수 있습니다.

윈도우 프레임과 이동 평균

ORDER BY 가 있는 윈도우의 기본 프레임은 "처음부터 현재 행까지" 입니다. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` 처럼 바꾸면 최근 3행의 평균, 즉 **이동 평균**이 됩니다.

예제 10-08 · 이동 평균 - 윈도우 프레임 (ROWS BETWEEN)

```
WITH daily AS (
    SELECT order_date::date AS d, count(*) AS cnt
    FROM orders WHERE order_date >= '2026-05-01'
    GROUP BY 1
)
SELECT d, cnt,
       round(avg(cnt) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS 이동평균3
FROM daily
ORDER BY d
LIMIT 8;
```

▶ 실행 결과

```
     d      | cnt | 이동평균3
------------+-----+-----------
 2026-05-06 |   1 |      1.00
 2026-05-10 |   1 |      1.00
 2026-05-17 |   2 |      1.33
 2026-05-30 |   1 |      1.33
 2026-05-31 |   1 |      1.33
 2026-06-07 |   1 |      1.00
 2026-06-14 |   2 |      1.33
 2026-06-18 |   2 |      1.67
(8 rows)
```

예제 10-09 · FIRST_VALUE / LAST_VALUE - 프레임을 전체로 지정해야 하는 이유

```
SELECT name, category_id, price,
       FIRST_VALUE(name) OVER w AS 최저가상품,
       LAST_VALUE(name)  OVER w AS 최고가상품
FROM products
WHERE category_id IN (4, 5)
WINDOW w AS (PARTITION BY category_id ORDER BY price
             ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

▶ 실행 결과

```
        name        | category_id |  price  |     최저가상품     |  최고가상품
--------------------+-------------+---------+--------------------+--------------
 비즈니스 노트북 15 |           4 |  990000 | 비즈니스 노트북 15 | 울트라북 14
 울트라북 14        |           4 | 1290000 | 비즈니스 노트북 15 | 울트라북 14
 스마트폰 미니      |           5 |  690000 | 스마트폰 미니      | 스마트폰 S25
 스마트폰 S25       |           5 | 1150000 | 스마트폰 미니      | 스마트폰 S25
(4 rows)
```

■ 주의 LAST_VALUE 의 함정

10-09 에서 프레임을 지정하지 않으면 기본 프레임이 "현재 행까지" 라서 LAST_VALUE 가 **현재 행 자신**을 돌려줍니다. 그래서 `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` 으로 전체를 지정했습니다. 가장 많이 틀리는 부분이니 프레임을 빼고 실행해 차이를 직접 확인해 보세요.

예제 10-10 · NTILE / PERCENT_RANK - 고객을 구매액 기준 4개 그룹으로

```
WITH s AS (
    SELECT o.customer_id, sum(oi.quantity * oi.unit_price) AS total
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.customer_id
)
SELECT customer_id, total,
       NTILE(4) OVER (ORDER BY total DESC) AS 분위,
       round(PERCENT_RANK() OVER (ORDER BY total)::numeric, 2) AS 백분위
FROM s
ORDER BY total DESC
LIMIT 8;
```

▶ 실행 결과

```
 customer_id ¦  total   ¦ 분위 ¦ 백분위
-------------+----------+------+--------
          10 ¦ 20605300 ¦    1 ¦   1.00
           5 ¦  9892100 ¦    1 ¦   0.94
           8 ¦  9695000 ¦    1 ¦   0.88
           6 ¦  8905000 ¦    1 ¦   0.82
           1 ¦  7762600 ¦    1 ¦   0.76
           9 ¦  6789300 ¦    2 ¦   0.71
          14 ¦  6498800 ¦    2 ¦   0.65
           7 ¦  5878000 ¦    2 ¦   0.59
(8 rows)
```

예제 10-11 · 윈도우 함수는 WHERE 에서 쓸 수 없다

```
SELECT name, price FROM products
WHERE ROW_NUMBER() OVER (ORDER BY price DESC) <= 3;
```

▶ 실행 결과

```
ERROR:  window functions are not allowed in WHERE
LINE 2: WHERE ROW_NUMBER() OVER (ORDER BY price DESC) <= 3;
              ^
```

■ Q&A; 자주 묻는 질문

- **Q. 윈도우 함수와 GROUP BY 를 같이 쓸 수 있나요?** 네. GROUP BY 가 먼저 실행되고, 그 결과에 윈도우 함수가 적용됩니다. 10-04 의 `sum(oi.quantity)` 를 ORDER BY 에 쓴 부분이 그 예입니다.
- **Q. 성능은?** 윈도우 함수는 PARTITION/ORDER 기준으로 정렬이 필요합니다. 큰 테이블에서는 그 컬럼의 인덱스가 도움이 됩니다.

실습 문제

번호	문제
10-Q1	고객별 주문에 순번을 매기고 각 고객의 첫 주문만 출력하세요 (고객 1~5)
10-Q2	분류별로 상품을 가격 순위로 매기고 분류 내 최고가와의 차이를 구하세요 (분류 4, 5, 6)
10-Q3	2025년 월별 주문 수와 3개월 이동 합계를 구하세요
10-Q4	직원을 연봉 기준으로 상위/중위/하위 3그룹(NTILE)으로 나누고 그룹별 인원과 평균을 구하세요
10-Q5	고객 1번의 주문별 금액과 누적 구매액을 구하세요 (취소 제외)

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 윈도우 함수는 행을 유지하면서 그룹 계산 결과를 붙인다.
- 순위 3종의 차이는 동점 처리. TOP-N 은 서브쿼리로 감싸서 거른다.
- 누적은 `sum() OVER (ORDER BY)`, 전월 대비는 `LAG`, 이동 평균은 `ROWS BETWEEN`.

11장 · 데이터 모델링과 정규화

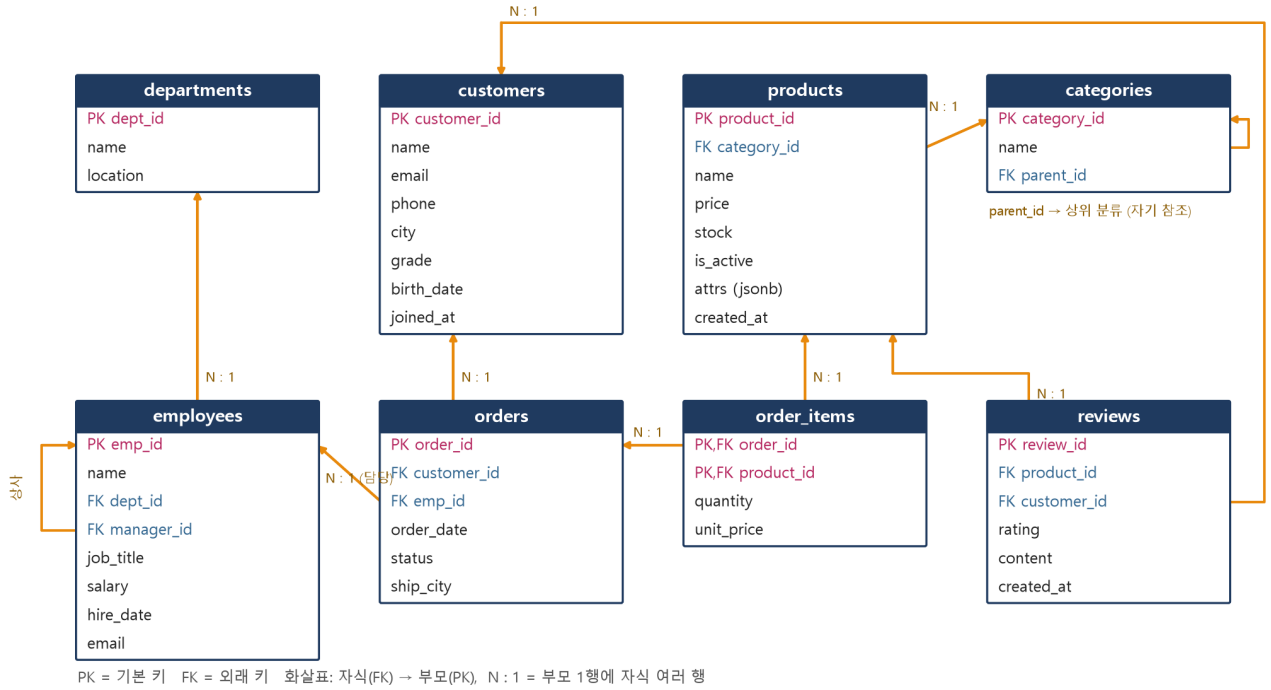
학습 목표

- 엔터티, 속성, 관계(1:1, 1:N, N:M)를 ERD 로 표현할 수 있다.
- 이상 현상(삽입·수정·삭제 이상)을 예로 설명할 수 있다.
- 제1~제3 정규형을 이해하고 비정규 테이블을 분리할 수 있다.
- 반정규화가 필요한 경우를 판단할 수 있다.

예제 파일: `examples/11_modeling.sql` (`model` 스키마에 만들어 shop 테이블과 섞이지 않음)

모델링 절차

단계	하는 일	결과물
요구사항 분석	"고객은 여러 번 주문한다", "주문 하나에 상품이 여러 개" 같은 업무 규칙 수집	요구사항 정의서
개념 모델링	엔터티(고객, 주문, 상품)와 관계 도출	개념 ERD
논리 모델링	속성, 키, 정규화	논리 ERD
물리 모델링	자료형, 인덱스, 제약 조건, DBMS 특성 반영	DDL 스크립트 (`setup/01_schema.sql`)



> **TIP DBeaver 로 ERD 보기**
> Navigator 에서 `shop → Schemas → public` 을 우클릭 → **View Diagram** 을 누르면 외래 키를 바탕으로 ER 다이어그램을 자동으로 그려 줍니다. 테이블을 끌어 배치를 바꾸고 이미지로 저장(Export Diagram)할 수 있어 설계 토론 자료로 좋습니다. 테이블 하나를 열어 **ER Diagram** 탭을 누르면 그 테이블과 연결된 테이블만 볼 수 있습니다.

■ 핵심 포인트 ERD 읽기 연습
shop ERD 를 보며 스스로 답해 보세요. "고객 1명은 주문을 몇 개 가질 수 있나?"(0~N), "주문 1개에 고객은 몇 명?"(정확히 1), "주문과 상품의 관계는?"(N:M → order_items 로 풀었다). 관계를 말로 읽을 수 있으면 JOIN 조건도 저절로 보입니다.

이상 현상 (Anomaly)

한 테이블에 여러 주제의 데이터를 섞으면 다음 문제가 생깁니다.

이상	예 (엑셀식 주문표)
수정 이상	김철수의 전화번호를 바꾸려면 김철수의 모든 주문 행을 고쳐야 함. 하나라도 빠지면 불일치
삽입 이상	주문이 없는 신규 고객은 등록할 수 없음 (주문번호가 비어 버림)
삭제 이상	이영희의 유일한 주문을 지우면 고객 정보도 함께 사라짐

예제 11-01 · 정규화 전 - 한 테이블에 모든 것을 넣은 "엑셀식" 주문표

```
CREATE TABLE flat_orders (
    order_no      INTEGER,
    order_date    DATE,
    customer_name VARCHAR(50),
    customer_tel  VARCHAR(20),
    products      TEXT,           -- '마우스, 키보드' 처럼 여러 값을 한 칸에 (1NF 위반)
    total_price   INTEGER
);
INSERT INTO flat_orders VALUES
    (1, '2026-10-01', '김철수', '010-1111-2222', '마우스, 키보드', 118000),
    (2, '2026-10-02', '김철수', '010-1111-2222', '모니터', 329000),
    (3, '2026-10-02', '이영희', '010-3333-4444', '마우스', 29000);
SELECT * FROM flat_orders;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 3
 order_no ¦ order_date ¦ customer_name ¦ customer_tel  ¦    products    ¦ total_price
----------+------------+---------------+---------------+----------------+-------------
        1 ¦ 2026-10-01 ¦ 김철수        ¦ 010-1111-2222 ¦ 마우스, 키보드 ¦      118000
        2 ¦ 2026-10-02 ¦ 김철수        ¦ 010-1111-2222 ¦ 모니터         ¦      329000
        3 ¦ 2026-10-02 ¦ 이영희        ¦ 010-3333-4444 ¦ 마우스         ¦       29000
(3 rows)
```

예제 11-02 · 이상 현상(Anomaly) - 김철수의 전화번호를 한 행만 바꾸면?

```
UPDATE flat_orders SET customer_tel = '010-9999-9999' WHERE order_no = 1;
SELECT order_no, customer_name, customer_tel FROM flat_orders WHERE customer_name = '김철수'
ORDER BY order_no;
```

▶ 실행 결과

```
UPDATE 1
 order_no ¦ customer_name ¦ customer_tel
----------+---------------+---------------
        1 ¦ 김철수        ¦ 010-9999-9999
        2 ¦ 김철수        ¦ 010-1111-2222
(2 rows)
```

예제 11-03 · '마우스'를 산 주문 찾기도 어렵다 (한 칸에 여러 값)

```
SELECT order_no, products FROM flat_orders WHERE products LIKE '%마우스%' ORDER BY order_no;
```

▶ 실행 결과

```
 order_no ¦    products
----------+----------------
        1 ¦ 마우스, 키보드
        3 ¦ 마우스
(2 rows)
```

정규화

정규형	규칙	위반 예 → 해결
제1정규형 (1NF)	모든 칸은 **하나의 값**(원자값)	`products = '마우스, 키보드'` → 주문상세 행으로 분리
제2정규형 (2NF)	복합 키의 **일부에만** 종속된 컬럼이 없어야	(주문번호, 상품번호) 키에서 상품명은 상품번호에만 종속 → 상품 테이블로
제3정규형 (3NF)	키가 아닌 컬럼에 종속된 컬럼이 없어야 (이행 종속 제거)	주문 테이블의 고객전화는 고객번호에 종속 → 고객 테이블로

한 줄 요약: **"모든 컬럼은 키에, 키 전체에, 오직 키에만 종속되어야 한다."**

예제 11-04 · 정규화 후 - 고객 / 상품 / 주문 / 주문상세로 분리 (3NF)

```
CREATE TABLE customer (
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        VARCHAR(50) NOT NULL,
    tel         VARCHAR(20)
);
CREATE TABLE product (
    product_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        VARCHAR(50) NOT NULL UNIQUE,
    price       INTEGER     NOT NULL
);
CREATE TABLE "order" (                              -- order 는 예약어라 따옴표 필요 (권장 X)
    order_no    INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customer,
    order_date  DATE    NOT NULL
);
CREATE TABLE order_line (
    order_no    INTEGER REFERENCES "order",
    product_id  INTEGER REFERENCES product,
    qty         INTEGER NOT NULL DEFAULT 1,
    PRIMARY KEY (order_no, product_id)
);
INSERT INTO customer (name, tel) VALUES ('김철수', '010-1111-2222'), ('이영희', '010-3333-4444');
INSERT INTO product (name, price) VALUES ('마우스', 29000), ('키보드', 89000), ('모니터', 329000);
INSERT INTO "order" VALUES (1, 1, '2026-10-01'), (2, 1, '2026-10-02'), (3, 2, '2026-10-02');
INSERT INTO order_line VALUES (1, 1, 1), (1, 2, 1), (2, 3, 1), (3, 1, 1);
```

▶ 실행 결과

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 2
INSERT 0 3
INSERT 0 3
INSERT 0 4
```

예제 11-05 · 정규화된 테이블에서 원래 주문표 복원 (JOIN)

```
SELECT o.order_no, o.order_date, c.name, c.tel,
       string_agg(p.name, ', ' ORDER BY p.product_id) AS products,
       sum(p.price * ol.qty) AS total_price           -- 합계는 저장하지 않고 계산
FROM "order" o
JOIN customer c    ON c.customer_id = o.customer_id
JOIN order_line ol ON ol.order_no = o.order_no
JOIN product p     ON p.product_id = ol.product_id
GROUP BY o.order_no, o.order_date, c.name, c.tel
ORDER BY o.order_no;
```

▶ 실행 결과

```
 order_no ¦ order_date ¦  name  ¦      tel      ¦    products    ¦ total_price
----------+------------+--------+---------------+----------------+-------------
        1 ¦ 2026-10-01 ¦ 김철수 ¦ 010-1111-2222 ¦ 마우스, 키보드 ¦      118000
        2 ¦ 2026-10-02 ¦ 김철수 ¦ 010-1111-2222 ¦ 모니터         ¦      329000
        3 ¦ 2026-10-02 ¦ 이영희 ¦ 010-3333-4444 ¦ 마우스         ¦       29000
(3 rows)
```

예제 11-06 · 이제 전화번호 변경은 한 곳만 고치면 된다

```
UPDATE customer SET tel = '010-9999-9999' WHERE name = '김철수';
SELECT o.order_no, c.name, c.tel
FROM "order" o JOIN customer c USING (customer_id)
ORDER BY o.order_no;
```

▶ 실행 결과

```
UPDATE 1
 order_no ¦  name  ¦      tel
----------+--------+---------------
        1 ¦ 김철수 ¦ 010-9999-9999
        2 ¦ 김철수 ¦ 010-9999-9999
        3 ¦ 이영희 ¦ 010-3333-4444
(3 rows)
```

■ 참고 **계산 가능한 값은 저장하지 않는다**

11-05 에서 `total_price` 는 저장하지 않고 조회 시 계산했습니다. 저장해 두면 상세가 바뀔 때마다 맞춰 줘야 하고 어긋날 위험이 있습니다. 단, `order_items.unit_price` 처럼 **"그 시점의 값"** 은 상품 가격이 나중에 바뀌어도 보존해야 하므로 저장하는 것이 맞습니다. 이것은 중복이 아니라 별개의 사실입니다.

관계의 종류와 구현

관계	예	구현
1 : N	고객 – 주문	N 쪽(orders)에 FK (`customer_id`)
N : M	주문 – 상품	연결 테이블(order_items)에 양쪽 FK, 복합 PK
1 : 1	고객 – 고객상세	자식 테이블의 PK 가 곧 FK
자기 참조	직원 – 상사, 분류 – 상위분류	같은 테이블을 가리키는 FK (`manager_id`, `parent_id`)

예제 11-07 · N:M 관계와 연결 테이블 - shop 의 주문(orders)↔상품(products)

```
SET search_path = public;
SELECT tc.table_name, kcu.column_name, ccu.table_name AS 참조테이블
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
  ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu
  ON tc.constraint_name = ccu.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY' AND tc.table_name = 'order_items';
```

▶ 실행 결과

```
SET
 table_name  ¦ column_name ¦ 참조테이블
-------------+-------------+------------
 order_items ¦ order_id    ¦ orders
 order_items ¦ product_id  ¦ products
(2 rows)
```

예제 11-08 · 1:1 관계 - 고객 상세정보를 별도 테이블로 (PK 이자 FK)

```
CREATE TABLE model.customer_profile (
    customer_id INTEGER PRIMARY KEY REFERENCES model.customer,
    address     TEXT,
    marketing_agree BOOLEAN NOT NULL DEFAULT false
);
SELECT column_name, data_type, character_maximum_length AS max_len,
       is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'model' AND table_name = 'customer_profile'
ORDER BY ordinal_position;
```

▶ 실행 결과

```
CREATE TABLE
   column_name   ¦ data_type ¦ max_len ¦ is_nullable ¦ column_default
-----------------+-----------+---------+-------------+----------------
 customer_id     ¦ integer   ¦         ¦ NO          ¦
 address         ¦ text      ¦         ¦ YES         ¦
 marketing_agree ¦ boolean   ¦         ¦ NO          ¦ false
(3 rows)
```

반정규화

정규화는 중복을 없애지만 조인이 늘어납니다. 조회 성능이 매우 중요하고 데이터 변경이 드문 곳에서는 일부러 중복을 허용(반정규화)하기도 합니다.

- 예: 주문 목록 화면을 위해 orders 에 `total_amount` 를 저장 → 주문 상세가 바뀔 때 트리거(15장)로 갱신
- 예: 대시보드 집계 → 머티리얼라이즈드 뷰(12장)나 집계 테이블
- 원칙: **먼저 정규화하고, 측정해서 느릴 때 근거를 가지고 반정규화**합니다.

■ 핵심 포인트 설계 연습

"도서관 대출 시스템" 또는 "수강 신청 시스템"의 테이블을 직접 설계해 보세요. 엔터티 3~5개, 관계, PK/FK, 제약 조건을 종이에 그린 뒤 실습 문제 02-Q1~Q2(도서·회원·대출) 정답과 비교해 봅니다. DBeaver 의 View Diagram 으로 자신이 만든 테이블의 ERD 도 확인해 보세요.

■ Q&A; 자주 묻는 질문

- **Q. 정규화를 어디까지 해야 하나요?** 실무에서는 보통 3NF(또는 BCNF)까지면 충분합니다.
- **Q. 이름에 order 같은 예약어를 써도 되나요?** 11-04 처럼 큰따옴표로 가능하지만 매번 따옴표가 필요해 불편합니다. `orders` 처럼 복수형으로 짓는 이유이기도 합니다.

정리

- 하나의 테이블에는 하나의 주제만. 섞으면 수정·삽입·삭제 이상이 생긴다.
- 1NF 원자값, 2NF 부분 종속 제거, 3NF 이행 종속 제거.
- 계산 가능한 값은 저장하지 않되, "그 시점의 값"은 저장한다.

12장 · 뷰, 머티리얼라이즈드 뷰, 시퀀스, 스키마

학습 목표

- 뷰를 만들어 복잡한 조인을 재사용하고 민감한 컬럼을 숨길 수 있다.
- 머티리얼라이즈드 뷰와 일반 뷰의 차이를 설명하고 REFRESH 할 수 있다.
- 시퀀스를 직접 다루고 SERIAL 과 IDENTITY 의 차이를 설명할 수 있다.
- 스키마로 테이블을 구분하고 search_path 를 이해할 수 있다.

예제 파일: `examples/12_view_sequence_schema.sql`

뷰 (VIEW)

뷰는 **이름을 붙여 저장한 SELECT 문**입니다. 데이터를 따로 저장하지 않고, 조회할 때마다 원래 쿼리를 실행합니다.

용도	예
복잡한 조인 재사용	`v_order_detail` – 주문·고객·상품을 미리 조인
보안	`v_customer_public` – 이메일·전화번호를 뺀 뷰에만 권한 부여(17장)
호환성	테이블 구조가 바뀌어도 뷰 이름과 컬럼을 유지해 기존 프로그램 보호

예제 12-01 · 뷰 만들기 - 자주 쓰는 조인을 저장

```
CREATE VIEW v_order_detail AS
SELECT o.order_id, o.order_date, o.status,
       c.customer_id, c.name AS customer_name,
       p.product_id, p.name AS product_name,
       oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS amount
FROM orders o
JOIN customers c    ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p     ON p.product_id = oi.product_id;

SELECT order_id, customer_name, product_name, amount
FROM v_order_detail
WHERE order_id = 2;
```

▶ 실행 결과

```
CREATE VIEW
 order_id ¦ customer_name ¦    product_name    ¦ amount
----------+---------------+--------------------+---------
        2 ¦ 임준서        ¦ 비즈니스 노트북 15 ¦ 2673000
        2 ¦ 임준서        ¦ 27인치 모니터      ¦  658000
        2 ¦ 임준서        ¦ 남성 셔츠          ¦   35100
        2 ¦ 임준서        ¦ 유기농 녹차        ¦   12000
(4 rows)
```

예제 12-02 · 뷰는 테이블처럼 쓸 수 있다

```
SELECT customer_name, sum(amount) AS 총액
FROM v_order_detail
WHERE status = 'DELIVERED'
GROUP BY customer_name
ORDER BY 총액 DESC
LIMIT 3;
```

▶ 실행 결과

```
 customer_name ¦   총액
---------------+----------
 임준서        ¦ 20481300
 정하은        ¦  8952100
 윤지호        ¦  8729000
(3 rows)
```

예제 12-03 · 뷰는 데이터를 저장하지 않는다 - 원본이 바뀌면 바로 반영

```
UPDATE order_items SET quantity = 10 WHERE order_id = 1 AND product_id = 13;
SELECT order_id, product_name, quantity, amount FROM v_order_detail WHERE order_id = 1;
```

▶ 실행 결과

```
UPDATE 1
 order_id ¦ product_name ¦ quantity ¦ amount
----------+--------------+----------+--------
        1 ¦ 여성 니트    ¦       10 ¦ 490000
(1 row)
```

예제 12-04 · 보안 목적의 뷰 - 민감한 컬럼 감추기

```
CREATE VIEW v_customer_public AS
SELECT customer_id, name, city, grade FROM customers;      -- email, phone 제외
SELECT * FROM v_customer_public LIMIT 3;
```

▶ 실행 결과

```
CREATE VIEW
 customer_id ¦  name  ¦ city ¦ grade
-------------+--------+------+--------
           1 ¦ 김민준 ¦ 서울 ¦ VIP
           2 ¦ 이서연 ¦ 부산 ¦ GOLD
           3 ¦ 박도윤 ¦ 서울 ¦ SILVER
(3 rows)
```

갱신 가능한 뷰와 CHECK OPTION

하나의 테이블에서 집계 없이 만든 단순 뷰는 INSERT/UPDATE/DELETE 가 가능합니다. `WITH CHECK OPTION` 은 뷰의 조건을 벗어나는 변경(서울 고객 뷰에서 도시를 부산으로)을 막습니다.

예제 12-05 · 단순 뷰는 수정도 가능 + WITH CHECK OPTION

```
CREATE VIEW v_seoul_customers AS
SELECT customer_id, name, city, email FROM customers WHERE city = '서울'
WITH CHECK OPTION;
UPDATE v_seoul_customers SET name = '김민준(수정)' WHERE customer_id = 1;
SELECT customer_id, name FROM customers WHERE customer_id = 1;
```

▶ 실행 결과

```
CREATE VIEW
UPDATE 1
 customer_id ¦     name
-------------+--------------
           1 ¦ 김민준(수정)
(1 row)
```

예제 12-06 · CHECK OPTION 위반 - 뷰 조건을 벗어나는 수정

```
UPDATE v_seoul_customers SET city = '부산' WHERE customer_id = 1;
```

▶ 실행 결과

```
ERROR:  new row violates check option for view "v_seoul_customers"
DETAIL:  Failing row contains (1, 김민준(수정), minjun@example.com, 010-1234-0001, 부산, VIP, 1988-02-11, 2024-01-05 10:12:00).
```

머티리얼라이즈드 뷰

구분	뷰	머티리얼라이즈드 뷰
저장	쿼리만 저장	**결과를 디스크에 저장**
조회 속도	원본 쿼리만큼	테이블 조회처럼 빠름 (인덱스도 가능)
최신성	항상 최신	REFRESH 해야 갱신
용도	일반 재사용	대시보드·일별 리포트 같은 무거운 집계

예제 12-07 · 머티리얼라이즈드 뷰 - 결과를 저장해 두는 뷰

```
CREATE MATERIALIZED VIEW mv_monthly_sales AS
SELECT date_trunc('month', order_date)::date AS month,
       sum(amount) AS sales
FROM v_order_detail
WHERE status <> 'CANCELLED'
GROUP BY 1;
SELECT * FROM mv_monthly_sales WHERE month >= '2026-04-01' ORDER BY month;
```

▶ 실행 결과

```
SELECT 18
   month    ¦  sales
------------+----------
 2026-04-01 ¦  6616000
 2026-05-01 ¦  2584100
 2026-06-01 ¦ 10031000
(3 rows)
```

예제 12-08 · 머티리얼라이즈드 뷰는 REFRESH 해야 최신화된다

```
INSERT INTO orders (customer_id, order_date, status, ship_city)
VALUES (1, '2026-06-30 10:00', 'PAID', '서울') RETURNING order_id;
INSERT INTO order_items VALUES (currval(pg_get_serial_sequence('orders', 'order_id')), 3, 1, 2490000);
SELECT * FROM mv_monthly_sales WHERE month = '2026-06-01';   -- 아직 그대로
REFRESH MATERIALIZED VIEW mv_monthly_sales;
SELECT * FROM mv_monthly_sales WHERE month = '2026-06-01';   -- 반영됨
```

▶ 실행 결과

```
 order_id
----------
      121
(1 row)

INSERT 0 1
INSERT 0 1
   month    ¦  sales
------------+----------
 2026-06-01 ¦ 10031000
(1 row)

REFRESH MATERIALIZED VIEW
   month    ¦  sales
------------+----------
```

```
 2026-06-01 ¦ 12521000
(1 row)
```

> **■ TIP 갱신 중에도 조회하려면**
> `REFRESH MATERIALIZED VIEW` 는 갱신하는 동안 조회를 막습니다. 머티리얼라이즈드 뷰에 UNIQUE 인덱스를 만들고 `REFRESH MATERIALIZED VIEW CONCURRENTLY` 를 쓰면 조회를 막지 않고 갱신합니다. 주기적 갱신은 cron 이나 pg_cron 확장으로 예약합니다.

시퀀스

시퀀스는 번호를 하나씩 발급하는 객체입니다. IDENTITY 컬럼도 내부적으로 시퀀스를 씁니다.

함수	설명
`nextval('seq')`	다음 번호를 발급 (롤백해도 되돌아가지 않음 → 번호에 빈 곳이 생길 수 있음)
`currval('seq')`	이 세션에서 마지막으로 발급받은 번호
`setval('seq', n)`	현재 값을 n 으로 설정 (`setup/02_seed.sql` 마지막에서 사용)

예제 12-09 · 시퀀스 직접 다루기

```
CREATE SEQUENCE invoice_seq START WITH 1000 INCREMENT BY 10;
SELECT nextval('invoice_seq') AS 첫번째, nextval('invoice_seq') AS 두번째;
SELECT currval('invoice_seq') AS 현재값;
SELECT 'INV-' ¦¦ to_char(current_date, 'YYYYMMDD') ¦¦ '-' ¦¦ nextval('invoice_seq') AS 송장번호;
```

▶ 실행 결과

```
CREATE SEQUENCE
 첫번째 ¦ 두번째
--------+--------
   1000 ¦   1010
(1 row)

 현재값
--------
   1010
(1 row)

     송장번호
-------------------
 INV-20261023-1020
(1 row)
```

예제 12-10 · SERIAL 과 IDENTITY 비교 (IDENTITY 가 표준이자 권장)

```
CREATE TABLE t_serial   (id SERIAL PRIMARY KEY, v TEXT);
CREATE TABLE t_identity (id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, v TEXT);
INSERT INTO t_serial (id, v) VALUES (1, '직접 지정 가능');     -- 시퀀스는 모른 채 1 이 들어감
```

▶ 실행 결과

```
CREATE TABLE
CREATE TABLE
INSERT 0 1
```

예제 12-11 · SERIAL 의 중복 키 사고와 IDENTITY ALWAYS 의 보호

```
INSERT INTO t_serial (v) VALUES ('자동');                    -- 시퀀스가 1 을 줘서 충돌!
INSERT INTO t_identity (id, v) VALUES (1, '직접 지정');       -- ALWAYS 는 막아 준다
```

▶ 실행 결과

```
ERROR:  duplicate key value violates unique constraint "t_serial_pkey"
DETAIL:  Key (id)=(1) already exists.
ERROR:  cannot insert a non-DEFAULT value into column "id"
DETAIL:  Column "id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
```

■ 주의 번호의 빈 곳은 정상

주문번호가 1, 2, 4 처럼 건너뛰어 놀랄 수 있습니다. 실패한 INSERT 나 ROLLBACK 된 트랜잭션도 시퀀스 번호를 소비하기 때문이며 정상입니다. "빈 번호 없는 연속 번호"가 법적으로 필요한 경우(세금계산서 등)에는 별도 채번 테이블을 잠금과 함께 씁니다.

스키마

스키마는 데이터베이스 안의 **폴더(네임스페이스)** 입니다. 같은 DB 안에서 `report.daily_kpi`, `model.customer` 처럼 이름을 구분하고, 스키마 단위로 권한을 줄 수 있습니다. 이름 앞에 스키마를 쓰지 않으면 `search_path` 순서대로 찾습니다(기본: 사용자 이름과 같은 스키마 → public).

예제 12-12 · 스키마 - 테이블을 묶는 폴더(네임스페이스)

```
CREATE SCHEMA report;
CREATE TABLE report.daily_kpi (kpi_date DATE PRIMARY KEY, sales NUMERIC);
INSERT INTO report.daily_kpi VALUES ('2026-06-30', 2490000);
SELECT * FROM report.daily_kpi;
SHOW search_path;
SELECT schema_name FROM information_schema.schemata
WHERE schema_name NOT LIKE 'pg_%' AND schema_name <> 'information_schema'
ORDER BY 1;
```

▶ 실행 결과

```
CREATE SCHEMA
CREATE TABLE
INSERT 0 1
  kpi_date  ¦  sales
------------+---------
 2026-06-30 ¦ 2490000
(1 row)

   search_path
-----------------
 "$user", public
(1 row)

 schema_name
-------------
 public
 report
(2 rows)
```

■ Q&A; 자주 묻는 질문

- **Q. 뷰를 쓰면 느려지나요?** 일반 뷰는 쿼리에 그대로 펼쳐져 최적화되므로 원래 쿼리와 성능이 같습니다. 뷰 위에 뷰를 여러 겹 쌓으면 불필요한 조인이 생길 수 있으니 주의합니다.
- **Q. 뷰의 원본 테이블 컬럼을 바꾸면?** 뷰가 참조하는 컬럼은 삭제·타입 변경이 막힙니다. 뷰를 먼저 지우고(`DROP VIEW`) 다시 만들어야 합니다.

실습 문제

번호	문제
12-Q1	상품명, 분류명, 판매수량, 평균평점을 보여 주는 뷰 v_product_summary 를 만들고 판매수량 상위 3개를 조회하세요
12-Q2	등급별 고객 수와 총구매액을 담은 머티리얼라이즈드 뷰를 만들고 조회하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 뷰는 저장된 쿼리, 머티리얼라이즈드 뷰는 저장된 결과(REFRESH 필요).
- 자동 증가는 IDENTITY(ALWAYS), 번호 건너뜀은 정상.
- 스키마로 이름과 권한을 구분한다.

13장 · 인덱스와 실행 계획

학습 목표

- 인덱스가 검색을 빠르게 하는 원리(B-tree)와 그 비용을 설명할 수 있다.
- EXPLAIN / EXPLAIN ANALYZE 로 실행 계획을 읽고 Seq Scan, Index Scan, Bitmap Scan 을 구분할 수 있다.
- 단일·복합·표현식·부분 인덱스를 상황에 맞게 만들 수 있다.
- 인덱스를 쓰지 못하는 쿼리 패턴을 알아보고 고칠 수 있다.

예제 파일: `examples/13_index_explain.sql` (100만 건 생성에 수 초~수십 초 소요)

인덱스의 원리

인덱스는 책 뒤의 **찾아보기**와 같습니다. 컬럼 값을 정렬된 트리(B-tree)로 따로 저장해 두고, 값을 찾을 때 트리를 따라 내려가 해당 행의 위치로 바로 갑니다. 100만 건에서 한 건을 찾을 때 전체를 읽는 대신 3~4개 페이지만 읽습니다.

장점	비용
WHERE, JOIN, ORDER BY 가 빨라짐	디스크 공간 (13-09 에서 크기 확인)
UNIQUE 제약의 검사 수단	INSERT/UPDATE/DELETE 때마다 인덱스도 수정 → 쓰기 느려짐
	많이 만들수록 관리 부담, 쓰이지 않는 인덱스는 순수 비용

PRIMARY KEY 와 UNIQUE 제약에는 인덱스가 **자동으로** 만들어지지만, 외래 키에는 만들어지지 **않습니다**(13-12).

예제 13-01 · 실습용 대용량 테이블 만들기 (100만 건)

```
SELECT setseed(0.42);                                   -- 난수 고정
CREATE TABLE big_orders AS
SELECT g                                                AS id,
       (random() * 99999)::int + 1                      AS customer_id,
       timestamp '2024-01-01' + random() * interval '900 days' AS order_date,
       (ARRAY['PAID','SHIPPED','DELIVERED','CANCELLED'])[1 + (random() * 3)::int] AS status,
       round((random() * 500000)::numeric, -2)          AS amount
FROM generate_series(1, 1000000) AS g;
ALTER TABLE big_orders ADD PRIMARY KEY (id);
ANALYZE big_orders;
SELECT count(*), pg_size_pretty(pg_total_relation_size('big_orders')) AS 크기 FROM big_orders;
```

▶ 실행 결과

```
 setseed
---------

(1 row)

SELECT 1000000
ALTER TABLE
ANALYZE
  count  ¦ 크기
---------+-------
 1000000 ¦ 83 MB
(1 row)
```

실행 계획 읽기

`EXPLAIN 쿼리` 는 옵티마이저가 세운 계획을, `EXPLAIN ANALYZE 쿼리` 는 실제로 실행한 결과(실제 행 수, 시간)를 보여 줍니다. 교재은 숫자가 PC 마다 다른 cost·시간을 `COSTS OFF, TIMING OFF` 로 숨겼습니다. 직접 실행할 때는 옵션 없이 실행해 실제 시간도 확인해 보세요.

노드	의미
`Seq Scan`	테이블 전체를 처음부터 끝까지 읽음
`Index Scan`	인덱스로 위치를 찾아 테이블 행을 하나씩 읽음 (소수 행)
`Index Only Scan`	인덱스만 읽고 끝 (필요한 컬럼이 모두 인덱스에 있을 때)
`Bitmap Index/Heap Scan`	인덱스로 위치 목록을 만든 뒤 테이블을 페이지 순서대로 읽음 (중간 정도 행)
`Gather` + `Parallel ...`	여러 프로세스가 나눠서 병렬 처리
`Hash Join` / `Nested Loop` / `Merge Join`	조인 방법

예제 13-02 · 인덱스가 없을 때 - Seq Scan (전체 테이블 읽기)

```
EXPLAIN (COSTS OFF)
SELECT * FROM big_orders WHERE customer_id = 12345;
```

▶ 실행 결과

```
             QUERY PLAN
-------------------------------------
 Gather
   Workers Planned: 2
   ->  Parallel Seq Scan on big_orders
         Filter: (customer_id = 12345)
(4 rows)
```

예제 13-03 · EXPLAIN ANALYZE - 실제로 실행해서 읽은 행 수 확인

```
EXPLAIN (ANALYZE, BUFFERS OFF, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT * FROM big_orders WHERE customer_id = 12345;
```

▶ 실행 결과

```
                          QUERY PLAN
---------------------------------------------------------------
 Gather (actual rows=9.00 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on big_orders (actual rows=3.00 loops=3)
         Filter: (customer_id = 12345)
         Rows Removed by Filter: 333330
(6 rows)
```

예제 13-04 · B-tree 인덱스 생성 후 - Index/Bitmap Scan 으로 바뀐다

```
CREATE INDEX idx_big_orders_customer ON big_orders (customer_id);
EXPLAIN (ANALYZE, BUFFERS OFF, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT * FROM big_orders WHERE customer_id = 12345;
```

▶ 실행 결과

```
CREATE INDEX
                                QUERY PLAN
-------------------------------------------------------------------------------
 Bitmap Heap Scan on big_orders (actual rows=9.00 loops=1)
   Recheck Cond: (customer_id = 12345)
   Heap Blocks: exact=9
   ->  Bitmap Index Scan on idx_big_orders_customer (actual rows=9.00 loops=1)
         Index Cond: (customer_id = 12345)
         Index Searches: 1
(6 rows)
```

■ TIP DBeaver 의 실행 계획 보기

쿼리에 커서를 두고 `Ctrl+Shift+E`(Explain Execution Plan)를 누르면 실행 계획이 트리 형태로 나오고, 노드마다 비용과 행 수를 표로 보여 줍니다. 처음에는 이 화면으로 구조를 익히고, 정확한 실제 값이 필요할 때는 `EXPLAIN (ANALYZE) ...` 를 직접 실행해 보세요. 결과 그리드에 한 줄씩 나오는 QUERY PLAN 은 오른쪽 클릭 → Copy 로 복사해 읽으면 편합니다.

■ 핵심 포인트 인덱스 전/후 비교

13-03 의 `Rows Removed by Filter: 333330` (× 3개 프로세스)은 9건을 찾으려고 100만 건을 읽었다는 뜻입니다. 인덱스를 만든 13-04 에서는 인덱스로 9건의 위치만 찾습니다. 같은 쿼리를 인덱스 전/후로 실행해 DBeaver 결과창 아래의 실행 시간을 비교하면 수십 ms → 1ms 안팎으로 줄어드는 것을 직접 볼 수 있습니다.

예제 13-05 · psql 의 \timing 으로 속도 비교 (숫자는 환경마다 다름)

```
-- \timing on
-- SELECT count(*) FROM big_orders WHERE customer_id = 12345;  -- 인덱스 O : 수 ms
-- DROP INDEX idx_big_orders_customer;
-- SELECT count(*) FROM big_orders WHERE customer_id = 12345;  -- 인덱스 X : 수십 ms
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'big_orders';
```

▶ 실행 결과

```
        indexname        ┆                                      indexdef
-------------------------+-------------------------------------------------------------------------------------
 big_orders_pkey         ┆ CREATE UNIQUE INDEX big_orders_pkey ON public.big_orders USING btree (id)
 idx_big_orders_customer ┆ CREATE INDEX idx_big_orders_customer ON public.big_orders USING btree (customer_id)
(2 rows)
```

복합 인덱스

여러 컬럼을 묶은 인덱스는 **왼쪽 컬럼부터 정렬**되어 있습니다(전화번호부가 성 → 이름 순인 것과 같음). 그래서 `(status, order_date)` 인덱스는 `status` 조건이 있을 때 가장 효과적입니다.

예제 13-06 · 복합 인덱스 - 컬럼 순서가 중요하다 (왼쪽부터 사용)

```
CREATE INDEX idx_big_status_date ON big_orders (status, order_date);
EXPLAIN (COSTS OFF)
SELECT count(*) FROM big_orders
WHERE status = 'CANCELLED' AND order_date >= '2026-01-01';
-- 두 번째 컬럼만 조건으로 쓰면: 17 이하 버전은 인덱스를 거의 못 쓰고,
-- 18 부터는 선두 컬럼 값의 종류가 적을 때 skip scan 으로 활용할 수 있다
EXPLAIN (COSTS OFF)
SELECT count(*) FROM big_orders WHERE order_date >= '2026-06-01';
```

▶ 실행 결과

```
CREATE INDEX
                                                
```

패턴	이유	해결
`WHERE col + 0 = 5`, `WHERE lower(email) = ...`	컬럼을 가공하면 인덱스의 정렬 순서와 달라짐	가공하지 않게 쿼리 수정, 또는 **표현식 인덱스**
`WHERE name LIKE '%노트북'`	앞부분이 정해지지 않음	pg_trgm GIN 인덱스(16장)
`WHERE price::text = '1000'`	형변환	컬럼 타입에 맞는 값으로 비교
결과가 테이블의 상당 부분	인덱스로 하나씩 읽는 것보다 전체 읽기가 빠름	정상 (옵티마이저의 올바른 판단)
통계가 오래됨	행 수 추정이 틀림	`ANALYZE` (18장)

표현식 인덱스와 부분 인덱스

예제 13-08 · 표현식 인덱스 - 대소문자 무시 검색

```
CREATE INDEX idx_customers_lower_email ON customers (lower(email));
SET enable_seqscan = off;                -- 작은 테이블이라 강제로 인덱스 사용을 확인
EXPLAIN (COSTS OFF)
SELECT * FROM customers WHERE lower(email) = 'minjun@example.com';
RESET enable_seqscan;
```

▶ 실행 결과

```
CREATE INDEX
SET
                           QUERY PLAN
-----------------------------------------------------------------
 Index Scan using idx_customers_lower_email on customers
   Index Cond: (lower((email)::text) = 'minjun@example.com'::text)
(2 rows)

RESET
```

예제 13-09 · 부분 인덱스 - 자주 찾는 일부 행만 인덱싱

```
CREATE INDEX idx_big_pending ON big_orders (order_date) WHERE status = 'PAID';
SELECT indexname, pg_size_pretty(pg_relation_size(indexname::regclass)) AS 크기
FROM pg_indexes WHERE tablename = 'big_orders' ORDER BY indexname;
```

▶ 실행 결과

```
CREATE INDEX
        indexname        |  크기
-------------------------+---------
 big_orders_pkey         | 21 MB
 idx_big_orders_customer | 9088 kB
 idx_big_pending         | 3688 kB
 idx_big_status_date     | 34 MB
(4 rows)
```

> ■ 참고 **enable_seqscan = off 는 실습용**
> shop 의 customers 는 20행뿐이라 인덱스보다 전체 읽기가 빠르므로 옵티마이저가 인덱스를 쓰지 않습니다. 인덱스를 **쓸 수 있는지** 확인하려고 잠시 Seq Scan 을 끈 것이며, 운영 환경에서는 절대 바꾸지 않습니다.

부분 인덱스(13-09)는 `WHERE status = 'PAID'` 처럼 자주 찾는 일부 행만 인덱싱해 크기가 1/10 수준입니다. "처리 대기 주문", "삭제되지 않은 행" 처럼 소수 행을 반복 조회하는 경우에 효과적입니다.

LIKE 와 인덱스

예제 13-10 · LIKE 검색과 인덱스 - 앞부분 일치만 B-tree 가 도움

```
CREATE INDEX idx_products_name ON products (name text_pattern_ops);
SET enable_seqscan = off;
EXPLAIN (COSTS OFF) SELECT * FROM products WHERE name LIKE '스마트%';   -- 인덱스 가능
EXPLAIN (COSTS OFF) SELECT * FROM products WHERE name LIKE '%노트북';   -- 불가
RESET enable_seqscan;
```

▶ 실행 결과

```
CREATE INDEX
SET
                                        QUERY PLAN
-------------------------------------------------------------------------------------------
 Index Scan using idx_products_name on products
   Index Cond: (((name)::text ~>=~ '스마트'::text) AND ((name)::text ~<~ '스마특'::text))
   Filter: ((name)::text ~~ '스마트%'::text)
(3 rows)

                 QUERY PLAN
--------------------------------------------
 Seq Scan on products
   Disabled: true
   Filter: ((name)::text ~~ '%노트북'::text)
(3 rows)

RESET
```

로케일이 C 가 아닌 데이터베이스(한국어 설치 기본값)에서 `LIKE '접두사%'` 에 B-tree 인덱스를 쓰려면 `text_pattern_ops` 연산자 클래스로 인덱스를 만들어야 합니다.

조인 계획과 외래 키 인덱스

예제 13-11 · JOIN 실행 계획 읽기 - Hash Join / Nested Loop

```
EXPLAIN (COSTS OFF)
SELECT c.name, count(*)
FROM customers c JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.name;
```

▶ 실행 결과

```
                     QUERY PLAN
-----------------------------------------------------
 HashAggregate
   Group Key: c.name
   ->  Hash Join
         Hash Cond: (o.customer_id = c.customer_id)
         ->  Seq Scan on orders o
         ->  Hash
               ->  Seq Scan on customers c
(7 rows)
```

예제 13-12 · 외래 키 컬럼에는 인덱스를 직접 만들어야 한다

```
SELECT conrelid::regclass AS 테이블, conname AS 외래키
FROM pg_constraint WHERE contype = 'f' AND conrelid = 'orders'::regclass;
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_order_items_product ON order_items (product_id);
```

▶ 실행 결과

```
 테이블 ┆         외래키
--------+------------------------
 orders ┆ orders_customer_id_fkey
 orders ┆ orders_emp_id_fkey
(2 rows)

CREATE INDEX
CREATE INDEX
```

■ 주의 외래 키 컬럼에는 인덱스를

`orders.customer_id` 에 인덱스가 없으면 "고객 1명의 주문 조회"와 "고객 삭제 시 참조 검사"가 모두 전체 스캔이 됩니다. PostgreSQL 은 FK 인덱스를 자동으로 만들지 않으므로 직접 만들어야 합니다. 신입 개발자들이 가장 많이 놓치는 부분입니다.

인덱스 관리

예제 13-13 · 사용되지 않는 인덱스 찾기 (통계 뷰)

```
SELECT relname AS 테이블, indexrelname AS 인덱스, idx_scan AS 사용횟수
FROM pg_stat_user_indexes
WHERE relname = 'big_orders'
ORDER BY indexrelname;
```

▶ 실행 결과

```
   테이블   ┆         인덱스          ┆ 사용횟수
------------+-------------------------+----------
 big_orders ┆ big_orders_pkey         ┆        0
 big_orders ┆ idx_big_orders_customer ┆        1
 big_orders ┆ idx_big_pending         ┆        0
 big_orders ┆ idx_big_status_date     ┆        0
(4 rows)
```

명령	용도
`CREATE INDEX CONCURRENTLY ...`	운영 중인 테이블에 쓰기를 막지 않고 인덱스 생성
`DROP INDEX [CONCURRENTLY] 이름`	인덱스 삭제
`REINDEX INDEX 이름`	부풀어 오른 인덱스 재생성
`pg_stat_user_indexes.idx_scan`	사용 횟수, 0 이면 삭제 후보

■ Q&A; 자주 묻는 질문

- **Q. 모든 컬럼에 인덱스를 만들면 좋지 않나요?** 쓰기가 느려지고 공간을 차지합니다. 실제로 느린 쿼리의 WHERE / JOIN / ORDER BY 컬럼을 근거로 만듭니다(18장 pg_stat_statements).
- **Q. 인덱스를 만들었는데 안 쓰여요.** ① 컬럼을 가공했는지 ② 결과 행이 많은지 ③ `ANALYZE` 를 했는지 ④ 테이블이 작은지 순서로 확인합니다.

실습 문제

번호	문제
13-Q1	orders 에서 (customer_id, order_date) 복합 인덱스를 만들고, 고객 8번의 최근 주문 3건 조회 계획을 확인하세요
13-Q2	다음 조건 중 일반 B-tree 인덱스(email)를 쓸 수 없는 것은? 표현식 인덱스로 해결하세요: WHERE upper(email) = 'JIHO@EXAMPLE.COM'

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- EXPLAIN 으로 확인하고, ANALYZE 로 실제를 확인한다. 추측하지 말고 측정한다.
- 복합 인덱스는 왼쪽 컬럼부터, 가공된 컬럼은 표현식 인덱스로.
- 외래 키 컬럼 인덱스는 직접 만든다.

14장 · 트랜잭션과 동시성

학습 목표

- 트랜잭션과 ACID 속성을 계좌 이체 예로 설명할 수 있다.
- BEGIN / COMMIT / ROLLBACK / SAVEPOINT 를 사용할 수 있다.
- MVCC 와 격리 수준(READ COMMITTED, REPEATABLE READ, SERIALIZABLE)의 차이를 설명할 수 있다.
- 갱신 손실과 데드락을 이해하고 `FOR UPDATE`, 원자적 UPDATE, 잠금 순서로 예방할 수 있다.

예제 파일: `examples/14_transaction.sql`, 두 세션 실험: `python/05_concurrency_demo.py` (19장)

트랜잭션과 ACID

트랜잭션은 **"모두 성공하거나 모두 실패해야 하는 작업 묶음"** 입니다. 계좌 이체에서 출금만 되고 입금이 안 되면 안 됩니다.

속성	의미	PostgreSQL 의 구현
Atomicity (원자성)	전부 반영되거나 전부 취소	실패 시 트랜잭션 전체 ROLLBACK
Consistency (일관성)	제약 조건을 항상 만족	CHECK, FK, UNIQUE 등
Isolation (격리성)	동시에 실행되는 트랜잭션끼리 간섭하지 않음	MVCC + 격리 수준 + 잠금
Durability (지속성)	COMMIT 된 결과는 장애 후에도 유지	WAL(선행 기록 로그)

psql 과 대부분의 도구는 **자동 커밋(autocommit)** 모드입니다. `BEGIN` 없이 실행한 문장은 각각 하나의 트랜잭션으로 즉시 커밋됩니다.

예제 14-01 · 트랜잭션 - 계좌 이체는 두 UPDATE 가 함께 성공해야 한다

```
BEGIN;
UPDATE accounts SET balance = balance - 30000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 30000 WHERE account_id = 2;
COMMIT;
SELECT * FROM accounts ORDER BY account_id;
```

▶ 실행 결과

```
BEGIN
UPDATE 1
UPDATE 1
COMMIT
 account_id ¦ owner  ¦ balance
------------+--------+---------
          1 ¦ 김철수 ¦   70000
          2 ¦ 이영희 ¦   80000
(2 rows)
```

예제 14-02 · ROLLBACK - 진행 중인 변경을 모두 취소

```
BEGIN;
DELETE FROM accounts;
SELECT count(*) AS 삭제후 FROM accounts;
ROLLBACK;
SELECT count(*) AS 롤백후 FROM accounts;
```

▶ 실행 결과

```
BEGIN
DELETE 2
 삭제후
--------
      0
(1 row)

ROLLBACK
 롤백후
--------
      2
(1 row)
```

■ TIP **DBeaver 의 트랜잭션 모드**

DBeaver 도 기본은 **Auto-Commit** 입니다. 이 상태에서도 예제처럼 `BEGIN; ... COMMIT;` 을 선택해 `Alt+X` 로 실행하면 그대로 동작합니다. 툴바의 트랜잭션 모드 버튼으로 **Manual Commit** 으로 바꾸면 BEGIN 없이도 모든 실행이 하나의 트랜잭션에 쌓이고, 툴바의 **Commit / Rollback** 버튼(또는 SQL 의 COMMIT/ROLLBACK)으로 끝냅니다. 미커밋 변경이 있으면 툴바에 건수가 표시됩니다. 실무에서 운영 DB 에 접속할 때는 Manual Commit 을 쓰는 습관이 사고를 막아 줍니다.

오류가 나면 트랜잭션 전체가 취소된다

예제 14-03 · 원자성(Atomicity) - 잔액 부족으로 두 번째 문장이 실패하면?

```
BEGIN;
UPDATE accounts SET balance = balance + 999999 WHERE account_id = 2;  -- 입금 먼저 (성공)
UPDATE accounts SET balance = balance - 999999 WHERE account_id = 1;  -- CHECK 위반 (실패)
SELECT * FROM accounts;          -- 오류 이후에는 어떤 명령도 거부된다
COMMIT;                          -- COMMIT 을 해도 실제로는 ROLLBACK 된다
```

▶ 실행 결과

```
BEGIN
UPDATE 1
ERROR:  new row for relation "accounts" violates check constraint "accounts_balance_check"
DETAIL:  Failing row contains (1, 김철수, -929999).
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ROLLBACK
```

예제 14-04 · 실패한 트랜잭션은 전부 취소되었다

```
SELECT * FROM accounts ORDER BY account_id;
```

▶ 실행 결과

```
 account_id ¦ owner  ¦ balance
------------+--------+---------
          1 ¦ 김철수 ¦   70000
          2 ¦ 이영희 ¦   80000
(2 rows)
```

■ **핵심 포인트** **"current transaction is aborted"**
PostgreSQL 은 트랜잭션 안에서 오류가 한 번 나면 이후 모든 명령을 거부하고, COMMIT 을 해도 ROLLBACK 으로 처리합니다(14-03 의 마지막 `ROLLBACK` 출력). DBeaver 에서 오류 후 아무것도 실행되지 않으면 대부분 이 상태이니 **툴바의 Rollback 을 누르거나 ROLLBACK 을 실행**하세요. 오류가 나도 일부를 살리고 싶으면 SAVEPOINT 를 씁니다.

예제 14-05 · SAVEPOINT - 일부만 되돌리기

```
BEGIN;
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 1;
SAVEPOINT sp1;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;
ROLLBACK TO SAVEPOINT sp1;       -- sp1 이후 작업만 취소
COMMIT;
SELECT * FROM accounts ORDER BY account_id;
```

▶ 실행 결과

```
BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
COMMIT
 account_id ¦ owner  ¦ balance
------------+--------+---------
          1 ¦ 김철수 ¦   71000
          2 ¦ 이영희 ¦   80000
(2 rows)
```

예제 14-06 · 격리 수준 확인과 설정

```
SHOW transaction_isolation;      -- 기본값: read committed
BEGIN ISOLATION LEVEL REPEATABLE READ;
SHOW transaction_isolation;
COMMIT;
```

▶ 실행 결과

```
 transaction_isolation
-----------------------
 read committed
(1 row)

BEGIN
 transaction_isolation
-----------------------
 repeatable read
(1 row)

COMMIT
```

MVCC - 읽기와 쓰기가 서로 막지 않는다

PostgreSQL 은 행을 수정할 때 덮어쓰지 않고 **새 버전을 추가**합니다(Multi-Version Concurrency Control). 각 버전에는 만든 트랜잭션 번호(xmin)와 지운 트랜잭션 번호(xmax)가 기록되고, 각 트랜잭션은 자신이 볼 수 있는 버전만 봅니다. 그래서 **읽기는 쓰기를 기다리지 않고, 쓰기도 읽기를 기다리지 않습니다.** 남은 옛 버전(dead tuple)은 VACUUM(18장)이 정리합니다.

예제 14-07 · MVCC - 행마다 숨어 있는 버전 정보(xmin, xmax)

```
SELECT xmin, xmax, account_id, balance FROM accounts ORDER BY account_id;
UPDATE accounts SET balance = balance WHERE account_id = 1;   -- 값이 같아도 새 버전 생성
SELECT xmin, xmax, account_id, balance FROM accounts ORDER BY account_id;
```

▶ 실행 결과

```
 xmin  ┆ xmax  ┆ account_id ┆ balance
-------+-------+------------+---------
 23967 ┆     0 ┆          1 ┆   71000
 23964 ┆ 23968 ┆          2 ┆   80000
(2 rows)

UPDATE 1
 xmin  ┆ xmax  ┆ account_id ┆ balance
-------+-------+------------+---------
 23969 ┆     0 ┆          1 ┆   71000
 23964 ┆ 23968 ┆          2 ┆   80000
(2 rows)
```

xmin/xmax 의 숫자는 환경마다 다릅니다. UPDATE 후 1번 행의 `xmin` 이 바뀐 것(새 버전)을 보여 주는 것이 핵심입니다.

격리 수준

격리 수준	Dirty Read	Non-repeatable Read	Phantom Read	직렬화 이상
READ UNCOMMITTED	(PostgreSQL 은 READ COMMITTED 로 동작)	가능	가능	가능
READ COMMITTED (기본)	없음	가능	가능	가능
REPEATABLE READ	없음	없음	없음 (PostgreSQL)	가능
SERIALIZABLE	없음	없음	없음	없음

- **READ COMMITTED**: 문장(SELECT)마다 그 시점에 커밋된 데이터를 봅니다. 같은 트랜잭션에서 두 번 읽으면 값이 다를 수 있습니다.
- **REPEATABLE READ**: 트랜잭션의 첫 문장 시점 스냅샷을 끝까지 봅니다. 다른 트랜잭션이 이미 고친 행을 수정하려 하면 `could not serialize access` 오류가 나고, 애플리케이션이 **재시도**해야 합니다.
- **SERIALIZABLE**: 트랜잭션들을 한 번에 하나씩 실행한 것과 같은 결과를 보장합니다. 충돌 시 오류 → 재시도.

두 세션 실습 (DBeaver 편집기 2개)

순서	세션 A	세션 B
1	`BEGIN; SELECT balance FROM accounts WHERE account_id = 1;` → 현재 잔액	

순서	세션 A	세션 B
2		`UPDATE accounts SET balance = 0 WHERE account_id = 1;` (자동 커밋)
3	`SELECT balance ...;` → **0** (READ COMMITTED)	
4	`COMMIT;` → 이번엔 `BEGIN ISOLATION LEVEL REPEATABLE READ;` 로 반복	
5	3단계에서 **처음 값 그대로** 보임	

DBeaver 는 기본적으로 **편집기마다 별도 연결**을 쓰므로(`Open separate connection for each editor`), SQL 편집기 두 개를 나란히 열면 세션 A, B 가 됩니다. 세션 A 편집기는 툴바에서 Manual Commit 으로 바꿔 두면 편합니다. 같은 실험을 Python 커넥션 2개로 자동 재현한 결과는 19장 `python/05_concurrency_demo.py` 에 있습니다(예제 ①, ②).

잠금과 갱신 손실

MVCC 덕분에 읽기는 막히지 않지만, **같은 행을 동시에 수정**하면 나중 트랜잭션은 앞 트랜잭션이 끝날 때까지 기다립니다(행 잠금).

예제 14-08 · 행 잠금 - SELECT ... FOR UPDATE 와 pg_locks

```
BEGIN;
SELECT * FROM accounts WHERE account_id = 1 FOR UPDATE;   -- 다른 세션은 이 행 수정 시 대기
SELECT l.locktype, l.mode, l.granted
FROM pg_locks l
WHERE l.relation = 'accounts'::regclass AND l.pid = pg_backend_pid();
COMMIT;
```

▶ 실행 결과

```
BEGIN
 account_id ¦ owner  ¦ balance
------------+--------+---------
          1 ¦ 김철수 ¦   71000
(1 row)

 locktype ¦     mode     ¦ granted
----------+--------------+---------
 relation ¦ RowShareLock ¦ t
(1 row)

COMMIT
```

■ 참고 **행 잠금은 pg_locks 에 보이지 않는다**
14-08 에서 pg_locks 에는 테이블 수준의 `RowShareLock` 만 보입니다. PostgreSQL 은 행 잠금 정보를 행 자체(xmax)에 기록하므로 행 잠금이 아무리 많아도 메모리를 차지하지 않습니다. 다른 세션이 기다리는 중이라면 그 세션의 `transactionid` 잠금 대기가 pg_locks 에 나타납니다.

갱신 손실(Lost Update): 두 세션이 같은 값(1000)을 읽고, 각자 계산해서(1000-300, 1000-200) 저장하면 먼저 저장한 결과가 사라집니다. 예방법은 세 가지입니다.

1. **원자적 UPDATE**: `SET balance = balance - 300` 처럼 DB 가 현재 값으로 계산 (14-09, 가장 간단)
2. **비관적 잠금**: `SELECT ... FOR UPDATE` 로 읽을 때 잠가 두기 (15장 `place_order` 함수)
3. **낙관적 잠금**: `version` 컬럼을 두고 `UPDATE ... WHERE id = ? AND version = ?` 의 결과 행 수가 0 이면 재시도

예제 14-09 · 갱신 손실 방지 - 읽고 계산하지 말고 한 문장으로 갱신

```
-- (나쁜 예) SELECT balance … → 앱에서 계산 → UPDATE SET balance = 계산값
-- (좋은 예) 데이터베이스가 원자적으로 계산
UPDATE accounts SET balance = balance - 10000
WHERE account_id = 1 AND balance >= 10000
RETURNING account_id, balance;
```

▶ 실행 결과

```
 account_id ¦ balance
------------+---------
          1 ¦   61000
(1 row)

UPDATE 1
```

예제 14-10 · 잠금 대기 시간 제한 - lock_timeout / NOWAIT

```
SET lock_timeout = '3s';
SHOW lock_timeout;
BEGIN;
SELECT account_id FROM accounts WHERE account_id = 2 FOR UPDATE NOWAIT;  -- 잠겨 있으면 즉시 오류
COMMIT;
RESET lock_timeout;
```

▶ 실행 결과

```
SET
 lock_timeout
--------------
 3s
(1 row)

BEGIN
 account_id
------------
          2
(1 row)

COMMIT
RESET
```

데드락

세션 A 가 1번 행을 잠그고 2번을 기다리는데, 세션 B 는 2번을 잠그고 1번을 기다리면 영원히 끝나지 않습니다. PostgreSQL 은 이를 감지(기본 1초, `deadlock_timeout`)해 한쪽을 `deadlock detected` 오류로 취소합니다.

순서	세션 A	세션 B
1	`BEGIN; UPDATE accounts ... WHERE account_id = 1;`	
2		`BEGIN; UPDATE accounts ... WHERE account_id = 2;`
3	`UPDATE ... WHERE account_id = 2;` → 대기	
4		`UPDATE ... WHERE account_id = 1;` → **deadlock detected** (취소됨)
5	B 가 취소되며 A 의 UPDATE 진행 → `COMMIT;`	`ROLLBACK;`

■ TIP 데드락 예방

- 여러 행을 잠글 때는 **항상 같은 순서**(예: id 오름차순)로 잠급니다.
- 트랜잭션을 짧게 유지합니다. 트랜잭션 안에서 사용자 입력이나 외부 API 호출을 기다리지 않습니다.
- `idle in transaction` 상태로 오래 남은 세션은 잠금을 붙잡고 있으므로 18장의 방법으로 찾아 정리합니다.

■ Q&A; 자주 묻는 질문

- **Q. 격리 수준을 무조건 SERIALIZABLE 로 하면 안전하지 않나요?** 안전하지만 충돌 시 오류가 자주 나고 모든 코드에 재시도 로직이 필요합니다. 대부분의 서비스는 기본값(READ COMMITTED) + 원자적 UPDATE / FOR UPDATE 로 충분합니다.
- **Q. DDL 도 롤백되나요?** PostgreSQL 은 CREATE/ALTER/DROP TABLE 도 트랜잭션 안에서 롤백됩니다(일부 예외: CREATE DATABASE, CREATE INDEX CONCURRENTLY, VACUUM). 다른 DBMS 와 다른 큰 장점입니다.

실습 문제

번호	문제
14-Q1	wallet(id, balance≥0) 에서 1번→2번으로 7,000원 이체를 트랜잭션으로 처리하되 잔액이 부족하면 전체 취소되는 것을 확인하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 트랜잭션 안의 오류는 전체 취소. 부분 취소는 SAVEPOINT.
- MVCC: 읽기와 쓰기가 서로 막지 않는다. 같은 행 쓰기끼리만 기다린다.
- 갱신 손실은 원자적 UPDATE / FOR UPDATE / 버전 컬럼으로, 데드락은 일정한 잠금 순서로 예방.

15장 · 함수, 프로시저, 트리거 - PL/pgSQL

학습 목표

- SQL 함수와 PL/pgSQL 함수를 만들고 호출할 수 있다.
- 변수, IF, LOOP, 예외 처리(EXCEPTION), RAISE 를 사용할 수 있다.
- 함수와 프로시저(CALL)의 차이를 설명할 수 있다.
- BEFORE / AFTER 트리거로 데이터 보정과 변경 이력을 자동화할 수 있다.

예제 파일: `examples/15_plpgsql.sql`

함수의 기본 구조

```
CREATE OR REPLACE FUNCTION 함수명(매개변수 타입, ...)
RETURNS 반환타입
LANGUAGE plpgsql            -- 또는 sql
AS $$
DECLARE
    변수 타입;
BEGIN
    ...
    RETURN 값;
END;
$$;
```

`$$ ... $$` 는 **달러 인용(dollar quoting)** 으로, 함수 본문 안에서 작은따옴표를 이스케이프하지 않고 쓸 수 있게 해 줍니다.

구분	LANGUAGE sql	LANGUAGE plpgsql
내용	SQL 문 하나(또는 여러 개)	변수, IF, LOOP, 예외 처리 가능
장점	간단, 인라인 최적화 가능	절차적 로직
예	부가세 계산, 리포트 조회	주문 처리, 등급 계산

함수 변동성(volatility) 표시: `IMMUTABLE`(같은 입력 → 항상 같은 결과, 예: 계산), `STABLE`(한 쿼리 안에서는 같은 결과, 예: 조회), `VOLATILE`(기본값, 데이터 변경·난수).

예제 15-01 · SQL 함수 - 가장 간단한 사용자 정의 함수

```
CREATE OR REPLACE FUNCTION vat(price NUMERIC)
RETURNS NUMERIC
LANGUAGE sql IMMUTABLE
AS $$
    SELECT round(price * 1.1);
$$;
SELECT name, price, vat(price) AS 부가세포함 FROM products WHERE product_id <= 3;
```

▶ 실행 결과

```
CREATE FUNCTION
        name        ¦  price  ¦ 부가세포함
--------------------+---------+------------
 울트라북 14        ¦ 1290000 ¦    1419000
 비즈니스 노트북 15 ¦  990000 ¦    1089000
 게이밍 노트북 X17  ¦ 2490000 ¦    2739000
(3 rows)
```

예제 15-02 · PL/pgSQL 함수 - 변수, IF, RETURN

```
CREATE OR REPLACE FUNCTION customer_level(p_customer_id INTEGER)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COALESCE(sum(oi.quantity * oi.unit_price), 0)
      INTO v_total
    FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.customer_id = p_customer_id AND o.status <> 'CANCELLED';
    IF v_total >= 10000000 THEN
        RETURN 'VIP';
    ELSIF v_total >= 5000000 THEN
        RETURN 'GOLD';
    ELSIF v_total > 0 THEN
        RETURN 'SILVER';
    ELSE
        RETURN 'NEW';
    END IF;
END;
$$;
SELECT customer_id, name, grade AS 현재등급, customer_level(customer_id) AS 계산등급
FROM customers WHERE customer_id IN (1, 4, 10, 19);
```

▶ 실행 결과

```
CREATE FUNCTION
 customer_id ¦  name  ¦ 현재등급 ¦ 계산등급
-------------+--------+----------+----------
           1 ¦ 김민준 ¦ VIP      ¦ GOLD
           4 ¦ 최지우 ¦ BRONZE   ¦ SILVER
          10 ¦ 임준서 ¦ GOLD     ¦ VIP
          19 ¦ 류다은 ¦ BRONZE   ¦ NEW
(4 rows)
```

■ 핵심 포인트 매개변수 이름 규칙

15-02 의 매개변수 `p_customer_id`, 변수 `v_total` 처럼 접두사를 붙이세요. 매개변수 이름이 컬럼 이름과 같으면(`customer_id`) 쿼리 안에서 어느 쪽인지 모호하다는 오류(`column reference is ambiguous`)가 납니다.

예제 15-03 · 테이블을 반환하는 함수 (RETURNS TABLE)

```
CREATE OR REPLACE FUNCTION top_products(p_limit INTEGER DEFAULT 3)
RETURNS TABLE (product_name VARCHAR, total_qty BIGINT)
LANGUAGE sql STABLE
AS $$
    SELECT p.name, sum(oi.quantity)
    FROM order_items oi JOIN products p ON p.product_id = oi.product_id
    GROUP BY p.name
    ORDER BY 2 DESC, 1
    LIMIT p_limit;
$$;
SELECT * FROM top_products();
SELECT * FROM top_products(p_limit => 5);
```

▶ 실행 결과

```
CREATE FUNCTION
 product_name  ¦ total_qty
---------------+-----------
 콜드브루 원액 ¦        54
 견과류 믹스   ¦        36
 유기농 녹차   ¦        36
```

```
(3 rows)

  product_name  ¦ total_qty
----------------+-----------
 콜드브루 원액  ¦        54
 견과류 믹스    ¦        36
 유기농 녹차    ¦        36
 수제 쿠키 세트 ¦        32
 기계식 키보드  ¦        28
(5 rows)
```

예제 15-04 · 반복문(FOR ... LOOP)과 RAISE NOTICE

```
DO $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN SELECT name, stock FROM products WHERE stock < 10 ORDER BY stock LOOP
        RAISE NOTICE '재고 부족: % (현재 %개)', r.name, r.stock;
    END LOOP;
END;
$$;
```

▶ 실행 결과

```
NOTICE:  재고 부족: USB-C 허브 (현재 0개)
NOTICE:  재고 부족: 탄산수 24입 (현재 0개)
NOTICE:  재고 부족: 남성 정장 세트 (현재 5개)
NOTICE:  재고 부족: 게이밍 노트북 X17 (현재 6개)
DO
```

■ TIP DBeaver 에서 함수 만들고 NOTICE 보기

- `CREATE FUNCTION ... $$ ... $$;` 처럼 긴 문장은 **전체를 선택한 뒤 실행**합니다. 예제 파일의 함수 본문에는 빈 줄을 넣지 않아 `Ctrl+Enter` 로도 한 문장으로 인식되지만, 직접 작성할 때 본문에 빈 줄을 넣으면 DBeaver 가 문장을 잘라 `unterminated dollar-quoted string` 오류가 납니다.
- `RAISE NOTICE` 메시지는 결과 그리드가 아니라 **서버 출력(Output) 창**에 나옵니다. `Ctrl+Shift+O` 로 창을 열어 두세요.
- 만든 함수·트리거는 Navigator 의 `Functions`, 테이블의 `Triggers` 에서 확인하고 소스를 편집할 수 있습니다.

예외 처리와 업무 규칙

예제 15-05 · 예외 처리 - EXCEPTION 블록

```
CREATE OR REPLACE FUNCTION safe_divide(a NUMERIC, b NUMERIC)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN a / b;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE '0 으로 나눌 수 없어 NULL 을 반환합니다';
        RETURN NULL;
END;
$$;
SELECT safe_divide(10, 4) AS 정상, safe_divide(10, 0) AS 영나누기;
```

▶ 실행 결과

```
CREATE FUNCTION
NOTICE:  0 으로 나눌 수 없어 NULL 을 반환합니다
        정상        ¦ 영나누기
--------------------+----------
```

```
 2.5000000000000000 |
(1 row)
```

RAISE 의 수준: DEBUG, LOG, INFO, NOTICE(화면에 표시), WARNING, EXCEPTION(오류 발생 → 트랜잭션 취소).

15-06 의 place_order 는 재고 확인 → 주문 생성 → 주문 상세 → 재고 차감을 하나의 함수로 묶었습니다. 함수는 호출한 문장의 트랜잭션 안에서 실행되므로 중간에 오류가 나면 **모든 변경이 함께 취소**됩니다. FOR UPDATE 로 상품 행을 잠가 동시 주문 시 재고가 음수가 되는 것(14장 갱신 손실)을 막았습니다.

예제 15-06 · RAISE EXCEPTION - 업무 규칙 위반 시 오류 발생

```
CREATE OR REPLACE FUNCTION place_order(p_customer INTEGER, p_product INTEGER, p_qty INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_stock INTEGER;
    v_price NUMERIC;
    v_order INTEGER;
BEGIN
    SELECT stock, price INTO v_stock, v_price
    FROM products WHERE product_id = p_product
    FOR UPDATE;                                    -- 동시 주문 대비 잠금
    IF NOT FOUND THEN
        RAISE EXCEPTION '상품 %이(가) 없습니다', p_product;
    END IF;
    IF v_stock < p_qty THEN
        RAISE EXCEPTION '재고 부족: 요청 %개, 재고 %개', p_qty, v_stock
              USING HINT = '수량을 줄여 다시 주문하세요';
    END IF;
    INSERT INTO orders (customer_id, order_date, status, ship_city)
    SELECT p_customer, now(), 'PAID', city FROM customers WHERE customer_id = p_customer
    RETURNING order_id INTO v_order;
    INSERT INTO order_items VALUES (v_order, p_product, p_qty, v_price);
    UPDATE products SET stock = stock - p_qty WHERE product_id = p_product;
    RETURN v_order;
END;
$$;
SELECT place_order(1, 3, 100);       -- 재고 6개뿐
```

▶ 실행 결과

```
CREATE FUNCTION
ERROR:  재고 부족: 요청 100개, 재고 6개
HINT:  수량을 줄여 다시 주문하세요
CONTEXT:  PL/pgSQL function place_order(integer,integer,integer) line 14 at RAISE
```

예제 15-07 · 정상 주문 - 함수 하나가 하나의 트랜잭션으로 실행된다

```
SELECT place_order(1, 3, 2) AS 새주문번호;
SELECT stock FROM products WHERE product_id = 3;
```

▶ 실행 결과

```
 새주문번호
------------
        121
(1 row)

 stock
-------
     4
(1 row)
```

프로시저

구분	FUNCTION	PROCEDURE (11+)
호출	`SELECT f()` (쿼리 안에서 사용 가능)	`CALL p()`
반환값	있음	없음 (OUT 매개변수는 가능)
트랜잭션 제어	불가	내부에서 `COMMIT` / `ROLLBACK` 가능
용도	계산, 조회, 단일 업무 처리	배치 작업 (대량 데이터를 나눠 커밋)

예제 15-08 · 프로시저(PROCEDURE) - CALL 로 실행, 내부에서 COMMIT 가능

```
CREATE OR REPLACE PROCEDURE cancel_old_pending(p_days INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cnt INTEGER;
BEGIN
    UPDATE orders SET status = 'CANCELLED'
    WHERE status = 'PENDING' AND order_date < now() - make_interval(days => p_days);
    GET DIAGNOSTICS v_cnt = ROW_COUNT;
    RAISE NOTICE '% 건 자동 취소', v_cnt;
    COMMIT;
END;
$$;
INSERT INTO orders (customer_id, order_date, status, ship_city)
VALUES (2, now() - interval '10 days', 'PENDING', '부산');
CALL cancel_old_pending(7);
```

▶ 실행 결과

```
CREATE PROCEDURE
INSERT 0 1
NOTICE:  1 건 자동 취소
CALL
```

트리거

트리거는 테이블에 INSERT / UPDATE / DELETE 가 일어날 때 **자동으로 실행되는 함수**입니다.

구분	설명
`BEFORE`	변경 전에 실행. `NEW` 를 고쳐서 저장될 값을 바꾸거나, `RETURN NULL` 로 변경을 취소
`AFTER`	변경 후 실행. 이력 기록, 다른 테이블 갱신
`FOR EACH ROW` / `STATEMENT`	행마다 / 문장마다 한 번
`OLD` / `NEW`	변경 전 행 / 변경 후 행 (DELETE 는 OLD 만, INSERT 는 NEW 만)
`WHEN (조건)`	조건이 참일 때만 실행

예제 15-09 · 트리거 1 - 가격 변경 이력 자동 기록 (AFTER UPDATE)

```
CREATE TABLE product_price_log (
    log_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id INTEGER,
    old_price  NUMERIC,
    new_price  NUMERIC,
    changed_by TEXT DEFAULT current_user,
    changed_at TIMESTAMP DEFAULT now()
);
CREATE OR REPLACE FUNCTION trg_log_price()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO product_price_log (product_id, old_price, new_price)
    VALUES (OLD.product_id, OLD.price, NEW.price);
    RETURN NEW;
END;
$$;
CREATE TRIGGER products_price_audit
AFTER UPDATE OF price ON products
FOR EACH ROW
WHEN (OLD.price IS DISTINCT FROM NEW.price)
EXECUTE FUNCTION trg_log_price();

UPDATE products SET price = price - 10000 WHERE product_id IN (6, 7);
UPDATE products SET stock = stock + 1 WHERE product_id = 6;      -- 가격 변경 아님 → 기록 X
SELECT product_id, old_price, new_price, changed_by FROM product_price_log;
```

▶ 실행 결과

```
CREATE TABLE
CREATE FUNCTION
CREATE TRIGGER
UPDATE 2
UPDATE 1
 product_id ¦ old_price ¦ new_price ¦ changed_by
------------+-----------+-----------+------------
          6 ¦     29000 ¦     19000 ¦ postgres
          7 ¦     89000 ¦     79000 ¦ postgres
(2 rows)
```

예제 15-10 · 트리거 2 - 데이터 검증/보정 (BEFORE INSERT OR UPDATE)

```
CREATE OR REPLACE FUNCTION trg_normalize_customer()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.email := lower(trim(NEW.email));                -- 이메일 정규화
    NEW.phone := regexp_replace(NEW.phone, '[^0-9]', '', 'g');
    IF length(NEW.phone) = 11 THEN                       -- 01012345678 → 010-1234-5678
        NEW.phone := substr(NEW.phone, 1, 3) ¦¦ '-' ¦¦ substr(NEW.phone, 4, 4)
                     ¦¦ '-' ¦¦ substr(NEW.phone, 8);
    END IF;
    RETURN NEW;                                          -- 수정된 NEW 가 저장된다
END;
$$;
CREATE TRIGGER customers_normalize
BEFORE INSERT OR UPDATE ON customers
FOR EACH ROW EXECUTE FUNCTION trg_normalize_customer();

INSERT INTO customers (name, email, phone, city)
VALUES ('트리거테스트', '  Test.User@Example.COM ', '010 5555 1234', '서울')
RETURNING customer_id, email, phone;
```

▶ 실행 결과

```
CREATE FUNCTION
CREATE TRIGGER
 customer_id ¦         email          ¦     phone
-------------+----------------------+---------------
          21 ¦ test.user@example.com ¦ 010-5555-1234
(1 row)

INSERT 0 1
```

예제 15-11 · 함수/트리거 목록 확인

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'public'
ORDER BY routine_type, routine_name;
SELECT tgname, tgrelid::regclass AS 테이블
FROM pg_trigger WHERE NOT tgisinternal ORDER BY tgname;
```

▶ 실행 결과

```
      routine_name      ¦ routine_type
------------------------+--------------
 customer_level         ¦ FUNCTION
 place_order            ¦ FUNCTION
 safe_divide            ¦ FUNCTION
 top_products           ¦ FUNCTION
 trg_log_price          ¦ FUNCTION
 trg_normalize_customer ¦ FUNCTION
 vat                    ¦ FUNCTION
 cancel_old_pending     ¦ PROCEDURE
(8 rows)

        tgname        ¦  테이블
----------------------+-----------
 customers_normalize  ¦ customers
 products_price_audit ¦ products
(2 rows)
```

■ 주의 트리거는 보이지 않는 로직

트리거는 편리하지만 SQL 만 봐서는 실행되는지 알 수 없어 디버깅이 어렵습니다. 감사 로그, `updated_at` 자동 갱신, 데이터 정규화처럼 **"항상 반드시"** 일어나야 하는 일에만 쓰고, 업무 로직 대부분은 애플리케이션이나 명시적 함수 호출에 두도록 권장합니다. 트리거 안에서 같은 테이블을 다시 수정하면 무한 반복될 수 있으니 주의합니다.

■ Q&A; 자주 묻는 질문

- **Q. 함수를 수정하려면?** `CREATE OR REPLACE FUNCTION` 으로 다시 만듭니다. 매개변수나 반환 타입을 바꾸려면 `DROP FUNCTION 이름(타입들)` 후 다시 만듭니다.
- **Q. PL/pgSQL 디버깅은?** `RAISE NOTICE '값: %', 변수;` 로 중간값을 찍는 것이 가장 간단합니다. DBeaver 에서는 Output 창(`Ctrl+Shift+O`)에서 확인합니다.

실습 문제

번호	문제
15-Q1	상품 id 를 받아 '상품명 (분류명)' 문자열을 돌려주는 함수 product_label 을 만드세요
15-Q2	wallet 잔액이 바뀔 때마다 wallet_log(id, before, after, at)에 기록하는 트리거를 만드세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 단순 계산·조회는 SQL 함수, 절차적 로직은 PL/pgSQL.
- 함수 안의 오류는 호출한 트랜잭션 전체를 취소한다. 트랜잭션 제어가 필요하면 프로시저.
- 트리거는 BEFORE(값 보정), AFTER(이력 기록). 꼭 필요한 곳에만.

16장 · JSONB, 배열, 전문 검색

학습 목표

- JSONB 컬럼에서 값을 꺼내고(->, ->>, #>>), 검색하고(@>, ?), 수정할 수 있다.
- 테이블 데이터를 JSON 으로 만들어 API 응답 형태로 돌려줄 수 있다.
- 배열 타입을 만들고 ANY, @>, &&, unnest 로 다룰 수 있다.
- GIN 인덱스, 전문 검색(tsvector), pg_trgm 의 쓰임새를 설명할 수 있다.

예제 파일: examples/16_jsonb_array_fts.sql

JSON 과 JSONB

구분	JSON	JSONB
저장 형태	입력한 텍스트 그대로	분해한 이진 형태 (키 순서·공백·중복 키 정리)
조회 속도	매번 파싱 → 느림	빠름
인덱스	불가	GIN 인덱스 가능
권장	입력 원문을 그대로 보관해야 할 때만	**거의 항상 JSONB**

상품마다 속성이 다른 경우(노트북은 RAM, 옷은 사이즈) 컬럼을 계속 늘리는 대신 attrs JSONB 에 담으면 유연합니다. 단, 모든 상품에 공통이고 자주 검색·조인하는 값(가격, 분류)은 일반 컬럼으로 둡니다.

연산자	의미	예
->	키/인덱스로 꺼내기 (결과 JSONB)	attrs -> 'sizes'
->>	꺼내서 **텍스트**로	attrs ->> 'brand'
#> / #>>	경로로 꺼내기	attrs #>> '{sizes,1}'
@>	포함하는가 (GIN 인덱스 사용)	attrs @> '{"brand":"한빛"}'
?	키가 있는가	attrs ? 'sizes'
|| / -	합치기(추가·덮어쓰기) / 키 삭제	attrs || '{"a":1}'

예제 16-01 · JSONB 컬럼 조회 - 키로 값 꺼내기 (-> 는 JSON, ->> 는 텍스트)

```
SELECT name,
       attrs -> 'brand'   AS brand_json,
       attrs ->> 'brand'  AS brand_text,
       attrs ->> 'color'  AS color
FROM products
WHERE product_id IN (1, 4, 6, 18);
```

▶ 실행 결과

```
     name      ¦ brand_json ¦ brand_text ¦ color
---------------+------------+------------+--------
 울트라북 14   ¦ "한빛"     ¦ 한빛       ¦ silver
 스마트폰 S25  ¦ "누리"     ¦ 누리       ¦ blue
 무선 마우스   ¦ "가온"     ¦ 가온       ¦ black
 노트북 거치대 ¦            ¦            ¦
(4 rows)
```

예제 16-02 · 중첩 값과 배열 원소 꺼내기 (#>>, -> 인덱스)

```
SELECT name,
       attrs -> 'sizes'           AS sizes,
       attrs -> 'sizes' ->> 0     AS 첫사이즈,
       attrs #>> '{sizes,1}'      AS 두번째사이즈,
       jsonb_array_length(attrs -> 'sizes') AS 사이즈수
FROM products
WHERE attrs ? 'sizes';                      -- ? : 키가 존재하는가
```

▶ 실행 결과

```
    name     ┆       sizes        ┆ 첫사이즈 ┆ 두번째사이즈 ┆ 사이즈수
-------------+--------------------+----------+--------------+----------
 남성 셔츠   ┆ ["M", "L", "XL"]   ┆ M        ┆ L            ┆        3
 남성 청바지 ┆ ["30", "32", "34"] ┆ 30       ┆ 32           ┆        3
 여성 원피스 ┆ ["S", "M"]         ┆ S        ┆ M            ┆        2
 여성 니트   ┆ ["S", "M", "L"]    ┆ S        ┆ M            ┆        3
(4 rows)
```

예제 16-03 · JSONB 로 검색 - 포함 연산자 @> 와 형변환

```
SELECT name, attrs ->> 'brand' AS brand
FROM products
WHERE attrs @> '{"brand": "한빛"}';
SELECT name, (attrs ->> 'ram_gb')::int AS ram
FROM products
WHERE (attrs ->> 'ram_gb')::int >= 16;
```

▶ 실행 결과

```
       name        ┆ brand
-------------------+-------
 울트라북 14       ┆ 한빛
 게이밍 노트북 X17 ┆ 한빛
 기계식 키보드     ┆ 한빛
(3 rows)

       name        ┆ ram
-------------------+-----
 울트라북 14       ┆  16
 게이밍 노트북 X17 ┆  32
(2 rows)
```

■ 핵심 포인트 -> 와 ->> 구분

16-01 의 brand_json 은 따옴표가 붙은 JSON 문자열("한빛"), brand_text 는 일반 텍스트입니다. **비교·출력은 ->>**, 더 깊이 들어가려면 -> 를 이어서 씁니다. 숫자 비교는 (attrs ->> 'ram_gb')::int 처럼 형변환이 필요합니다.

예제 16-04 · JSONB 수정 - 키 추가/변경(||, jsonb_set), 삭제(-)

```
UPDATE products
SET attrs = attrs || '{"warranty_years": 2}'
WHERE product_id = 1;
UPDATE products
SET attrs = jsonb_set(attrs, '{color}', '"space gray"')
WHERE product_id = 1;
UPDATE products SET attrs = attrs - 'weight_kg' WHERE product_id = 1;
SELECT attrs FROM products WHERE product_id = 1;
```

▶ 실행 결과

```
UPDATE 1
UPDATE 1
UPDATE 1
                                    attrs
-------------------------------------------------------------------------------
 {"brand": "한빛", "color": "space gray", "ram_gb": 16, "warranty_years": 2}
```

```
(1 row)
```

예제 16-05 · JSONB 를 행으로 펼치기 - jsonb_each_text

```
SELECT p.name, kv.key, kv.value
FROM products p, jsonb_each_text(p.attrs) AS kv
WHERE p.product_id = 3;
```

▶ 실행 결과

```
      name        |    key    | value
------------------+-----------+-------
 게이밍 노트북 X17 | gpu       | RTX
 게이밍 노트북 X17 | brand     | 한빛
 게이밍 노트북 X17 | color     | black
 게이밍 노트북 X17 | ram_gb    | 32
 게이밍 노트북 X17 | weight_kg | 2.6
(5 rows)
```

예제 16-06 · 테이블 → JSON 만들기 (API 응답 형태)

```
SELECT jsonb_build_object(
           'customer', c.name,
           'grade',    c.grade,
           'orders',   (SELECT jsonb_agg(jsonb_build_object('id', o.order_id,
                                                            'date', o.order_date::date)
                                         ORDER BY o.order_id)
                        FROM orders o
                        WHERE o.customer_id = c.customer_id AND o.order_id < 20)
       ) AS result
FROM customers c
WHERE c.customer_id = 10;
```

▶ 실행 결과

```
                                                                                                                                                  result
-------------------------------------------------------------------------------------------------------------------------------------------------------
 {"grade": "GOLD", "orders": [{"id": 1, "date": "2025-01-03"}, {"id": 2, "date": "2025-01-04"}], "customer": "임준서"}
(1 row)
```

GIN 인덱스

GIN(Generalized Inverted Index)은 JSONB 의 키·값, 배열 원소, 검색어처럼 **한 행에 여러 값이 들어 있는 컬럼**을 위한 역색인입니다.

예제 16-07 · JSONB 인덱스 - GIN

```
CREATE TABLE events (
    id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    payload JSONB NOT NULL
);
INSERT INTO events (payload)
SELECT jsonb_build_object('type', (ARRAY['click','view','buy'])[1 + g % 3],
                          'user', g % 1000,
                          'page', '/p/' || (g % 50))
FROM generate_series(1, 30000) g;
CREATE INDEX idx_events_payload ON events USING GIN (payload);
ANALYZE events;
EXPLAIN (COSTS OFF)
SELECT count(*) FROM events WHERE payload @> '{"type": "buy", "user": 7}';
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 30000
CREATE INDEX
ANALYZE
```

```
                                QUERY PLAN
---------------------------------------------------------------------------
 Aggregate
   ->  Bitmap Heap Scan on events
         Recheck Cond: (payload @> '{"type": "buy", "user": 7}'::jsonb)
         ->  Bitmap Index Scan on idx_events_payload
               Index Cond: (payload @> '{"type": "buy", "user": 7}'::jsonb)
(5 rows)
```

배열

예제 16-08 · 배열 타입 - 생성, 원소 접근, 포함 검사

```
ALTER TABLE products ADD COLUMN tags TEXT[] DEFAULT '{}';
UPDATE products SET tags = ARRAY['신상품', '무료배송'] WHERE product_id IN (1, 4);
UPDATE products SET tags = ARRAY['무료배송']          WHERE product_id IN (6, 7, 8);
UPDATE products SET tags = ARRAY['할인', '무료배송']  WHERE product_id = 12;
SELECT name, tags, tags[1] AS 첫태그, cardinality(tags) AS 태그수
FROM products WHERE cardinality(tags) > 0 ORDER BY product_id;
```

▶ 실행 결과

```
ALTER TABLE
UPDATE 2
UPDATE 3
UPDATE 1
     name      ┆       tags        ┆  첫태그  ┆ 태그수
---------------+-------------------+----------+--------
 울트라북 14   ┆ {신상품,무료배송} ┆ 신상품   ┆      2
 스마트폰 S25  ┆ {신상품,무료배송} ┆ 신상품   ┆      2
 무선 마우스   ┆ {무료배송}        ┆ 무료배송 ┆      1
 기계식 키보드 ┆ {무료배송}        ┆ 무료배송 ┆      1
 27인치 모니터 ┆ {무료배송}        ┆ 무료배송 ┆      1
 여성 원피스   ┆ {할인,무료배송}   ┆ 할인     ┆      2
(6 rows)
```

예제 16-09 · 배열 검색 - ANY, @>(모두 포함), &&(하나라도 겹침)

```
SELECT name FROM products WHERE '할인' = ANY (tags);
SELECT name FROM products WHERE tags @> ARRAY['신상품', '무료배송'];
SELECT count(*) AS 겹침 FROM products WHERE tags && ARRAY['할인', '신상품'];
```

▶ 실행 결과

```
    name
-------------
 여성 원피스
(1 row)

     name
--------------
 스마트폰 S25
 울트라북 14
(2 rows)

 겹침
------
    3
(1 row)
```

예제 16-10 · unnest - 배열을 행으로 펼쳐서 집계

```
SELECT tag, count(*) AS 상품수
FROM products, unnest(tags) AS tag
GROUP BY tag
ORDER BY 상품수 DESC, tag;
```

▶ 실행 결과

```
   tag    ¦ 상품수
----------+--------
 무료배송 ¦      6
 신상품   ¦      2
 할인     ¦      1
(3 rows)
```

■ 주의 **배열·JSONB 남용 주의**
태그처럼 단순 목록은 배열이 편하지만, 원소마다 추가 정보가 붙거나(태그 생성일), 다른 테이블과 조인해야 하거나, 원소 단위로 FK 무결성이 필요하면 **별도 테이블로 정규화**하는 것이 맞습니다(11장 1NF). 배열 인덱스는 1부터 시작한다는 점도 다른 언어와 다릅니다.

전문 검색 (Full Text Search)

LIKE 는 단어 경계, 어형 변화, 관련도 순위를 모릅니다. 전문 검색은 문서를 단어(lexeme) 목록인 `tsvector` 로, 검색어를 `tsquery` 로 바꿔 `@@` 연산자로 비교합니다.

예제 16-11 · 전문 검색(Full Text Search) 기초 - tsvector, tsquery

```
SELECT to_tsvector('simple', '배송이 빠르고 품질 최고') AS tsvector;
SELECT to_tsvector('simple', '배송이 빠르고 품질 최고') @@ to_tsquery('simple', '품질') AS 일치,
       to_tsvector('simple', '배송이 빠르고 품질 최고') @@ to_tsquery('simple', '품질 & 가격') AS 둘다;
```

▶ 실행 결과

```
                tsvector
----------------------------------------
 '배송이':1 '빠르고':2 '최고':4 '품질':3
(1 row)

 일치 ¦ 둘다
------+------
 t    ¦ f
(1 row)
```

예제 16-12 · 리뷰 검색 + 순위 (영문 설정은 어간 추출도 지원)

```
CREATE TABLE articles (id INT PRIMARY KEY, body TEXT, tsv TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', body)) STORED);
INSERT INTO articles (id, body) VALUES
    (1, 'PostgreSQL indexes make queries faster'),
    (2, 'Indexing JSON documents with GIN'),
    (3, 'Window functions and running totals');
CREATE INDEX idx_articles_tsv ON articles USING GIN (tsv);
SELECT id, body, round(ts_rank(tsv, q)::numeric, 3) AS rank
FROM articles, to_tsquery('english', 'index') q
WHERE tsv @@ q
ORDER BY rank DESC;
```

▶ 실행 결과

```
CREATE TABLE
INSERT 0 3
CREATE INDEX
 id ¦                 body                  ¦ rank
----+----------------------------------------+-------
```

```
  1 ¦ PostgreSQL indexes make queries faster ¦ 0.061
  2 ¦ Indexing JSON documents with GIN       ¦ 0.061
(2 rows)
```

■ 참고 한국어 전문 검색
PostgreSQL 기본 설치에는 한국어 형태소 분석 사전이 없어, `simple` 설정은 띄어쓰기 단위로만 나눕니다('배송이' 와 '배송' 은 다른 단어). 16-12 의 영어 설정은 어간 추출(indexes, indexing → index)을 합니다. 한국어 검색 품질이 중요하면 **pg_trgm(아래)**, 외부 형태소 분석기 확장(textsearch_ko, pg_bigm 등), 또는 Elasticsearch/OpenSearch 같은 검색 엔진을 함께 씁니다.

[확장] pg_trgm - 부분 문자열 검색

pg_trgm 은 문자열을 3글자 조각(trigram)으로 나눠 유사도를 계산하고, `LIKE '%단어%'` 검색에도 GIN 인덱스를 쓸 수 있게 해 줍니다. PostgreSQL 설치에 포함된 확장이라 `CREATE EXTENSION` 만 하면 됩니다.

예제 16-13 · [확장] pg_trgm - 한글 부분 검색(LIKE '%..%')을 인덱스로

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
SELECT similarity('노트북 거치대', '노트북') AS 유사도;
CREATE INDEX idx_reviews_content_trgm ON reviews USING GIN (content gin_trgm_ops);
SET enable_seqscan = off;
EXPLAIN (COSTS OFF) SELECT * FROM reviews WHERE content LIKE '%배송%';
RESET enable_seqscan;
SELECT review_id, rating, content FROM reviews WHERE content LIKE '%배송%' ORDER BY review_id LIMIT 4;
```

▶ 실행 결과

```
CREATE EXTENSION
 유사도
--------
    0.5
(1 row)

CREATE INDEX
SET
                     QUERY PLAN
-----------------------------------------------------
 Bitmap Heap Scan on reviews
   Recheck Cond: (content ~~ '%배송%'::text)
   ->  Bitmap Index Scan on idx_reviews_content_trgm
         Index Cond: (content ~~ '%배송%'::text)
(4 rows)

RESET
 review_id ¦ rating ¦         content
-----------+--------+-------------------------
         9 ¦      5 ¦ 배송도 빠르고 품질 최고
        10 ¦      5 ¦ 배송도 빠르고 품질 최고
        11 ¦      5 ¦ 배송도 빠르고 품질 최고
        28 ¦      2 ¦ 배송이 너무 늦었어요
(4 rows)
```

■ Q&A; 자주 묻는 질문

- **Q. 그냥 MongoDB 를 쓰면 되지 않나요?** 정형 데이터(주문, 결제)와 반정형 데이터(상품 속성)를 한 DB 에서 트랜잭션·조인과 함께 쓸 수 있는 것이 PostgreSQL JSONB 의 장점입니다. 문서 위주 서비스라면 문서 DB 가 맞을 수 있습니다.
- **Q. JSON 안의 값을 표 형태로 펼치려면?** `jsonb_each_text`(16-05), `jsonb_array_elements`, `jsonb_to_recordset`, 또는 PostgreSQL 17+ 의 `JSON_TABLE` 을 씁니다.

실습 문제

번호	문제
16-Q1	attrs 에 'wireless' 키가 true 인 상품과, 브랜드별 상품 수를 구하세요
16-Q2	각 상품의 사이즈 목록(attrs.sizes)을 행으로 펼쳐 사이즈별 상품 수를 구하세요

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

정리

- 유연한 속성은 JSONB, 검색은 `@>` + GIN 인덱스, 꺼낼 때는 `->>`.
- 배열은 단순 목록에, 관계가 있으면 테이블로.
- 전문 검색은 tsvector/tsquery, 한글 부분 검색은 pg_trgm.

17장 · 사용자와 권한 - ROLE, GRANT, 행 수준 보안

학습 목표

- ROLE(사용자·그룹)을 만들고 로그인 권한과 그룹 상속을 설정할 수 있다.
- GRANT / REVOKE 로 스키마·테이블·컬럼 단위 권한을 관리할 수 있다.
- 최소 권한 원칙에 따라 읽기 전용 계정과 애플리케이션 계정을 설계할 수 있다.
- 행 수준 보안(RLS)으로 사용자마다 볼 수 있는 행을 제한할 수 있다.

예제 파일: `examples/17_security.sql` (superuser 로 실행. ROLE 은 서버 전체 객체)

ROLE

PostgreSQL 은 사용자와 그룹을 구분하지 않고 모두 **ROLE** 입니다. `LOGIN` 속성이 있으면 사용자, 없으면 권한을 묶어 두는 그룹으로 씁니다. (`CREATE USER` 는 `CREATE ROLE ... LOGIN` 의 별칭입니다.)

속성	의미
`LOGIN` / `NOLOGIN`	접속 가능 여부
`PASSWORD '...'`	비밀번호 (scram-sha-256 으로 저장)
`SUPERUSER`	모든 권한 (운영에서는 최소한으로)
`CREATEDB`, `CREATEROLE`	DB / 롤 생성 권한
`CONNECTION LIMIT n`	동시 접속 수 제한
`VALID UNTIL '날짜'`	비밀번호 만료일

예제 17-01 · ROLE 만들기 - 로그인 가능한 사용자와 그룹 역할

```
CREATE ROLE readonly NOLOGIN;                                  -- 권한 묶음(그룹)
CREATE ROLE analyst  LOGIN PASSWORD 'analyst123';              -- 실습용 비밀번호
CREATE ROLE app_user LOGIN PASSWORD 'app123' CONNECTION LIMIT 20;
GRANT readonly TO analyst;                                     -- analyst 는 readonly 의 권한을 상속
SELECT rolname, rolcanlogin, rolsuper, rolconnlimit
FROM pg_roles WHERE rolname IN ('readonly', 'analyst', 'app_user') ORDER BY rolname;
```

▶ 실행 결과

```
CREATE ROLE
CREATE ROLE
CREATE ROLE
GRANT ROLE
 rolname  ¦ rolcanlogin ¦ rolsuper ¦ rolconnlimit
----------+-------------+----------+--------------
 analyst  ¦ t           ¦ f        ¦           -1
 app_user ¦ t           ¦ f        ¦           20
 readonly ¦ f           ¦ f        ¦           -1
(3 rows)
```

예제 17-02 · 권한이 없으면 조회할 수 없다

```
SET ROLE analyst;
SELECT current_user;
SELECT count(*) FROM customers;
RESET ROLE;
```

▶ 실행 결과

```
SET
 current_user
--------------
 analyst
(1 row)

ERROR:  permission denied for table customers
RESET
```

GRANT 와 REVOKE

객체를 만든 사람(소유자)과 superuser 외에는 **명시적으로 권한을 받아야** 접근할 수 있습니다. 테이블에 접근하려면 그 테이블이 있는 **스키마의 USAGE 권한도** 필요합니다.

대상	주요 권한
DATABASE	CONNECT, CREATE, TEMP
SCHEMA	USAGE(안의 객체 접근), CREATE(객체 생성)
TABLE	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
컬럼	SELECT (컬럼들), UPDATE (컬럼들)
SEQUENCE	USAGE, SELECT, UPDATE
FUNCTION	EXECUTE

예제 17-03 · GRANT - 읽기 전용 권한 부여

```
GRANT USAGE  ON SCHEMA public TO readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
-- 앞으로 새로 만들 테이블에도 자동 적용
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO readonly;
SET ROLE analyst;
SELECT count(*) AS 고객수 FROM customers;
RESET ROLE;
```

▶ 실행 결과

```
GRANT
GRANT
ALTER DEFAULT PRIVILEGES
SET
 고객수
--------
     20
(1 row)

RESET
```

예제 17-04 · 읽기 전용 사용자는 수정할 수 없다

```
SET ROLE analyst;
UPDATE customers SET grade = 'VIP' WHERE customer_id = 2;
RESET ROLE;
```

▶ 실행 결과

```
SET
ERROR:  permission denied for table customers
RESET
```

■ TIP **ALTER DEFAULT PRIVILEGES**

`GRANT SELECT ON ALL TABLES` 는 **지금 있는 테이블**에만 적용됩니다. 앞으로 만들 테이블에도 자동으로 권한을 주려면 17-03 처럼 `ALTER DEFAULT PRIVILEGES` 를 함께 설정해야 합니다. "새로 만든 테이블이 분석 계정에서 안 보여요" 라는 문의의 단골 원인입니다.

예제 17-05 · 컬럼 단위 권한 - 앱 계정은 필요한 컬럼만

```
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT (product_id, name, price, stock) ON products TO app_user;
GRANT UPDATE (stock) ON products TO app_user;
SET ROLE app_user;
SELECT product_id, name, stock FROM products WHERE product_id = 1;
UPDATE products SET stock = stock - 1 WHERE product_id = 1;
RESET ROLE;
```

▶ 실행 결과

```
GRANT
GRANT
GRANT
SET
 product_id ¦    name     ¦ stock
------------+-------------+-------
          1 ¦ 울트라북 14 ¦    15
(1 row)

UPDATE 1
RESET
```

예제 17-06 · 허용되지 않은 컬럼 접근

```
SET ROLE app_user;
SELECT attrs FROM products WHERE product_id = 1;
UPDATE products SET price = 1 WHERE product_id = 1;
RESET ROLE;
```

▶ 실행 결과

```
SET
ERROR:  permission denied for table products
ERROR:  permission denied for table products
RESET
```

예제 17-07 · 권한 확인과 회수(REVOKE)

```
SELECT grantee, privilege_type, count(*) AS 테이블수
FROM information_schema.role_table_grants
WHERE grantee IN ('readonly', 'app_user')
GROUP BY grantee, privilege_type ORDER BY 1, 2;
SELECT grantee, column_name, privilege_type          -- 컬럼 권한은 별도 뷰에 있다
FROM information_schema.column_privileges
WHERE grantee = 'app_user' ORDER BY privilege_type, column_name;
REVOKE UPDATE (stock) ON products FROM app_user;
```

▶ 실행 결과

```
 grantee  ¦ privilege_type ¦ 테이블수
----------+----------------+----------
 readonly ¦ SELECT         ¦        8
(1 row)

 grantee  ¦ column_name ¦ privilege_type
----------+-------------+----------------
 app_user ¦ name        ¦ SELECT
 app_user ¦ price       ¦ SELECT
 app_user ¦ product_id  ¦ SELECT
 app_user ¦ stock       ¦ SELECT
 app_user ¦ stock       ¦ UPDATE
(5 rows)

REVOKE
```

■ 핵심 포인트 **최소 권한 원칙으로 계정 나누기**

실무에서 권장하는 구성입니다.

- `postgres`(superuser): DBA 만, 원격 접속 금지
- `app_owner`: 테이블 소유자, 스키마 변경(마이그레이션)에만 사용
- `app_user`: 애플리케이션 접속용, 필요한 테이블의 SELECT/INSERT/UPDATE/DELETE 만
- `readonly` 그룹 → `analyst` 등 개인 계정: 분석·조회용

애플리케이션이 superuser 로 접속하면 SQL 인젝션(19장) 한 번에 DB 전체가 위험해집니다.

행 수준 보안 (Row Level Security)

RLS 는 테이블에 **정책(POLICY)** 을 걸어 사용자마다 보이는 행, 수정할 수 있는 행을 제한합니다. 멀티테넌트 SaaS(회사별 데이터 분리), 영업사원별 고객 관리 등에 씁니다.

```
ALTER TABLE 테이블 ENABLE ROW LEVEL SECURITY;
CREATE POLICY 정책명 ON 테이블
    FOR SELECT ¦ INSERT ¦ UPDATE ¦ DELETE ¦ ALL
    TO 롤                  -- 생략 시 모든 롤
    USING (보이는 행 조건)
    WITH CHECK (쓰는 행 조건);
```

예제 17-08 · 행 수준 보안(RLS) - 영업사원은 자기 담당 주문만 보기

```
CREATE ROLE sales_kang LOGIN PASSWORD 'kang123';            -- 직원 6번 강민수
CREATE ROLE sales_cho  LOGIN PASSWORD 'cho123';             -- 직원 7번 조은지
CREATE TABLE sales_reps (                                   -- DB 계정 ↔ 직원 번호 매핑
    role_name NAME PRIMARY KEY,
    emp_id    INTEGER NOT NULL REFERENCES employees
);
INSERT INTO sales_reps VALUES ('sales_kang', 6), ('sales_cho', 7);
GRANT USAGE ON SCHEMA public TO sales_kang, sales_cho;
GRANT SELECT ON orders, sales_reps TO sales_kang, sales_cho;

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY orders_by_owner ON orders
    FOR SELECT
    USING (emp_id = (SELECT emp_id FROM sales_reps WHERE role_name = current_user));

SET ROLE sales_kang;
SELECT current_user, count(*) AS 보이는주문, min(emp_id), max(emp_id) FROM orders;
RESET ROLE;
SET ROLE sales_cho;
SELECT current_user, count(*) AS 보이는주문, min(emp_id), max(emp_id) FROM orders;
RESET ROLE;
SELECT current_user, count(*) AS 보이는주문 FROM orders;    -- 테이블 소유자/superuser 는 전체
```

▶ 실행 결과

```
CREATE ROLE
CREATE ROLE
CREATE TABLE
INSERT 0 2
GRANT
GRANT
ALTER TABLE
CREATE POLICY
SET
 current_user ¦ 보이는주문 ¦ min ¦ max
--------------+------------+-----+-----
 sales_kang   ¦         20 ¦   6 ¦   6
(1 row)

RESET
SET
 current_user ¦ 보이는주문 ¦ min ¦ max
--------------+------------+-----+-----
 sales_cho    ¦         10 ¦   7 ¦   7
(1 row)

RESET
 current_user ¦ 보이는주문
--------------+------------
 postgres     ¦        120
(1 row)
```

예제 17-09 · 정리 - RLS 해제와 역할 삭제

```
ALTER TABLE orders DISABLE ROW LEVEL SECURITY;
DROP POLICY orders_by_owner ON orders;
DROP OWNED BY sales_kang, sales_cho;      -- 부여된 권한 제거
DROP ROLE sales_kang, sales_cho;
SELECT count(*) AS 남은sales역할 FROM pg_roles WHERE rolname LIKE 'sales_%';
```

▶ 실행 결과

```
ALTER TABLE
DROP POLICY
DROP OWNED
DROP ROLE
 남은sales역할
---------------
             0
(1 row)
```

■ 참고 **RLS 의 예외**

테이블 소유자와 superuser, `BYPASSRLS` 속성이 있는 롤에는 정책이 적용되지 않습니다(17-08 의 postgres 는 120건 전체). 소유자에게도 적용하려면 `ALTER TABLE ... FORCE ROW LEVEL SECURITY` 를 씁니다. 애플리케이션이 하나의 계정으로 접속하는 구조라면 `current_user` 대신 `SET app.user_id = '6'` 같은 설정값과 `current_setting('app.user_id')` 를 정책에 사용합니다.

접속 제어 - pg_hba.conf

누가 어디에서 어떤 방식으로 접속할 수 있는지는 데이터 디렉터리의 `pg_hba.conf` 파일이 정합니다(위에서부터 처음 일치하는 줄 적용). 파일 위치는 `SHOW hba_file;` 로 확인합니다. Docker 컨테이너에서는 `docker exec -it pg-shop bash` 로 들어가 편집합니다. 수정 후 `SELECT pg_reload_conf();` 로 반영합니다.

```
# TYPE  DATABASE  USER       ADDRESS          METHOD
local   all       postgres                    scram-sha-256
host    shop      app_user   10.0.1.0/24      scram-sha-256
host    shop      analyst    192.168.0.0/16   scram-sha-256
host    all       all        0.0.0.0/0        reject
```

■ 주의 **보안 체크리스트**

- `trust` 인증은 로컬 실습 외에는 쓰지 않는다.
- `listen_addresses` 와 방화벽으로 DB 포트(5432)를 인터넷에 직접 노출하지 않는다.
- 비밀번호를 코드·깃 저장소에 넣지 않는다(19장의 환경변수 방식).
- 민감 정보(비밀번호)는 해시로 저장한다(18장 pgcrypto `crypt`).

■ Q&A; **자주 묻는 질문**

- **Q. 사용자를 지우려는데 오류가 나요.** 그 롤이 소유한 객체나 받은 권한이 남아 있으면 지울 수 없습니다. 17-09 처럼 `REASSIGN OWNED BY 롤 TO 다른롤; DROP OWNED BY 롤;` 후 `DROP ROLE` 합니다. 권한은 DB 마다 따로 있으므로 각 DB 에서 실행해야 합니다.

정리

- ROLE = 사용자 + 그룹. 그룹 롤에 권한을 주고 사용자를 그룹에 넣는다.
- 테이블 권한 + 스키마 USAGE, 앞으로의 객체는 DEFAULT PRIVILEGES.
- 행 단위 제한은 RLS, 접속 제한은 pg_hba.conf.

18장 · 운영 기초 - 모니터링, VACUUM, 백업과 복구

학습 목표

- 데이터베이스·테이블 크기, 접속 세션, 서버 설정을 조회할 수 있다.
- dead tuple 과 VACUUM / autovacuum / ANALYZE 의 역할을 설명할 수 있다.
- 확장(Extension)을 설치하고 pg_stat_statements 로 느린 쿼리를 찾을 수 있다.
- pg_dump / pg_restore 로 백업·복구하고 `\copy` 로 CSV 를 주고받을 수 있다.

예제 파일: `examples/18_admin.sql`

모니터링 기본 쿼리

예제 18-01 · 데이터베이스와 테이블 크기

```
SELECT pg_size_pretty(pg_database_size(current_database())) AS db크기;
SELECT relname AS 테이블,
       pg_size_pretty(pg_total_relation_size(relid)) AS 전체크기,
       pg_size_pretty(pg_relation_size(relid))       AS 데이터,
       pg_size_pretty(pg_indexes_size(relid))        AS 인덱스
FROM pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 5;
```

▶ 실행 결과

```
 db크기
---------
 8702 kB
(1 row)

   테이블    ¦ 전체크기 ¦   데이터   ¦ 인덱스
-------------+----------+------------+--------
 orders      ¦ 56 kB    ¦ 16 kB      ¦ 16 kB
 order_items ¦ 56 kB    ¦ 16 kB      ¦ 16 kB
 customers   ¦ 40 kB    ¦ 8192 bytes ¦ 32 kB
 departments ¦ 40 kB    ¦ 8192 bytes ¦ 32 kB
 employees   ¦ 40 kB    ¦ 8192 bytes ¦ 32 kB
(5 rows)
```

예제 18-02 · 현재 접속 중인 세션 보기 - pg_stat_activity

```
SELECT usename, datname, state, backend_type,
       left(query, 40) AS query
FROM pg_stat_activity
WHERE datname = current_database();
```

▶ 실행 결과

```
 usename  ¦ datname ¦ state  ¦  backend_type  ¦                  query
----------+---------+--------+-----------------+-----------------------------------------
 postgres ¦ shop    ¦ active ¦ client backend ¦ SELECT usename, datname, state, backend_
(1 row)
```

예제 18-03 · 문제 세션 정리 - 쿼리 취소와 강제 종료 (사용법)

```
-- SELECT pg_cancel_backend(12345);     -- 실행 중인 쿼리만 취소
-- SELECT pg_terminate_backend(12345);  -- 접속 자체를 끊음
-- 5분 이상 'idle in transaction' 인 세션 찾기
SELECT pid, usename, state, now() - state_change AS 지속시간
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND now() - state_change > interval '5 minutes';
```

▶ 실행 결과

```
 pid ¦ usename ¦ state ¦ 지속시간
-----+---------+-------+----------
(0 rows)
```

시스템 뷰 / 함수	내용
pg_stat_activity	현재 세션, 실행 중인 쿼리, 상태, 대기 이벤트
pg_stat_user_tables	테이블별 읽기/쓰기 횟수, live/dead 행 수, 마지막 VACUUM
pg_stat_user_indexes	인덱스 사용 횟수 (13장)
pg_locks	잠금 (14장)
pg_size_pretty(pg_total_relation_size('t'))	테이블 + 인덱스 크기

■ TIP 세션 상태

active(실행 중), idle(대기), idle in transaction(트랜잭션을 연 채 아무것도 안 함). 마지막 상태가 오래 지속되면 잠금을 붙잡고 VACUUM 을 방해하므로 원인 프로그램을 찾아 고쳐야 합니다. idle_in_transaction_session_timeout 설정으로 자동 종료할 수도 있습니다.

서버 설정

설정은 postgresql.conf(데이터 디렉터리, 위치는 SHOW config_file;), ALTER SYSTEM, 또는 Docker 의 command: -c 이름=값(실습용 docker-compose.yml 참고)으로 바꾸고, 범위에 따라 세션·DB·롤 단위로도 덮어쓸 수 있습니다. pg_settings.context 가 postmaster 인 항목(shared_buffers 등)은 **서버 재시작**이 필요합니다(Docker 는 docker compose restart).

예제 18-04 · 서버 설정 확인 - SHOW 와 pg_settings

```
SHOW max_connections;
SELECT name, setting, unit, context
FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'maintenance_work_mem',
               'effective_cache_size', 'autovacuum', 'timezone')
ORDER BY name;
```

▶ 실행 결과

```
 max_connections
-----------------
 100
(1 row)

         name         ¦ setting ¦ unit ¦  context
----------------------+---------+------+------------
 autovacuum           ¦ on      ¦      ¦ sighup
 effective_cache_size ¦ 524288  ¦ 8kB  ¦ user
 maintenance_work_mem ¦ 65536   ¦ kB   ¦ user
 shared_buffers       ¦ 16384   ¦ 8kB  ¦ postmaster
 work_mem             ¦ 4096    ¦ kB   ¦ user
```

```
(5 rows)
```

예제 18-05 · 설정 바꾸기 - 세션 / 데이터베이스 / 서버 단위

```
SET work_mem = '64MB';                          -- 현재 세션만
SHOW work_mem;
RESET work_mem;
ALTER DATABASE shop SET statement_timeout = '60s';   -- 이 DB 에 새로 접속하는 세션부터
ALTER DATABASE shop RESET statement_timeout;
-- ALTER SYSTEM SET work_mem = '16MB';  SELECT pg_reload_conf();   -- 서버 전체 (postgresql.auto.conf)
```

▶ 실행 결과

```
SET
 work_mem
----------
 64MB
(1 row)

RESET
ALTER DATABASE
ALTER DATABASE
```

설정	의미	출발점 (전용 DB 서버 기준)
shared_buffers	PostgreSQL 전용 캐시	메모리의 25%
effective_cache_size	OS 캐시 포함 예상 캐시 (계획용)	메모리의 50~75%
work_mem	정렬·해시 작업당 메모리	4~64MB (동시 작업 수 고려)
maintenance_work_mem	VACUUM, 인덱스 생성용	256MB~1GB
max_connections	최대 접속 수	기본 100, 많으면 커넥션 풀(PgBouncer)

VACUUM 과 ANALYZE

14장 MVCC 에서 UPDATE/DELETE 는 옛 버전(dead tuple)을 남긴다고 했습니다. VACUUM 은 이 공간을 재사용 가능하게 정리하고, ANALYZE 는 옵티마이저가 쓰는 통계를 갱신합니다. 평소에는 **autovacuum** 이 자동으로 실행합니다.

예제 18-06 · MVCC 의 부산물 - 죽은 튜플(dead tuple)과 VACUUM

```
CREATE TABLE vac_test AS SELECT g AS id, 'x' AS v FROM generate_series(1, 100000) g;
UPDATE vac_test SET v = 'y';                    -- 10만 행 모두 새 버전 생성
SELECT pg_stat_force_next_flush();              -- 통계를 즉시 반영 (실습용)
SELECT relname, n_live_tup, n_dead_tup FROM pg_stat_user_tables WHERE relname = 'vac_test';
VACUUM vac_test;
SELECT pg_stat_force_next_flush();
SELECT relname, n_live_tup, n_dead_tup, last_vacuum IS NOT NULL AS vacuum됨
FROM pg_stat_user_tables WHERE relname = 'vac_test';
```

▶ 실행 결과

```
SELECT 100000
UPDATE 100000
 pg_stat_force_next_flush
--------------------------

(1 row)

 relname  ¦ n_live_tup ¦ n_dead_tup
----------+------------+------------
 vac_test ¦     100000 ¦     100000
(1 row)
```

```
VACUUM
 pg_stat_force_next_flush
--------------------------

(1 row)

 relname  ¦ n_live_tup ¦ n_dead_tup ¦ vacuum됨
----------+------------+------------+----------
 vac_test ¦     100000 ¦          0 ¦ t
(1 row)
```

예제 18-07 · ANALYZE 와 통계 정보 - 옵티마이저가 보는 값

```
ANALYZE orders;
SELECT attname, n_distinct, most_common_vals, most_common_freqs
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'status';
```

▶ 실행 결과

```
ANALYZE
 attname ¦ n_distinct ¦          most_common_vals          ¦          most_common_freqs
---------+------------+------------------------------------+------------------------------------
 status  ¦          4 ¦ {DELIVERED,CANCELLED,PAID,SHIPPED} ¦ {0.8333333,0.125,0.025,0.016666668}
(1 row)
```

명령	하는 일	잠금
VACUUM	dead tuple 정리, 공간 재사용 표시	읽기·쓰기 허용
VACUUM ANALYZE	정리 + 통계 갱신	읽기·쓰기 허용
VACUUM FULL	테이블을 새로 써서 디스크 크기 축소	**테이블 전체 잠금** (운영 시간 피함)
ANALYZE	통계만 갱신 (대량 입력 직후 권장)	읽기·쓰기 허용

■ 핵심 포인트 **VACUUM 을 비유로 이해하기**
"식당에서 손님이 나간 테이블(dead tuple)을 치우는 직원이 autovacuum" 이라고 생각하면 쉽습니다. 치우지 않으면 새 손님을 앉힐 자리가 없어 가게(테이블 파일)를 계속 넓혀야 하고(bloat), 오래 앉아 있는 손님(긴 트랜잭션)이 있으면 그 주변은 치울 수 없습니다.

확장 (Extension)

확장	기능
pgcrypto	해시, 암호화 (crypt, gen_salt, digest)
pg_trgm	유사도, LIKE '%...%' 인덱스 (16장)
pg_stat_statements	쿼리별 실행 횟수·시간 통계
uuid-ossp	UUID 생성 (13+ 는 gen_random_uuid() 기본 내장)
postgis	지리 정보 (별도 설치)
pgvector	벡터 유사도 검색 (AI 임베딩, 별도 설치)

예제 18-08 · 확장(Extension) - pgcrypto 로 비밀번호 해시, UUID 생성

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
SELECT gen_random_uuid() IS NOT NULL AS uuid생성,              -- 13 버전부터 기본 내장
       length(crypt('my-password', gen_salt('bf'))) AS 해시길이,
       crypt('my-password', '$2a$06$abcdefghijklmnopqrstuu')
           = crypt('my-password', '$2a$06$abcdefghijklmnopqrstuu') AS 같은솔트같은해시;
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

▶ 실행 결과

```
CREATE EXTENSION
 uuid생성 ¦ 해시길이 ¦ 같은솔트같은해시
----------+----------+------------------
 t        ¦       60 ¦ t
(1 row)

 extname  ¦ extversion
----------+------------
 pgcrypto ¦ 1.4
 plpgsql  ¦ 1.0
(2 rows)
```

pg_stat_statements 는 서버 시작 시 로드되어야 하므로 원래는 `postgresql.conf` 에 `shared_preload_libraries = 'pg_stat_statements'` 를 추가하고 재시작해야 합니다. 실습용 `docker-compose.yml` 에는 이 설정이 이미 들어 있어 바로 `CREATE EXTENSION` 하면 됩니다. 아래 결과는 그때까지 실행한 쿼리 통계라 환경마다 다릅니다.

예제 18-09 · [설정 필요] pg_stat_statements - 느린/자주 실행되는 쿼리 찾기

```
-- postgresql.conf 에 shared_preload_libraries = 'pg_stat_statements' 추가 후 서버 재시작 필요
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT calls, round(mean_exec_time::numeric, 2) AS 평균ms, left(query, 50) AS query
FROM pg_stat_statements
WHERE query ILIKE 'select%orders%'
ORDER BY total_exec_time DESC
LIMIT 3;
```

▶ 실행 결과

```
CREATE EXTENSION
 calls ¦ 평균ms ¦                       query
-------+--------+--------------------------------------------------
     1 ¦  68.35 ¦ SELECT count(*), pg_size_pretty(pg_total_relation_
     1 ¦  66.83 ¦ SELECT count(*), pg_size_pretty(pg_total_relation_
     1 ¦  66.68 ¦ SELECT count(*), pg_size_pretty(pg_total_relation_
(3 rows)
```

백업과 복구

방식	도구	특징
논리 백업	`pg_dump` / `pg_dumpall`	SQL 또는 아카이브 파일. DB·테이블 단위, 버전 간 이전 가능
물리 백업	`pg_basebackup`	데이터 디렉터리 전체 복사. 대용량에 빠름
시점 복구(PITR)	물리 백업 + WAL 보관	특정 시각으로 되돌리기 (실수로 DELETE 하기 직전 등)

실습 서버가 Docker 컨테이너이므로 `pg_dump` 등은 **컨테이너 안에서** 실행합니다(`docker exec`). 백업 파일은 컨테이너의 `/work` 폴더에 만들면 PC 의 저장소 폴더 `work/` 에 그대로 보입니다(`docker-compose.yml` 에서 연결해 둠). 아래는 명령 프롬프트/터미널에서 실행합니다.

```
:: 1) 사용자 지정 형식(-F c, 압축)으로 백업 - 가장 권장  → PC 의 work/shop.dump
docker exec pg-shop pg_dump -U postgres -d shop -F c -f /work/shop.dump
```

```
:: 2) 새 DB 를 만들고 복구
docker exec pg-shop createdb -U postgres shop_restore
docker exec pg-shop pg_restore -U postgres -d shop_restore /work/shop.dump

:: 3) 일반 SQL 파일로 백업 (내용을 눈으로 확인 가능)
docker exec pg-shop pg_dump -U postgres -d shop -f /work/shop.sql

:: 4) 특정 테이블 구조만 / 백업 파일 목록 보기 / 롤 등 전역 객체 백업
docker exec pg-shop pg_dump -U postgres -d shop --schema-only -t customers
docker exec pg-shop pg_restore -l /work/shop.dump
docker exec pg-shop pg_dumpall -U postgres --globals-only -f /work/globals.sql
```

■ 참고 **DBeaver 의 Backup / Restore 메뉴**
DBeaver 의 데이터베이스 우클릭 → **Tools → Backup / Restore** 는 PC 에 설치된 `pg_dump` / `pg_restore` 를 실행하는 화면이라, PostgreSQL 을 Docker 로만 실행하는 이 실습 환경에서는 PC 에 PostgreSQL 클라이언트를 따로 설치해야 쓸 수 있습니다. 이 과정에서는 위의 `docker exec` 명령을 사용합니다.

■ 주의 **백업은 복구해 봐야 백업이다**
백업 파일이 있어도 복구 절차를 연습하지 않으면 장애 때 쓸 수 없습니다. 정기적으로 다른 서버에 복구해 보고, 백업 파일은 DB 서버와 다른 곳에 보관해야 합니다. 클라우드 관리형 DB(RDS 등)는 자동 백업과 PITR 을 제공합니다.

데이터 가져오기 / 내보내기

DBeaver (기본 방법)

- 내보내기: 테이블 우클릭(또는 결과 그리드 우클릭) → **Export Data** → CSV / Excel(XLSX) / JSON / SQL INSERT 등 형식 선택 → 인코딩 `UTF-8`, 헤더 포함 여부, 저장 폴더 지정 → Proceed.
- 가져오기: 대상 테이블 우클릭 → **Import Data** → CSV 선택 → 파일 지정 → 열 매핑(Columns mapping) 확인 → Proceed. 날짜 형식과 NULL 표시 문자열을 확인합니다.

psql / SQL

명령	파일 위치	권한
`COPY 테이블 TO/FROM '경로'`	**DB 서버**의 파일	superuser 또는 pg_read/write_server_files
`\copy 테이블 TO/FROM '경로'`	**psql 을 실행한 PC** 의 파일	일반 사용자 가능

18-10 은 psql 메타 명령이라 DBeaver 에서는 실행되지 않습니다. 명령 프롬프트에서 `docker exec -it -w /work pg-shop psql -U postgres -d shop` 으로 컨테이너의 psql 에 접속해 실행하면 CSV 파일이 PC 의 `work/` 폴더에 만들어집니다. 또는 위의 DBeaver 마법사로 같은 작업을 합니다.

예제 18-10 · [psql 전용] 데이터 내보내기/가져오기 - \copy (CSV)

```
-- DBeaver: 결과 그리드/테이블 우클릭 → Export Data / Import Data 마법사 (교안 참고)
\copy (SELECT customer_id, name, city, grade FROM customers ORDER BY customer_id) TO 'customers_export.csv' WITH (FORMAT csv, HEADER)
CREATE TABLE customers_import (customer_id INT, name TEXT, city TEXT, grade TEXT);
\copy customers_import FROM 'customers_export.csv' WITH (FORMAT csv, HEADER)
SELECT count(*) AS 가져온행수, min(name), max(name) FROM customers_import;
```

▶ 실행 결과

```
COPY 20
CREATE TABLE
COPY 20
 가져온행수 ¦  min   ¦  max
------------+--------+--------
         20 ¦ 강시우 ¦ 황민재
(1 row)
```

> ■ **TIP 엑셀 CSV 를 가져올 때**
> 엑셀에서 "CSV UTF-8" 형식으로 저장해야 한글이 깨지지 않습니다. 날짜 형식, 천 단위 쉼표, 빈 칸(NULL)을 미리 정리하고, 먼저 모든 컬럼이 TEXT 인 임시 테이블로 가져온 뒤 SQL 로 정제해 넣는 방법이 안전합니다.

정리

- 모니터링은 pg_stat_activity, pg_stat_user_tables, pg_stat_statements 부터.
- dead tuple 은 autovacuum 이 정리, 대량 변경 후에는 ANALYZE.
- 백업은 `pg_dump -F c`, 복구는 `pg_restore`. 복구 연습까지가 백업이다.

19장 · Python 으로 PostgreSQL 다루기

학습 목표

- psycopg 3 로 접속하고 쿼리 결과를 받아 올 수 있다.
- 파라미터 바인딩으로 SQL 인젝션을 막을 수 있다.
- 애플리케이션 코드에서 트랜잭션(commit, rollback, `with conn.transaction()`)을 다룰 수 있다.
- 조회 결과를 pandas DataFrame 으로 분석하고 차트로 저장할 수 있다.

예제 폴더: `python/` (`db.py` 에 접속 정보, 01~05 예제)

환경 준비

```
cd postgresql-tutorial/python
python -m venv .venv
.venv\Scripts\activate          (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
set DATABASE_URL=postgresql://postgres:postgres@localhost:5432/shop
python 01_connect.py
```

라이브러리	설명
psycopg 3 (`psycopg`)	현재 권장 PostgreSQL 드라이버. `psycopg[binary]` 로 설치하면 별도 빌드 불필요
psycopg2	이전 세대 드라이버, 기존 코드에서 많이 보임 (사용법 거의 동일)
SQLAlchemy	ORM / SQL 빌더, 내부적으로 psycopg 사용
asyncpg	비동기 전용 고성능 드라이버

접속 문자열(URI) 형식: `postgresql://사용자:비밀번호@호스트:포트/데이터베이스`. 예제는 `db.py` 에서 환경변수 `DATABASE_URL` 을 읽어 **비밀번호를 코드에 쓰지 않습니다.** 환경변수가 없으면 실습용 Docker 서버의 기본값(`postgres:postgres@localhost:5432/shop`)으로 접속합니다.

접속과 조회

python/01_connect.py · 파이썬 실행 예제

```
"""19-01 접속하고 첫 쿼리 실행하기"""
from db import connect

# with 블록을 벗어나면 커넥션이 자동으로 닫힌다
with connect() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT current_database(), current_user, version()")
        db, user, version = cur.fetchone()
        print("DB      :", db)
        print("USER    :", user)
        print("VERSION :", version.split(",")[0])

        cur.execute("SELECT count(*) FROM customers")
        print("고객 수 :", cur.fetchone()[0])
```

▶ 실행 결과

```
DB      : shop
USER    : postgres
```

```
VERSION : PostgreSQL 18.6 on x86_64-windows
고객 수 : 20
```

메서드	설명
`cur.execute(sql, params)`	실행
`fetchone()` / `fetchall()` / `fetchmany(n)`	한 행 / 전부 / n행 (기본은 튜플)
`conn.execute(...)`	커서를 만들지 않고 바로 실행 (psycopg 3)
`row_factory=dict_row`	행을 딕셔너리로 받기

파라미터 바인딩과 SQL 인젝션

python/02_query_params.py · 파이썬 실행 예제

```
"""19-02 조회 결과 다루기와 파라미터 바인딩 (SQL 인젝션 방지)"""
from psycopg.rows import dict_row

from db import connect

with connect(row_factory=dict_row) as conn:      # 행을 dict 로 받기
    # 1) 여러 행 조회
    rows = conn.execute(
        "SELECT name, price FROM products WHERE category_id = %s ORDER BY price DESC",
        (6,),                                       # 값은 반드시 파라미터로 전달
    ).fetchall()
    print("[주변기기]")
    for r in rows:
        print(f"  {r['name']} : {r['price']:,}원")

    # 2) 이름 있는 파라미터
    row = conn.execute(
        "SELECT name, grade FROM customers WHERE city = %(city)s AND grade = %(grade)s",
        {"city": "서울", "grade": "VIP"},
    ).fetchone()
    print("\n[서울 VIP]", row)

    # 3) SQL 인젝션 - 사용자가 입력한 값이 악의적인 경우
    user_input = "' OR '1'='1"

    bad_sql = f"SELECT count(*) AS n FROM customers WHERE email = '{user_input}'"
    print("\n[나쁜 예] 문자열로 SQL 조립:", bad_sql)
    print("   → 조회된 고객 수:", conn.execute(bad_sql).fetchone()["n"], "(전체 고객이 노출!)")

    good = conn.execute("SELECT count(*) AS n FROM customers WHERE email = %s", (user_input,))
    print("[좋은 예] 파라미터 바인딩 → 조회된 고객 수:", good.fetchone()["n"])
```

▶ 실행 결과

```
[주변기기]
  27인치 모니터 : 329,000원
  기계식 키보드 : 89,000원
  USB-C 허브 : 45,000원
  노트북 거치대 : 35,000원
  무선 마우스 : 29,000원

[서울 VIP] {'name': '김민준', 'grade': 'VIP'}

[나쁜 예] 문자열로 SQL 조립: SELECT count(*) AS n FROM customers WHERE email = '' OR '1'='1'
   → 조회된 고객 수: 20 (전체 고객이 노출!)
[좋은 예] 파라미터 바인딩 → 조회된 고객 수: 0
```

> ■ 주의 **f-string 으로 SQL 을 만들지 않는다**
> 사용자 입력을 문자열로 이어 붙이면 입력값이 SQL 문법의 일부가 되어 `' OR '1'='1` 같은 값으로 조건을 무력화할 수 있습니다(위 결과에서 고객 20명 전체 노출). **값은 항상 `%s` 자리표시자로 넘기면** 드라이버가 안전하게 처리합니다. 단, 테이블·컬럼 이름은 파라미터가 될 수 없으므로 `psycopg.sql.Identifier` 를 씁니다. 자리표시자는 SQL 의 `%s` 이지 파이썬의 `%` 서식이 아닙니다.

INSERT 와 트랜잭션

psycopg 는 첫 쿼리 때 자동으로 트랜잭션을 시작하므로 `commit()` **을 호출해야 반영**됩니다. `with connect() as conn:` 블록은 정상 종료 시 commit, 예외 시 rollback 합니다. `with conn.transaction():` 블록은 그 안의 작업만 하나의 트랜잭션(또는 SAVEPOINT)으로 묶습니다.

python/03_transaction.py · 파이썬 실행 예제

```
"""19-03 INSERT, 여러 건 입력, 트랜잭션 (commit / rollback)"""
import psycopg

from db import connect

with connect() as conn:
    conn.execute("DROP TABLE IF EXISTS py_memo")
    conn.execute("""
        CREATE TABLE py_memo (
            id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            content TEXT NOT NULL CHECK (length(content) <= 20)
        )""")
    conn.commit()            # psycopg 는 기본적으로 트랜잭션을 열어 둔다 → commit 필요!

    # 1) 한 건 넣고 생성된 id 받기
    new_id = conn.execute("INSERT INTO py_memo (content) VALUES (%s) RETURNING id",
                          ("첫 번째 메모",)).fetchone()[0]
    conn.commit()
    print("새 메모 id:", new_id)

    # 2) 여러 건 한 번에
    with conn.cursor() as cur:
        cur.executemany("INSERT INTO py_memo (content) VALUES (%s)",
                        [("둘",), ("셋",), ("넷",)])
    conn.commit()

    # 3) 트랜잭션 블록 - 안에서 예외가 나면 블록 전체가 롤백된다
    try:
        with conn.transaction():
            conn.execute("INSERT INTO py_memo (content) VALUES ('정상 메모')")
            conn.execute("INSERT INTO py_memo (content) VALUES (%s)", ("너무 긴 메모" * 10,))
    except psycopg.errors.CheckViolation as e:
        print("오류 발생 → 롤백:", e.diag.message_primary)

    rows = conn.execute("SELECT id, content FROM py_memo ORDER BY id").fetchall()
    print("최종 메모:", rows)
    conn.execute("DROP TABLE py_memo")
    conn.commit()
```

▶ 실행 결과

```
새 메모 id: 1
오류 발생 → 롤백: new row for relation "py_memo" violates check constraint "py_memo_content_check"
최종 메모: [(1, '첫 번째 메모'), (2, '둘'), (3, '셋'), (4, '넷')]
```

■ 핵심 포인트 자주 하는 실수

- INSERT 후 `commit()` 을 빼먹어 "DB 에 데이터가 안 들어가요" → 다른 도구에서 조회하면 보이지 않음.
- 예외를 잡고 계속 쿼리를 실행 → 14장의 `current transaction is aborted` 오류. `rollback()` 을 먼저 해야 함.
- 반복문 안에서 매번 connect → 접속 비용이 큼. 연결은 재사용하고, 웹 서비스는 커넥션 풀(`psycopg_pool`)을 사용.

pandas 로 분석하기

python/04_pandas_report.py · 파이썬 실행 예제

```
"""19-04 조회 결과를 pandas DataFrame 으로 받아 분석하고 차트 저장"""
import pandas as pd

from db import connect

SQL = """
SELECT to_char(date_trunc('month', o.order_date), 'YYYY-MM') AS month,
       c.name AS category,
       sum(oi.quantity * oi.unit_price)::bigint AS sales
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p     ON p.product_id = oi.product_id
JOIN categories c   ON c.category_id = p.category_id
WHERE o.status <> 'CANCELLED' AND o.order_date >= %s
GROUP BY 1, 2
ORDER BY 1, 2
"""

with connect() as conn, conn.cursor() as cur:
    cur.execute(SQL, ("2026-01-01",))
    df = pd.DataFrame(cur.fetchall(), columns=[d.name for d in cur.description])

print(df.head(), "\n")

# 월 × 분류 피벗 (만원 단위)
pivot = df.pivot_table(index="month", columns="category", values="sales",
                       aggfunc="sum", fill_value=0) // 10000
print(pivot[["노트북", "스마트폰", "주변기기"]])

try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib import font_manager
    installed = {f.name for f in font_manager.fontManager.ttflist}
    for font in ["Malgun Gothic", "AppleGothic", "NanumGothic"]:   # 한글 폰트 (Win/Mac/Linux)
        if font in installed:
            plt.rcParams["font.family"] = font
            break
    ax = pivot.sum(axis=1).plot(kind="bar", title="2026년 월별 매출(만원)", figsize=(7, 3.5))
    ax.set_xlabel("")
    plt.tight_layout()
    plt.savefig("monthly_sales.png", dpi=120)
    print("\n차트 저장: monthly_sales.png")
except ImportError:
    print("\n(matplotlib 이 없어 차트는 건너뜀)")
```

▶ 실행 결과

```
     month category    sales
0  2026-01       간식   125000
1  2026-01  게이밍 노트북  2490000
2  2026-01     남성의류    70200
3  2026-01     스마트폰  5451000
4  2026-01     여성의류   202100
```

```
category  노트북  스마트폰  주변기기
month
2026-01     0   545    29
2026-02    99     0    69
2026-03   645     0     0
2026-04   297   345     2
2026-05     0   207     0
2026-06   258   575   107

차트 저장: monthly_sales.png
```

SQL 로 필요한 만큼 집계해서 가져오고, pandas 로는 피벗·시각화·통계 분석을 하는 분업이 효율적입니다. 수백만 행을 통째로 가져와 pandas 로 GROUP BY 하는 것보다 DB 에서 집계하는 것이 훨씬 빠릅니다.

동시성 데모 (14장 보충)

psql 창 두 개로 하던 격리 수준·갱신 손실·데드락 실험을 커넥션 두 개로 재현합니다. 수업에서 실행해 결과를 함께 해석하세요.

python/05_concurrency_demo.py · 파이썬 실행 예제

```python
"""19-05 (14장 보충) 두 세션이 동시에 일할 때 - 격리 수준, 갱신 손실, 데드락

psql 창 두 개로 하던 실험을 커넥션 두 개(A, B)로 재현한다.
"""
import threading
import time

import psycopg

from db import connect


def setup():
    with connect() as c:
        c.execute("DROP TABLE IF EXISTS demo_acc")
        c.execute("CREATE TABLE demo_acc (id INT PRIMARY KEY, balance INT NOT NULL)")
        c.execute("INSERT INTO demo_acc VALUES (1, 1000), (2, 1000)")


def balance(conn, id_=1):
    return conn.execute("SELECT balance FROM demo_acc WHERE id = %s", (id_,)).fetchone()[0]


print("① READ COMMITTED - 다른 세션이 커밋한 값이 바로 보인다")
setup()
with connect() as a, connect(autocommit=True) as b:
    print("   A 첫 조회 :", balance(a))
    b.execute("UPDATE demo_acc SET balance = 500 WHERE id = 1")      # B 가 수정 후 자동 커밋
    print("   A 재조회 :", balance(a), "(같은 트랜잭션인데 값이 바뀜 = Non-repeatable read)")

print("\n② REPEATABLE READ - 트랜잭션 시작 시점의 스냅샷을 계속 본다")
setup()
with connect() as a, connect(autocommit=True) as b:
    a.isolation_level = psycopg.IsolationLevel.REPEATABLE_READ
    print("   A 첫 조회 :", balance(a))
    b.execute("UPDATE demo_acc SET balance = 500 WHERE id = 1")
    print("   A 재조회 :", balance(a), "(B 의 변경이 보이지 않음)")
    try:
        a.execute("UPDATE demo_acc SET balance = balance - 100 WHERE id = 1")
    except psycopg.errors.SerializationFailure as e:
        print("   A 가 같은 행 수정 시도 →", e.diag.message_primary)
```

```
print("\n③ 갱신 손실(Lost Update) - 읽고 → 앱에서 계산 → 쓰기")
setup()
with connect() as a, connect() as b:
    va = balance(a)                    # A: 1000 읽음
    vb = balance(b)                    # B: 1000 읽음
    a.execute("UPDATE demo_acc SET balance = %s WHERE id = 1", (va - 300,)); a.commit()
    b.execute("UPDATE demo_acc SET balance = %s WHERE id = 1", (vb - 200,)); b.commit()
    print("   기대값 500, 실제값", balance(a), "→ A 의 출금 300 이 사라짐!")

print("\n④ 해결 - SELECT ... FOR UPDATE 로 먼저 잠그기")
setup()
with connect() as a, connect() as b:
    va = a.execute("SELECT balance FROM demo_acc WHERE id = 1 FOR UPDATE").fetchone()[0]

    def session_b():
        vb = b.execute("SELECT balance FROM demo_acc WHERE id = 1 FOR UPDATE").fetchone()[0]
        b.execute("UPDATE demo_acc SET balance = %s WHERE id = 1", (vb - 200,))
        b.commit()

    t = threading.Thread(target=session_b)
    t.start()
    time.sleep(0.5)                    # B 는 A 가 커밋할 때까지 대기한다
    print("   B 대기 중? :", t.is_alive())
    a.execute("UPDATE demo_acc SET balance = %s WHERE id = 1", (va - 300,))
    a.commit()
    t.join()
    print("   최종 잔액 :", balance(a), "(정상)")

print("\n⑤ 데드락 - 서로 상대가 잠근 행을 기다리면 PostgreSQL 이 한쪽을 강제 취소")
setup()
with connect() as a, connect() as b:
    a.execute("UPDATE demo_acc SET balance = balance - 1 WHERE id = 1")   # A: 1번 잠금
    b.execute("UPDATE demo_acc SET balance = balance - 1 WHERE id = 2")   # B: 2번 잠금
    errors = []

    def session_b():
        try:
            b.execute("UPDATE demo_acc SET balance = balance + 1 WHERE id = 1")  # A 를 기다림
            b.commit()
        except psycopg.errors.DeadlockDetected as e:
            errors.append(("B", e.diag.message_primary))
            b.rollback()

    t = threading.Thread(target=session_b)
    t.start()
    time.sleep(0.3)
    try:
        a.execute("UPDATE demo_acc SET balance = balance + 1 WHERE id = 2")      # B 를 기다림 → 교착
        a.commit()
    except psycopg.errors.DeadlockDetected as e:
        errors.append(("A", e.diag.message_primary))
        a.rollback()
    t.join()
    for who, msg in errors:
        print(f"   세션 {who} 취소됨 :", msg)
    print("   → 해결책: 여러 행을 잠글 때는 항상 같은 순서(예: id 오름차순)로 잠근다")

with connect() as c:
    c.execute("DROP TABLE demo_acc")
```

▶ 실행 결과

```
① READ COMMITTED - 다른 세션이 커밋한 값이 바로 보인다
   A 첫 조회 : 1000
   A 재조회 : 500 (같은 트랜잭션인데 값이 바뀜 = Non-repeatable read)

② REPEATABLE READ - 트랜잭션 시작 시점의 스냅샷을 계속 본다
```

```
   A 첫 조회 : 1000
   A 재조회 : 1000 (B 의 변경이 보이지 않음)
   A 가 같은 행 수정 시도 → could not serialize access due to concurrent update

③ 갱신 손실(Lost Update) - 읽고 → 앱에서 계산 → 쓰기
   기대값 500, 실제값 800 → A 의 출금 300 이 사라짐!

④ 해결 - SELECT ... FOR UPDATE 로 먼저 잠그기
   B 대기 중? : True
   최종 잔액 : 500 (정상)

⑤ 데드락 - 서로 상대가 잠근 행을 기다리면 PostgreSQL 이 한쪽을 강제 취소
   세션 B 취소됨 : deadlock detected
   → 해결책: 여러 행을 잠글 때는 항상 같은 순서(예: id 오름차순)로 잠근다
```

■ 참고 **데드락에서 누가 취소되나**

⑤ 에서는 B 가 먼저 A 를 기다리기 시작했으므로, 나중에 교착을 완성한 A 쪽 대기에서 감지될 수도 있고 B 쪽이 취소될 수도 있습니다(실행 타이밍에 따라 다름). 중요한 것은 **한쪽이 취소되고 다른 쪽은 진행된다**는 점과, 취소된 쪽은 재시도해야 한다는 점입니다.

■ Q&A; **자주 묻는 질문**

- **Q. ORM(SQLAlchemy, Django ORM)을 쓰면 SQL 을 몰라도 되나요?** ORM 이 만드는 SQL 을 이해하지 못하면 N+1 쿼리, 불필요한 조인 같은 성능 문제를 해결할 수 없습니다. ORM 을 쓰더라도 이 과정의 SQL 지식이 기반이 됩니다.

정리

- 접속 정보는 환경변수로, 값은 `%s` 파라미터로.
- psycopg 는 commit 해야 반영. 예외 후에는 rollback.
- 집계는 SQL, 분석·시각화는 pandas.

20장 · 종합 프로젝트 - 쇼핑몰 매출 분석 리포트

프로젝트 개요

shop 데이터베이스로 경영진에게 보고할 **매출 분석 리포트**를 만듭니다. 지금까지 배운 JOIN, 집계, CTE, 윈도우 함수, 뷰, 함수를 모두 활용합니다.

예제 파일: `examples/20_project.sql`

요구사항

번호	경영진의 질문	사용 기술
R1	전체 매출, 주문 수, 구매 고객 수, 객단가는?	집계, 뷰
R2	올해 상반기는 작년 같은 달보다 성장했나?	CTE, 셀프 조인
R3	우수 고객과 이탈 위험 고객은 누구인가?	RFM, NTILE
R4	한 번 산 고객이 다시 사는가? 언제까지 남는가?	재구매율, 코호트
R5	어떤 상품이 함께 팔리는가?	셀프 조인 (장바구니 분석)
R6	상품별·영업사원별 성과는?	윈도우 함수 (RANK, 비중)
R7	기간을 넣으면 분류별 매출을 주는 리포트	SQL 함수

1단계 - 분석용 뷰

모든 분석은 "취소 제외 판매 라인"에서 출발합니다. 이것을 뷰로 만들어 두면 이후 쿼리가 짧아지고, 기준(취소 제외)이 모든 리포트에서 일관됩니다.

예제 20-01 · 분석용 기본 뷰 - 취소 제외 판매 라인

```
CREATE VIEW v_sales AS
SELECT o.order_id, o.customer_id, o.emp_id, o.order_date,
       date_trunc('month', o.order_date)::date AS order_month,
       oi.product_id, p.category_id, oi.quantity,
       oi.quantity * oi.unit_price AS amount
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p     ON p.product_id = oi.product_id
WHERE o.status <> 'CANCELLED';
SELECT count(*) AS 판매라인, count(DISTINCT order_id) AS 주문수 FROM v_sales;
```

▶ 실행 결과

```
CREATE VIEW
 판매라인 ¦ 주문수
----------+--------
      207 ¦    105
(1 row)
```

2단계 - KPI 와 성장률

예제 20-02 · 핵심 KPI - 매출, 주문수, 구매고객수, 객단가(AOV)

```
SELECT sum(amount)                                AS 총매출,
       count(DISTINCT order_id)                   AS 주문수,
       count(DISTINCT customer_id)                AS 구매고객수,
       round(sum(amount) / count(DISTINCT order_id)) AS 객단가
FROM v_sales;
```

▶ 실행 결과

```
  총매출   ¦ 주문수 ¦ 구매고객수 ¦ 객단가
----------+--------+------------+--------
 89588700 ¦    105 ¦         18 ¦ 853226
(1 row)
```

예제 20-03 · 전년 동월 대비(YoY) 매출 성장률 - 2026년 상반기

```
WITH m AS (
    SELECT order_month, sum(amount) AS sales FROM v_sales GROUP BY order_month
)
SELECT to_char(cur.order_month, 'YYYY-MM') AS 월,
       prev.sales AS 전년동월, cur.sales AS 당월,
       round(100.0 * (cur.sales - prev.sales) / prev.sales, 1) AS yoy
FROM m cur
LEFT JOIN m prev ON prev.order_month = cur.order_month - interval '1 year'
WHERE cur.order_month >= '2026-01-01'
ORDER BY cur.order_month;
```

▶ 실행 결과

```
   월    ¦ 전년동월 ¦   당월   ¦  yoy
---------+----------+----------+--------
 2026-01 ¦  6014100 ¦  8668300 ¦   44.1
 2026-02 ¦  2499000 ¦  2023000 ¦  -19.0
 2026-03 ¦  6517300 ¦  6504000 ¦   -0.2
 2026-04 ¦  8691000 ¦  6616000 ¦  -23.9
 2026-05 ¦  3055200 ¦  2584100 ¦  -15.4
 2026-06 ¦   865100 ¦ 10031000 ¦ 1059.5
(6 rows)
```

■ 핵심 포인트 **숫자를 해석하기**

쿼리 결과를 보고 끝내지 말고 "6월 YoY 가 1059% 인 이유는?" 하고 스스로 질문해 보세요. 작년 6월 매출이 유난히 작았기 때문입니다. 성장률은 **기준값이 작으면 과장된다**는 점, 그래서 금액과 비율을 함께 보여 줘야 한다는 점을 기억하세요.

3단계 - 고객 분석 (RFM, 재구매율, 코호트)

지표	의미	점수 기준 (여기서는 3분위)
R (Recency)	마지막 구매 후 경과일	최근일수록 높은 점수
F (Frequency)	구매 횟수	많을수록 높은 점수
M (Monetary)	구매 금액	클수록 높은 점수

예제 20-04 · RFM 분석 - 최근성(Recency), 빈도(Frequency), 금액(Monetary)

```
WITH base AS (
    SELECT customer_id,
           date '2026-07-01' - max(order_date)::date AS recency_days,
           count(DISTINCT order_id)                  AS frequency,
           sum(amount)                               AS monetary
    FROM v_sales GROUP BY customer_id
), scored AS (
    SELECT *,
           NTILE(3) OVER (ORDER BY recency_days DESC) AS r,   -- 최근일수록 3점
           NTILE(3) OVER (ORDER BY frequency)         AS f,
           NTILE(3) OVER (ORDER BY monetary)          AS m
    FROM base
)
SELECT c.name, s.recency_days, s.frequency, s.monetary,
       concat(s.r, s.f, s.m) AS rfm,
       CASE WHEN s.r + s.f + s.m >= 8 THEN '최우수'
            WHEN s.r = 1              THEN '이탈위험'
            ELSE '일반' END AS 세그먼트
FROM scored s JOIN customers c USING (customer_id)
ORDER BY s.r + s.f + s.m DESC, s.monetary DESC
LIMIT 8;
```

▶ 실행 결과

```
  name  ┆ recency_days ┆ frequency ┆ monetary ┆ rfm ┆ 세그먼트
--------+--------------+-----------+----------+-----+----------
 정하은 ┆            8 ┆        10 ┆  9001100 ┆ 333 ┆ 최우수
 윤지호 ┆            7 ┆        14 ┆  8927000 ┆ 333 ┆ 최우수
 김민준 ┆            6 ┆        12 ┆  7692400 ┆ 333 ┆ 최우수
 임준서 ┆           17 ┆        11 ┆ 20481300 ┆ 233 ┆ 최우수
 강시우 ┆            9 ┆         4 ┆  7607000 ┆ 323 ┆ 최우수
 박도윤 ┆           13 ┆        12 ┆  3260300 ┆ 332 ┆ 최우수
 황민재 ┆          166 ┆         7 ┆  4111800 ┆ 232 ┆ 일반
 오건우 ┆            8 ┆         5 ┆  3064000 ┆ 322 ┆ 일반
(8 rows)
```

예제 20-05 · 재구매율 - 2회 이상 구매한 고객 비율

```
WITH cnt AS (
    SELECT customer_id, count(DISTINCT order_id) AS n FROM v_sales GROUP BY customer_id
)
SELECT count(*) AS 구매고객,
       count(*) FILTER (WHERE n >= 2) AS 재구매고객,
       round(100.0 * count(*) FILTER (WHERE n >= 2) / count(*), 1) AS 재구매율
FROM cnt;
```

▶ 실행 결과

```
 구매고객 ┆ 재구매고객 ┆ 재구매율
----------+------------+----------
       18 ┆         17 ┆     94.4
(1 row)
```

예제 20-06 · 분기 코호트 - 첫 구매 분기별로 이후 분기에 다시 산 고객 수

```
WITH first AS (
    SELECT customer_id, date_trunc('quarter', min(order_date))::date AS cohort
    FROM v_sales GROUP BY customer_id
), activity AS (
    SELECT DISTINCT v.customer_id, f.cohort,
           (EXTRACT(YEAR FROM age(date_trunc('quarter', v.order_date), f.cohort)) * 4
            + EXTRACT(MONTH FROM age(date_trunc('quarter', v.order_date), f.cohort)) / 3)::int AS q
    FROM v_sales v JOIN first f USING (customer_id)
)
SELECT to_char(cohort, 'YYYY-"Q"Q') AS 코호트,
       count(*) FILTER (WHERE q = 0) AS q0,
       count(*) FILTER (WHERE q = 1) AS q1,
       count(*) FILTER (WHERE q = 2) AS q2,
       count(*) FILTER (WHERE q = 3) AS q3,
       count(*) FILTER (WHERE q = 4) AS q4
FROM activity
GROUP BY cohort
ORDER BY cohort;
```

▶ 실행 결과

```
 코호트   ┆ q0 ┆ q1 ┆ q2 ┆ q3 ┆ q4
---------+----+----+----+----+----
 2025-Q1 ┆ 11 ┆  8 ┆  7 ┆  9 ┆  6
 2025-Q2 ┆  2 ┆  0 ┆  1 ┆  1 ┆  0
 2025-Q3 ┆  3 ┆  1 ┆  1 ┆  1 ┆  0
 2025-Q4 ┆  1 ┆  1 ┆  1 ┆  0 ┆  0
 2026-Q2 ┆  1 ┆  0 ┆  0 ┆  0 ┆  0
(5 rows)
```

■ 참고 **코호트 표 읽기**

20-06 의 각 행은 "같은 분기에 처음 구매한 고객 집단", q1~q4 는 그로부터 1~4분기 뒤에도 구매한 고객 수입니다. 2025-Q1 코호트 11명 중 1분기 뒤 8명, 4분기 뒤 6명이 다시 구매했습니다. 최근 코호트는 아직 시간이 지나지 않아 뒤쪽 칸이 0 인 것이 정상입니다.

4단계 - 상품과 영업 성과

예제 20-07 · 장바구니 분석 - 함께 많이 팔린 상품 쌍

```
SELECT pa.name AS 상품1, pb.name AS 상품2, count(*) AS 함께주문
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id AND a.product_id < b.product_id
JOIN products pa ON pa.product_id = a.product_id
JOIN products pb ON pb.product_id = b.product_id
GROUP BY pa.name, pb.name
HAVING count(*) >= 3
ORDER BY 함께주문 DESC, 상품1
LIMIT 5;
```

▶ 실행 결과

```
     상품1     ┆     상품2      ┆ 함께주문
---------------+----------------+----------
 견과류 믹스   ┆ 콜드브루 원액  ┆        7
 무선 마우스   ┆ 콜드브루 원액  ┆        5
 콜드브루 원액 ┆ 노트북 거치대  ┆        5
 기계식 키보드 ┆ 노트북 거치대  ┆        4
 남성 셔츠     ┆ 수제 쿠키 세트 ┆        4
(5 rows)
```

예제 20-08 · 상품 성과표 - 매출 순위 + 평균 평점 + 분류 내 비중

```
WITH sales AS (
    SELECT product_id, category_id, sum(amount) AS sales, sum(quantity) AS qty
    FROM v_sales GROUP BY product_id, category_id
), rating AS (
    SELECT product_id, round(avg(rating), 2) AS avg_rating, count(*) AS reviews
    FROM reviews GROUP BY product_id
)
SELECT RANK() OVER (ORDER BY s.sales DESC) AS 순위,
       p.name, s.sales AS 매출, s.qty AS 수량,
       COALESCE(r.avg_rating, 0) AS 평점, COALESCE(r.reviews, 0) AS 리뷰수,
       round(100.0 * s.sales / sum(s.sales) OVER (PARTITION BY s.category_id), 1) AS 분류내비중
FROM sales s
JOIN products p USING (product_id)
LEFT JOIN rating r USING (product_id)
ORDER BY 순위
LIMIT 8;
```

▶ 실행 결과

```
 순위 ┆        name        ┆   매출   ┆ 수량 ┆ 평점 ┆ 리뷰수 ┆ 분류내비중
------+--------------------+----------+------+------+--------+------------
    1 ┆ 울트라북 14        ┆ 21801000 ┆   17 ┆ 3.33 ┆      3 ┆       56.9
    2 ┆ 스마트폰 S25       ┆ 19320000 ┆   17 ┆ 3.00 ┆      1 ┆       55.6
    3 ┆ 비즈니스 노트북 15 ┆ 16533000 ┆   17 ┆ 3.25 ┆      4 ┆       43.1
    4 ┆ 스마트폰 미니      ┆ 15456000 ┆   23 ┆ 4.25 ┆      4 ┆       44.4
    5 ┆ 27인치 모니터      ┆  4244100 ┆   13 ┆ 4.00 ┆      2 ┆       50.2
    6 ┆ 게이밍 노트북 X17  ┆  2490000 ┆    1 ┆    0 ┆      0 ┆      100.0
    7 ┆ 기계식 키보드      ┆  2269500 ┆   26 ┆ 4.33 ┆      3 ┆       26.9
    8 ┆ 여성 원피스        ┆  1082300 ┆   14 ┆ 5.00 ┆      2 ┆       63.1
(8 rows)
```

예제 20-09 · 영업사원 실적 - 담당 매출과 팀 내 순위

```
SELECT e.name, e.job_title,
       count(DISTINCT v.order_id) AS 담당주문,
       sum(v.amount) AS 담당매출,
       RANK() OVER (ORDER BY sum(v.amount) DESC) AS 순위
FROM v_sales v JOIN employees e ON e.emp_id = v.emp_id
GROUP BY e.emp_id, e.name, e.job_title
ORDER BY 순위;
```

▶ 실행 결과

```
  name  ┆ job_title ┆ 담당주문 ┆ 담당매출 ┆ 순위
--------+-----------+----------+----------+------
 신동현 ┆ 영업사원  ┆       16 ┆ 26821100 ┆    1
 강민수 ┆ 영업대리  ┆       16 ┆ 24067100 ┆    2
 윤서준 ┆ 영업사원  ┆       23 ┆ 11358200 ┆    3
 조은지 ┆ 영업과장  ┆        9 ┆  4056000 ┆    4
(4 rows)
```

5단계 - 재사용 가능한 리포트 함수

예제 20-10 · 리포트를 함수로 - 기간을 넣으면 분류별 매출을 돌려준다

```
CREATE OR REPLACE FUNCTION sales_report(p_from DATE, p_to DATE)
RETURNS TABLE (category TEXT, orders BIGINT, sales NUMERIC, share NUMERIC)
LANGUAGE sql STABLE
AS $$
    SELECT c.name::text, count(DISTINCT v.order_id), sum(v.amount),
           round(100.0 * sum(v.amount) / sum(sum(v.amount)) OVER (), 1)
    FROM v_sales v JOIN categories c USING (category_id)
    WHERE v.order_date >= p_from AND v.order_date < p_to
    GROUP BY c.name
    ORDER BY 3 DESC;
$$;
SELECT * FROM sales_report('2026-04-01', '2026-07-01');
```

▶ 실행 결과

```
CREATE FUNCTION
 category ┆ orders ┆  sales   ┆ share
──────────┼────────┼──────────┼───────
 스마트폰 ┆      3 ┆ 11270000 ┆  58.6
 노트북   ┆      2 ┆  5550000 ┆  28.9
 주변기기 ┆      5 ┆  1101000 ┆   5.7
 여성의류 ┆      6 ┆   512000 ┆   2.7
 남성의류 ┆      5 ┆   354300 ┆   1.8
 음료     ┆      9 ┆   306000 ┆   1.6
 간식     ┆      5 ┆   137800 ┆   0.7
(7 rows)
```

조별 과제

아래 중 하나를 골라 쿼리를 작성하고, 결과를 표·차트(19장 pandas 활용 가능)로 정리해 5분간 발표합니다.

1. **등급 개편안**: 현재 grade 와 실제 구매액(15장 `customer_level` 함수)이 맞지 않는 고객을 찾고, 새 등급 기준을 제안하세요.
2. **재고 위험 보고서**: 최근 3개월 판매 속도로 볼 때 30일 안에 재고가 바닥날 상품을 찾으세요.
3. **도시별 마케팅 전략**: 도시별 고객 수, 객단가, 재구매율, 인기 분류를 한 표로 만드세요.
4. **리뷰 분석**: 평점 3점 이하 리뷰가 많은 상품과 그 상품의 매출 추이를 비교하세요.
5. **대시보드 설계**: 위 리포트들을 머티리얼라이즈드 뷰로 만들고 갱신 전략을 설명하세요.

평가 항목	배점
쿼리 정확성 (결과 검증 방법 포함)	40
쿼리 가독성 (CTE 단계 분리, 별칭, 주석)	20
결과 해석과 제안	30
발표	10

실습 문제

번호	문제
20-Q1	2025년 하반기에 구매했지만 2026년 상반기에 구매하지 않은 '이탈 고객' 명단을 만드세요 (취소 제외)
20-Q2	분류별 매출 1위 고객(가장 많이 산 고객)을 구하세요 (최상위 분류 기준: 전자제품/의류/식품)

문제지는 저장소의 `exercises/`, 정답은 `solutions/` 폴더에 있습니다. 먼저 직접 풀어 본 뒤 확인하세요.

과정 총정리

단계	핵심
기초 (1~6장)	테이블 설계, 데이터 입력·수정, SELECT/WHERE/ORDER BY, 함수, GROUP BY
중급 (7~12장)	JOIN, 서브쿼리, CTE·재귀, 윈도우 함수, 정규화, 뷰
심화 (13~19장)	인덱스·실행 계획, 트랜잭션·동시성, PL/pgSQL, JSONB, 권한·RLS, 운영, Python

TIP 수료 후 학습 로드맵

- 공식 문서(postgresql.org/docs) 의 Tutorial 과 SQL Commands 장
- SQL 연습: 프로그래머스 SQL 고득점 Kit, LeetCode Database, HackerRank SQL
- 자격증: SQLD(국가공인 SQL 개발자), 이후 SQLP
- 심화: 실행 계획 튜닝, 파티셔닝, 복제(Replication), 커넥션 풀(PgBouncer), 클라우드 관리형 DB

부록 · 빠른 참조

SQL 문법 요약

분류	문법
테이블 생성	`CREATE TABLE t (id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, ...)`
컬럼 추가	`ALTER TABLE t ADD COLUMN c TYPE`
입력	`INSERT INTO t (a, b) VALUES (1, 2) RETURNING *`
UPSERT	`INSERT ... ON CONFLICT (키) DO UPDATE SET c = EXCLUDED.c`
수정 / 삭제	`UPDATE t SET c = v WHERE ...` / `DELETE FROM t WHERE ...`
조회	`SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT n`
조인	`FROM a JOIN b ON a.k = b.k LEFT JOIN c ON ...`
CTE	`WITH x AS (SELECT ...) SELECT ... FROM x`
재귀	`WITH RECURSIVE x AS (앵커 UNION ALL 재귀부) ...`
윈도우	`함수() OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`
인덱스	`CREATE INDEX 이름 ON t (a, b)` / `USING GIN (jsonb컬럼)`
실행 계획	`EXPLAIN (ANALYZE, BUFFERS) 쿼리`
트랜잭션	`BEGIN; ... COMMIT;` / `ROLLBACK;` / `SAVEPOINT s; ROLLBACK TO s;`
권한	`GRANT SELECT ON t TO 롤;` / `REVOKE ...`

자주 만나는 오류 메시지

메시지	원인	해결
`relation "xxx" does not exist`	테이블 이름 오타, 다른 DB/스키마에 접속, 대소문자(따옴표)	`\c shop`, `\dt`, search_path 확인
`column "xxx" does not exist`	컬럼 오타, 문자열을 큰따옴표로 씀(`"서울"`)	문자열 값은 작은따옴표
`syntax error at or near "xxx"`	쉼표·괄호 누락, 예약어 사용	표시된 위치 바로 앞을 확인
`must appear in the GROUP BY clause`	GROUP BY 규칙 위반 (6장)	GROUP BY 에 추가 또는 집계 함수
`duplicate key value violates unique constraint`	PK/UNIQUE 중복	값 확인, UPSERT 고려
`violates foreign key constraint`	없는 부모 참조 / 자식이 있는 부모 삭제	부모 먼저 입력 / 자식 먼저 삭제
`current transaction is aborted`	트랜잭션 안에서 앞서 오류 발생 (14장)	`ROLLBACK;`
`operator does not exist: integer || integer` 등	자료형 불일치	`::text` 등 형변환
`division by zero`	0 으로 나눔	`NULLIF(분모, 0)`

메시지	원인	해결
`permission denied for table`	권한 없음 (17장)	GRANT, 스키마 USAGE 확인
`password authentication failed`	비밀번호 오류	실습 서버 비밀번호는 `postgres` (`docker-compose.yml`)
`Connection refused` / `could not connect to server`	서버(컨테이너) 중지, 포트 오류	Docker Desktop 실행, `docker compose ps` 로 상태 확인, 포트 확인 (1장)

psql 메타 명령

실습 서버의 psql 은 `docker exec -it pg-shop psql -U postgres -d shop` 으로 실행합니다. DBeaver 에서 같은 일을 하는 방법은 1장의 대응표를 참고하세요.

명령	의미	명령	의미
`\l`	DB 목록	`\c db`	DB 전환
`\dt [패턴]`	테이블	`\d t`	테이블 구조
`\dv` / `\dm`	뷰 / 머티리얼라이즈드 뷰	`\di`	인덱스
`\dn`	스키마	`\df`	함수
`\du`	롤	`\dx`	설치된 확장
`\x`	세로 출력 전환	`\timing`	실행 시간 표시
`\i 파일`	파일 실행	`\o 파일`	출력을 파일로
`\copy`	CSV 입출력 (18장)	`\e`	편집기
`\conninfo`	접속 정보	`\q`	종료